Book II — BLua32 and BIOS Reference

BMSX

Table of contents

About these manuals **1**

1 CPU and BLua32 instruction reference **3**
1.1 Programming model . . . 3
1.2 Worked computation: Numbers, words and stored bytes . . . 5
1.3 Instruction words and execution resources . . . 6
1.4 BLua operators and instruction costs . . . 9
1.5 Values and numeric instructions . . . 10
1.6 Numeric result details . . . 14
1.7 Calls, closures, globals, and tables . . . 18
1.8 Worked values: signed zero and inherited fields . . . 24
1.9 Memory transfers . . . 25
1.10 Branches . . . 27
1.11 Cycle accounting and execution boundaries . . . 28
1.12 CPU reset and program replacement . . . 29
1.13 CP0 and execution control . . . 29
1.14 Memory faults and saved PC . . . 32

2 BLua source programs and extensions **33**
2.1 BLua extensions . . . 33
2.2 Typed storage, layouts, and address views . . . 34
2.3 Worked static storage . . . 34
2.4 Declaration and layout reference . . . 37
2.5 Operator spellings and source conventions . . . 41
2.6 Module modes, static imports, and `<init>` . . . 42
2.7 Named BIOS function imports . . . 45
2.8 Direct access forms and intrinsics . . . 45
2.9 BLua source semantics . . . 49
2.10 Global names and source resolution . . . 53

3 Program images and startup **55**
3.1 Addresses used to enter a program . . . 55
3.2 ROM contents, RAM sections, and startup . . . 55
3.3 Reset and cartridge startup . . . 57
3.4 ROM header fields . . . 58
3.5 BLua32 executable image . . . 59
3.6 Locating BIOS call targets . . . 61

4 Protected calls and coroutines **63**
4.1 Control flow and hardware progress . . . 63
4.2 Protected calls . . . 63
4.3 Coroutines . . . 65
4.4 Language fault reasons . . . 69
4.5 Reset and startup boundary . . . 69
4.6 Programming examples . . . 69

5 BIOS base services **71**
5.1 Installed program environment . . . 71
5.2 Value checks and conversion . . . 72
5.3 Table and metatable access . . . 76
5.4 Traversal, argument selection, globals, and output . . . 76
5.5 Programming examples . . . 78

6 BIOS runtime compilation **81**
6.1 Programming runtime compilation . . . 81
6.2 Runtime compilation interface . . . 84
6.3 Runtime-loaded source language . . . 86
6.4 Constructing and compiling syntax trees . . . 90
6.5 Code-arena capacity and allocation . . . 97

7 BIOS diagnostics **99**
7.1 Diagnostic text and the BIOS console . . . 99
7.2 Memory faults and the monitor . . . 100
7.3 Optional BIOS diagnostic directory . . . 103

8 BIOS math library **107**
8.1 Calling the library . . . 107
8.2 Numeric notation . . . 107
8.3 Constants . . . 107
8.4 `math.abs(value)` . . . 108
8.5 `math.floor(value)` . . . 108
8.6 `math.ceil(value)` . . . 108
8.7 `math.round(value)` . . . 108
8.8 `math.fmod(left, right)` . . . 108
8.9 `math.modf(value)` . . . 108
8.10 `math.min(first, ...)` . . . 109
8.11 `math.max(first, ...)` . . . 109
8.12 `math.tointeger(value)` . . . 109
8.13 `math.type(value)` . . . 109
8.14 `math.ult(left, right)` . . . 109
8.15 `math.sign(value)` . . . 109
8.16 Execution cost and library bindings . . . 110
8.17 Angles and transcendental operations . . . 110
8.18 Random numbers . . . 114
8.19 Programming examples . . . 116

9 BIOS time and calendar services **117**
9.1 Example: time and calendar operations . . . 117
9.2 `os.clock()` — sampled time in seconds . . . 118
9.3 `os.time([date_table])` — whole seconds or civil conversion . . . 118
9.4 `os.difftime(t2, t1)` — subtract whole-second values . . . 120
9.5 Shared civil-day calculations . . . 121
9.6 `os.date` . . . 122

10 BIOS String operations **127**
10.1 Text values and encoded byte storage . . . 127
10.2 Example: codepoints and mapped UTF-8 . . . 128
10.3 Codepoint access, construction, length and UTF-8 input . . . 128
10.4 Case conversion, slicing, reversal and repetition . . . 130
10.5 `string.format` . . . 133
10.6 Pattern language . . . 136
10.7 Pattern searches and substitutions . . . 140
10.8 Floating-point word decoder . . . 142
10.9 String packet packing . . . 143

11 BIOS table library **151**
11.1 Tables, lengths and visible effects . . . 151
11.2 Packing and reading table ranges . . . 152
11.3 `table.insert` . . . 153
11.4 `table.remove` . . . 154
11.5 `table.sort` . . . 155
11.6 Errors and resource limits . . . 156
11.7 Programming examples . . . 157

About these manuals

The BMSX Developer Reference describes the console, its programming architecture and the interfaces available to cartridge software. The three books separate installed hardware, processor and firmware programming, and cartridge devices.

Book I — System and Hardware Reference introduces the machine and its memory system, drawing and display units, audio, direct memory access, interrupts and peripheral control. Start with the system overview and memory map. The interrupt and DMA chapters explain the shared service and transfer mechanisms used by the individual units. Each unit chapter then gives its operating explanation, programming sequences and complete technical reference.

Book II — BLua32 and BIOS Reference describes the processor, instruction set, BLua language extensions and resident BIOS services. It supplies the program and data representations needed to implement cartridge software. The BLua chapter explains the source-language extensions; the CPU chapter is the instruction reference. Program images and startup have their own chapter, separate from physical memory mapping. BIOS diagnostics describes console output, source and frame inspection, and the optional records that support them.

For BLua source programming, begin with the language-extensions chapter and use the CPU chapter for the underlying operations and timing. For instruction encoding or independent code generation, begin with the CPU and program-image references. The startup chapter explains the state established before application entry; the BIOS chapters specify callable services available to that application.

Book III — Cartridge and Expansion Reference describes cartridge sockets and mapped card devices, including their memory and interrupt interfaces.

Addresses are byte addresses unless stated otherwise. Register descriptions give bit positions within the named word; binary layouts give byte offsets within the described structure. Capacities, units and synchronization conditions are specified with the interface to which they apply.

1 CPU and BLua32 instruction reference

The BLua32 processor executes the cartridge program, performs arithmetic and controls the console's memory-mapped units. Its instruction set operates on function-frame value registers and transfers numeric scalars to and from byte-addressed storage. Function calls, closures and language error handling are part of the processor's execution architecture.

This chapter defines instruction encoding, operand values, execution timing and CP0 control. Book I, "Memory and bus access" defines the address map; Program images and startup defines program records; Book I, "Interrupt controller" defines interrupt-source service; Book I, "System control and timing" defines supervisor device contexts. Device commands and their result formats are defined in the corresponding Book I chapters.

1.1 Programming model

Function-local registers hold values used by arithmetic, branches and calls. Byte-addressed storage holds instructions, static data, transfer buffers and device registers. Scalar memory instructions connect these paths: a load produces a Number from addressed bytes, and a store writes the selected numeric representation.

Operation	Source	Result
Arithmetic and comparison	Frame registers or image constants	A language value or a branch/skip decision
Closure call	A closure and argument values	A new call frame; returned values go to the caller's result slots
Typed memory access	Numeric byte address and scalar format	A numeric load or a byte/word write through the memory map
Device programming	Numeric register address and command/operand words	Work admitted by the addressed unit under its own rules
Exception entry	Accepted interrupt or synchronous CPU exception	Saved CP0 state and an exception frame

Register R0 belongs to the current function frame; R0 in a called function is a different slot. An ordinary call retains the caller's continuation and uses the callee's function record to establish its entry PC and register capacity. A return places results in the caller's requested result range. Captured locals remain connected to their closures; the closure rules below describe their lifetime.

To control a device, prepare its addressed payload and operands, then write its command port. The receiving unit supplies a status or interrupt condition for its subsequent completion. Wait for that condition before reusing live input or output storage. Book I, "Memory and bus access", under "Device-specific buffer lifetimes" describes these lifetimes across DMA, geometry, drawing and sound.

1.1.1 Values are not register words

A CPU register holds a language value, not an unconditionally interpreted 32-bit word. The value kinds are nil, false, true, Number, String, Table, Closure, BuiltinFunction, and Thread. Copying a register or loading a constant preserves its value kind: copying false does not turn it into zero, and copying a Table preserves the reference to that Table.

An ordinary Number is a double-precision floating-point value. It can carry fractional values, signed zero, infinities, and NaN. Arithmetic does not implicitly wrap it to 32 bits or give it a fixed-point interpretation. A raw 32-bit word, by contrast, is a pattern of 32 bits. The bitwise and shift instructions perform a word conversion at their inputs and return the signed 32-bit interpretation as an ordinary Number; they do not change how other Numbers are stored or calculated.

Numeric operand reads are not general value conversions. A Number supplies its numeric value; any other value kind supplies NaN. In particular, a numeric string is not parsed, and a boolean is not converted to zero or one. Numeric results remain Numbers, including results that are NaN.

This distinction matters when choosing operations in BLua. Use ordinary arithmetic when fractions and non-wrapping numeric results are wanted. Use bitwise operations when the calculation concerns flags, masks, or particular word bits. Use the truth rules for conditions rather than assuming that a numeric zero is false.

1.1.2 Values and register notation

Register operands select logical value slots in the current call frame. A slot is not a byte address or a four-byte data word.

The instruction descriptions use the following notation:

Notation	Meaning
`R(i)` or `R[i]`	Logical register slot `i` in the current frame.
`K(i)` or `K[i]`	Constant `i` in the execution image.
`RK(x)`	`R(x)` when the decoded operand `x` is nonnegative; otherwise `K(-1-x)`. Thus `RK(-1)` selects `K(0)`.
`top`	The exclusive upper limit of the frame's active register range. An open value range beginning at `A` contains `max(top-A, 0)` values.
`index(base, key)`	A language-level indexed lookup or assignment, not a memory access at a byte offset.

Unless a dedicated rule below says otherwise, writing a register raises `top` to at least one past that register and does not lower it. Writing a table element, global, or upvalue does not itself extend the register range.

1.1.3 From stored bytes to a frame value

Consider four data bytes at consecutive addresses:

Byte address	`p`	`p+1`	`p+2`	`p+3`
Stored byte	`78H`	`56H`	`34H`	`12H`

Let R1 hold the numeric address `p`, four-byte aligned in readable data storage, and reserve R0 through R2 in the current frame. The following two instructions read from the same starting address:

Instruction	Meaning of its operands	Result
`LOAD_MEM 0,1,2`	Destination R0; address from R1; format 2, unsigned little-endian halfword	R0 receives Number 22136 (`5678H`)
`LOAD_MEM 2,1,3`	Destination R2; address from R1; format 3, unsigned little-endian word	R2 receives Number 305419896 (`12345678H`)

The address selects the bytes; the format selects how many are read and how they become a Number. R0 and R2 are both value slots, not permanently typed halfword and word registers. Neither load changes R1 or the stored bytes. Likewise, a scalar store converts a numeric value to its selected storage representation; it does not write the complete language value into memory.

These are the same bytes used by `halves[0]` and `*whole` in the worked computation. Those BLua pointer types select the memory operation; they do not give the destination value slot a different numeric kind. The instruction encoding below carries the destination, address selection and format as operands A, B and C. The memory reference defines all formats, alignment rules and fault effects.

1.1.4 Following a closure call

A call changes both the current instruction stream and the current register frame. Follow a call to the function whose record is at `10001230H`, using R3 for its closure:

Step	Caller state	Execution effect
Construct the function's closure in R3	R3 holds the callable value	Execution continues in the caller; the callee has not started
Execute `CALL 3,1,1`	R3 supplies that closure; B=1 supplies no arguments; C=1 requests one result	A new frame starts at the code address in the function record
Return normally from that callee	Its first result replaces the caller's R3, or nil is supplied if it produced none	The caller resumes at the instruction following the CALL

The callee uses its own R0 and subsequent slots. Its function-record address identifies the entry metadata, while its PC identifies the instructions being executed. The caller retains its next PC during the call; that continuation is not an argument or an ordinary returned value. Constructing the closure, entering the function and receiving its results are therefore separate steps. The call reference defines argument placement, variable result counts and other return routes. The encoding example below turns this same R3 call sequence into instruction bytes.

1.2 Worked computation: Numbers, words and stored bytes

Start after normal BIOS entry, which has initialized the cartridge's DATA and BSS. The following operations require no device setup. The storage declarations provide assigned byte addresses; they are not CPU register allocations.

```
module<entry>
rodata operands: f64[3] = { -7, 3, 2147483648 }
rodata bytes: u8[4] = { 0x78, 0x56, 0x34, 0x12 }
bss word_result: word
bss rounded: f32

local numbers<const>: *f64 = operands
local halves<const>: *u16 = bytes
local whole<const>: *u32 = bytes
local quotient = numbers[0] / numbers[1]
local floor_quotient = numbers[0] // numbers[1]
*word_result = numbers[2]
*rounded = numbers[2] + 1
```

Expression or stored result	Value	Reason
`quotient`	−7/3, approximately −2.3333333333333335	Number division retains a fraction.
`floor_quotient`	−3	Floor division rounds downward.
`numbers[0] % numbers[1]`	2	A nonzero remainder follows the divisor's sign.
`-numbers[0] % -numbers[1]`	−2	Reversing the signs changes the remainder's sign.
`*word_result`	`80000000H`, read as 2147483648	A word store retains raw low 32 bits.
`numbers[2] | 0`	−2147483648	Bitwise output uses signed-word interpretation.
`numbers[2] >> 1`	−1073741824	Right shift preserves the sign bit.
`1 << 32, 1 << -1`	1, −2147483648	Shift counts retain their low five bits.
`-1.75 | 0`	−1	Word conversion truncates toward zero.
`halves[0], halves[1]`	`5678H`, `1234H`	Halfword reads are little-endian.
`*whole`	`12345678H`	The same four bytes form one word.
`*rounded`	2147483648	The f32 store rounds 2147483649.

Arithmetic Number results do not wrap at the signed-word boundary. A stored word, a signed bitwise result and an f32 value are different representations. Finite printed Numbers are rounded text, not an exact substitute for those stored values. See the Number-to-text rules below.

1.3 Instruction words and execution resources

Resource	Specification
System-cycle timebase	33,868,800 cycles/s
Installed main RAM	4 MiB; mapped windows, reservations and ownership are in the Memory chapter
Language-allocation budget	3.5 MiB of shared accounting capacity, not an additional raw RAM region or wholly free register storage
Opcode field	6 bits, 64 opcode identities including the WIDE prefix
Instruction word	32 bits, four bytes, big-endian code bytes
Instruction length	One word; two words with one WIDE prefix
Numeric value	Double-precision Number, distinct from other language values
Data word and CP0 word	32 bits; scalar data transfers are little-endian
Frame registers	Logical value slots relative to the current call frame; the function record supplies its parameter count and maximum stack allocation
Register/count encoding	Format-dependent limits below, not a fixed physical register allocation or unlimited recursion guarantee

1.3.1 Word fields

Code bytes use the opposite byte order from multi-byte scalar data. Do not byte-swap one representation merely because the other is little-endian.

Word bits	`31:24`	`23:18`	`17:12`	`11:6`	`5:0`
Field	`EXT`	Opcode	Low A	Low B	Low C

The opcode chooses how the remaining bits are interpreted:

Format	Unprefixed operands	WIDE operands	EXT use
ABC	A: 8 bits; raw B/C: 9 bits each	A: 14 bits; raw B/C: 15 bits each	EXT[7:6] extends A; EXT[5:3] extends B; EXT[2:0] extends C
Bx	A: 6 bits; Bx: 20 bits	A: 12 bits; Bx: 26 bits	EXT is Bx’s bits 19:12; low B/C supply bits 11:0
Displacement	A/B/C: 6 bits each	A/B/C: 12 bits each	Unsigned 8-bit word displacement, unchanged by WIDE

The Bx family comprises LOADK, KSMI, the three branches, CLOSURE and the four global-name instructions. Memory forms ending in `_D` use displacement fields; other instruction bodies use ABC. Each instruction table identifies which operands are registers, counts, immediate values, constant indices or RK selectors. Apply that row’s interpretation after decoding the fields.

1.3.2 WIDE — opcode 00H

A WIDE word contributes its six-bit A/B/C fields above the body fields. For ABC, its A bits begin at decoded A bit 8, and its B/C bits begin at decoded B/C bit 9. For displacement, all three prefix fields begin at bit 6. For Bx, prefix A begins at A bit 6 and prefix B supplies Bx bits 25:20. Prefix C does not contribute to Bx. The prefix's own EXT byte is ignored.

Exactly one following word is decoded as the body. This is one eight-byte instruction, not a separately executed zero-cycle prefix followed by another instruction. If that body is itself WIDE and reaches execution, the CPU hard-halts; prefixes do not form an arbitrarily long chain. WIDE adds no separate base charge to its body's listed cost.

For `CLOSURE`, prefix C bit 5 sets the register-address flag, decoded as C bit 14. With that bit clear, Bx times 16 supplies the function-record address; with it set, Bx selects the register supplying that address. This flag does not add bits to the Bx index. The closure section supplies capture and fault behavior.

1.3.3 Constructing instruction bytes

To assemble a body word, put its six-bit fields `a`, `b`, `c` and extension byte `E` into:

```
word = (E << 24) | (opcode << 18) | (a << 12) | (b << 6) | c
```

Write that word most-significant byte first. The following table splits the complete operands into those fields and the six-bit prefix fields `pA`, `pB`, `pC`. `B` and `C` here mean raw fields, before any RK sign interpretation.

Format	Body fields	Body `E`	WIDE prefix fields

ABC	`a=A&63`, `b=B&63`, `c=C&63`	`(((A>>6)&3)<<6) |`
		`(((B>>6)&7)<<3) |`
		`((C>>6)&7)`
			`pA=A>>8`, `pB=B>>9`, `pC=C>>9`

Bx	`a=A&63`, `b=(U>>6)&63`, `c=U&63`	`(U>>12)&255`	`pA=A>>6`, `pB=U>>20`, `pC=0`, except the CLOSURE register-address flag
Displacement	`a=A&63`, `b=B&63`, `c=C&63`	`disp`	`pA=A>>6`, `pB=B>>6`, `pC=C>>6`

For an unsigned Bx operand, `U` is Bx itself. For a signed Bx operand, `U` is its two's-complement pattern in **20 bits without WIDE or 26 bits with WIDE**. Similarly, encode a negative RK operand in 9 or 15 bits before splitting its raw B/C field. Merely adding a zero-filled prefix to a narrow negative operand does not preserve its sign. A prefix word uses opcode zero, its `a/b/c` fields are `pA/pB/pC`, and its ignored extension byte can be zero.

The two loads above use ABC with opcode `31H`. Their operands fit the low fields, so EXT is zero and no WIDE prefix is needed:

Instruction	A	B	C	Instruction word	Code bytes, in address order
`LOAD_MEM 0,1,2`	0	1	2	`00C40042H`	`00 C4 00 42`
`LOAD_MEM 2,1,3`	2	1	3	`00C42043H`	`00 C4 20 43`

The instruction bytes describe which operation to perform. The data bytes `78 56 34 12` remain at `p` in their little-endian data order. Encoding or fetching either instruction does not change that stored representation.

1.3.4 Linkage operands and encoding examples

A known function target still uses `CLOSURE` followed by `CALL`; `CALL` has no function-address immediate. The immediate CLOSURE relocation is the absolute **function-record byte address divided by 16**, not a code address, function ordinal, image-relative offset or constant index. In its register-address form, encode the register index as Bx and set prefix `pC=32`; the register supplies the full byte address without division by 16. Both forms produce a closure for the same `CALL` mechanism. CP0 `EXEC`, by contrast, consumes a full byte address and replaces execution, as specified below.

The immediate form represents sixteen-byte-aligned targets: up to `00FFFFF0H` without WIDE, or `3FFFFFF0H` with WIDE. A cartridge-ROM target therefore needs WIDE even when its destination register is small. The register-address form does not discard the address's low four bits; record access and execution still follow their own mapped-memory rules.

Constant operands and global-name operands retain their zero-based image-table indices when their table order is unchanged; they are not absolute addresses. An address stored **as a Number constant's payload** is still an address and must be distinguished from the instruction's index selecting that constant. Program images and startup defines the referenced records and their address fields.

The first three rows below encode the R3 closure and call introduced in Following a closure call. All numbers in the table are hexadecimal except register numbers and argument/result counts. Spaces separate actual code bytes:

Operation	Code bytes	Decoded operands
Construct the closure for the record at `10001230H` in R3	`00 00 04 00 00 AC 31 23`	`pB=10H`, Bx=`01000123H`; Bx×16=`10001230H`
Construct a closure in R3 from the byte address held in R5	`00 00 00 20 00 AC 30 05`	Register-address flag set; Bx=5
Call the closure in R3 with no arguments and one requested result	`00 BC 30 41`	`CALL A=3, B=1, C=1`
Jump back to the branch's own first byte, without WIDE	`FF A0 0F FF`	Signed Bx=−1; one word from instruction end to start
Jump back to the prefix's own first byte, with WIDE	`00 00 0F C0 FF A0 0F FE`	Signed Bx=−2; two words from instruction end to prefix

For branch relocation, compute the signed word difference from the end of the **final-width** instruction to the target; the Branches section gives the exact PC formula and range. Inserting WIDE changes that origin by four bytes. This word-relative branch operand must not be confused with the byte address in a function record or with the unsigned word displacement of a `_D` memory access.

1.3.5 Register and constant selection

ABC RK consumers sign-extend their 9-bit unprefixed or 15-bit prefixed B/C field. A nonnegative result selects that frame register. A negative result selects constant `-1-RK`: −1 names K0, not a negative-number immediate. Thus each unprefixed RK can name R0–R255 or K0–K255; each WIDE RK can name R0–R16383 or K0–K16383. A raw count/key field instead keeps its unsigned encoded value.

An instruction's encoding range does not allocate its referenced registers or constants. A frame obtains its entry PC, parameter count and stack allocation from its function record. The reserved capacity is at least one slot, including when the record's maximum stack is zero. All reserved slots begin nil before arguments are copied; missing fixed parameters are nil. Ordinary register writes can extend the active range, while result placement and open varargs have the specific top rules in the call reference. Do not treat the instruction's maximum field value as available scratch storage in every function.

1.3.6 Execution addresses and instruction reads

1.3.6.1 Function records and instruction addresses

A function address points to a **function record**, not to the first instruction. The record's `CODE_ADDRESS` word supplies the instruction address. These two addresses are not interchangeable.

The record and the instructions also use different byte orders:

Item	Representation
Function record's `CODE_ADDRESS` word	32-bit little-endian
Instruction word	Four bytes, big-endian

Reading a function record therefore does not use the byte order used to decode its instructions.

1.3.6.2 Execution domains and cartridge sockets

BMSX has three execution domains:

Domain ID	Execution image
−1	System
0	Cartridge socket 0
1	Cartridge socket 1

For cartridge execution, the active image binds instruction reads to its cartridge socket. Domain identifiers are not RAM addresses and do not allocate isolated RAM.

1.3.7 Fetch and conditional skip

1.3.7.1 Instruction alignment and read faults

Instruction PC is an unsigned 32-bit byte address and must be four-byte aligned. Sequential advancement, WIDE-body addresses, branch targets and conditional skips retain the low 32 bits of the calculated address. A misaligned fetch records BadAddress and raises `AddressErrorLoad`. A new bus fault reading either the first word or the WIDE body raises `InstructionBusError`; the saved fetch PC is the instruction start, including the prefix address when the body read fails. Fetch exception entry has a one-cycle charge; an unfetched body is not dispatched with its nominal instruction cost.

In ordinary sequential execution, an instruction's effects precede a fault fetching the following instruction. The completed instruction's effects remain in place; the saved fetch PC identifies the instruction whose fetch failed.

Conditional-skip instructions decode the next instruction's full width and skip that whole instruction without executing its handler or charging its body's base cost. The skipped code can still cause a fetch exception while its width is determined. The branch section separately defines signed word relative targets; a branch instruction does not itself read its target.

1.4 BLua operators and instruction costs

The table relates BLua operators to their principal instruction families. Include operand preparation, typed loads/stores and branches when calculating a source expression's timing. A declared static array's # supplies its known element count; String and Table length use LEN.

Source operation	Principal instruction family
`+`, binary `-`, `*`, `/`	`ADD`, `SUB`, `MUL`, `DIV`
`//`, `%`, `^`	`FLOORDIV`, `MOD`, `POW`
`&`, binary `|`, binary `~`	`BAND`, `BOR`, `BXOR`
`<<`, `>>`	`SHL`, `SHR`
Unary `-`, `not`, `#`, unary `~`	`UNM`, `NOT`, `LEN`, `BNOT`, subject to static-array length above
`==`, `~=`	`EQ` with true or false expectation
`<`, `<=`	`LT`, `LE` with true expectation
`>`, `>=`	`LT`, `LE` with reversed operands and true expectation
`..`	CONCAT or CONCATN, with operand preparation

The arithmetic binary forms use RK operands. Unary forms use a source register. The comparison instructions are predicates that conditionally skip another instruction; they do not store a Boolean in `A`. Producing a comparison value can therefore require Boolean loads and a jump as well as the predicate. Use the instruction path, not the number of source operators, when calculating a cycle budget.

Source `and` and `or` select values by short-circuit evaluation rather than requesting a Boolean-valued arithmetic instruction. Their evaluation and result-adjustment rules are in BLua — Logical expressions.

1.5 Values and numeric instructions

1.5.1 Reading the instruction tables

The tables use the register and value notation introduced in the programming model. Unless a row says otherwise, A selects the destination register. Comparison instructions instead use A as an expected boolean result. Formulas describe instruction effects, not BLua source syntax.

The narrow and WIDE forms have the following encoding limits. These limits describe what the instruction can encode, not how many registers or constants are allocated for a particular function.

Operand	Ordinary encoding	WIDE encoding
RK B or C	Signed 9 bits: R0–R255 or K0–K255	Signed 15 bits: R0–R16383 or K0–K16383
A in ABC formats	8 bits	14 bits
A in Bx formats	6 bits	12 bits
LOADK Bx	Unsigned 20 bits	Unsigned 26 bits
KSMI sBx	−524288 through 524287	−33554432 through 33554431

A single-result destination write raises the current frame's top to at least A+1 and never lowers it. The positive-count LOADNIL range is described separately below.

Opcode numbers are zero-based. Nominal cycles are the base charge applied before an instruction executes. The constant, arithmetic, bitwise, text and comparison instructions below each cost one cycle except CONCATN, which costs two. LOADNIL adds no per-register charge, and CONCATN adds no per-input or per-byte charge. Readiness waits, counted-transfer surcharges and builtin charges are separate; Cycle accounting and execution boundaries defines their place in the execution budget.

1.5.2 Constants and copies

The constant and copy instructions move complete values or create a specific value kind. K0, K1, KM1, and KSMI create Numbers, not a separate integer kind. LOADNIL's B operand is a count: for example, A=4 and B=3 write nil to R4, R5, and R6, not through R7.

Mnemonic	Opcode	Operands and specified domain	Result	Nominal cycles
MOV	`01H`	A destination; B source register; C unused	`R[A] = R[B]`, preserving the complete value	1
LOADK	`02H`	A destination; Bx identifies an existing constant	`R[A] = K[Bx]`, preserving the constant's value kind	1
LOADNIL	`03H`	A first register; B register count; C unused	For B>0, set exactly `R[A]` through `R[A+B−1]` to nil and raise frame top to at least A+B without lowering it. B=0 changes neither slots nor top.	1
KNIL	`05H`	A destination; B and C unused	`R[A] = nil`	1
KFALSE	`06H`	A destination; B and C unused	`R[A] = false`	1
KTRUE	`07H`	A destination; B and C unused	`R[A] = true`	1
K0	`08H`	A destination; B and C unused	Number +0 in `R[A]`	1
K1	`09H`	A destination; B and C unused	Number 1 in `R[A]`	1
KM1	`0AH`	A destination; B and C unused	Number −1 in `R[A]`	1
KSMI	`0BH`	A destination; signed immediate sBx within the encoding range above	Number equal to sBx in `R[A]`	1
LOADKR	`35H`	A destination; B source register; C unused	Convert `R[B]` to unsigned 32-bit index i, then `R[A] = K[i]`, preserving the constant's value kind. An index outside the pool raises `INVALID_ARGUMENT` (10) before the destination is written. B is not an RK operand.	1

1.5.3 Arithmetic

ADD, SUB, MUL, DIV, MOD, FLOORDIV, and POW read two numeric operands through RK B and RK C and write a Number to R[A]. They do not saturate or apply integer-word wrapping. A non-Number operand supplies NaN rather than causing an arithmetic type fault.

DIV is ordinary floating-point division. FLOORDIV instead rounds the quotient downward, not toward zero: dividing −7 by 3 gives a floor quotient of −3, and dividing 7 by −3 also gives −3. The result is still a Number, not a different integer value kind. MOD adjusts a nonzero remainder to agree with the divisor's sign for finite inputs and a nonzero divisor: −7 modulo 3 is 2, whereas 7 modulo −3 is −2. A zero remainder keeps the sign of the initial remainder; it is not forced to the divisor's sign.

DIV, MOD, and FLOORDIV have no separate zero-divisor or nonfinite-operand guard. They use the arithmetic operations stated below, and their results remain Numbers. Do not assume a checked integer-divide trap, substitute a saturated result, or apply the finite-input quotient-and-remainder rules to every nonfinite case.

Mnemonic	Opcode	Operands	Number result in R[A]	Nominal cycles
ADD	`11H`	Numeric reads of RK(B), RK(C)	left + right	1
SUB	`12H`	Numeric reads of RK(B), RK(C)	left − right	1
MUL	`13H`	Numeric reads of RK(B), RK(C)	left × right	1
DIV	`14H`	Numeric reads of RK(B), RK(C)	left divided by right	1
MOD	`15H`	Numeric reads of RK(B), RK(C)	Start with the floating-point remainder r. If r>0 and right<0, or r<0 and right>0, return r+right; otherwise return r.	1
FLOORDIV	`16H`	Numeric reads of RK(B), RK(C)	Set q to left/right and t to q truncated toward zero. Return t−1 if t>q; otherwise return t.	1
POW	`17H`	Numeric reads of RK(B), RK(C)	left raised to the power right, with the special cases below	1
UNM	`1FH`	Numeric read of R[B], not RK(B); C unused	Negation of the operand, including its zero sign	1

1.5.4 Bitwise operations and shifts

Each bitwise input is converted to a 32-bit word by truncating its Number toward zero and reducing the integer modulo 2^{32}. Positive zero, negative zero, NaN, infinities, and fractions of magnitude below one become the zero word. Large finite values wrap modulo 2^{32} rather than saturating. A non-Number numeric operand first supplies NaN and therefore becomes the zero word in these operations.

The output word is interpreted as a signed 32-bit integer and written as a Number. For example, the word with all 32 bits set becomes Number −1. A left shift of 1 by 31 produces −2147483648. SHR is an arithmetic, sign-extending right shift: shifting −8 right by one produces −4, not a large positive value.

Shift counts undergo the same word conversion and then use only their low five bits. A count of 32 therefore acts as zero, and a count of −1 acts as 31. Negative counts do not reverse the shift direction, and large counts do not saturate.

Mnemonic	Opcode	Operands	Result in R[A]	Nominal cycles
BAND	18H	Word conversions of numeric reads of RK(B), RK(C)	Signed Number interpretation of the 32-bit AND	1
BOR	19H	Word conversions of numeric reads of RK(B), RK(C)	Signed Number interpretation of the 32-bit OR	1
BXOR	1AH	Word conversions of numeric reads of RK(B), RK(C)	Signed Number interpretation of the 32-bit exclusive OR	1
SHL	1BH	Left word from RK(B); count from the low five bits of the word converted from RK(C)	Shift left, discard bits leaving the 32-bit word, then write its signed Number interpretation	1
SHR	1CH	Left word from RK(B); count from the low five bits of the word converted from RK(C)	Sign-extending right shift of the signed 32-bit left operand, written as a Number	1
BNOT	22H	Word conversion of the numeric read of R[B], not RK(B); C unused	Complement all 32 bits, then write the signed Number interpretation	1

1.5.5 Truth, length, and text

Only nil and false are falsey. Every other value is truthy, including Number 0, Number NaN, and an empty String. NOT writes a boolean, not a Number. In particular, negating the truth of zero produces false.

LEN returns a Number. A String's length is its stored Unicode codepoint count, not its byte count. A Table's length is its maintained array length. LEN on any other value kind faults; it does not invoke a metatable length operation.

Concatenation uses language value-to-string conversion rather than accepting only strings and Numbers. CONCAT joins its two RK operands in B-then-C order. CONCATN joins exactly C consecutive registers starting at R[B], in increasing register order. All CONCATN sources are read before the result is written, so the destination may overlap the input range. A zero CONCATN count produces the empty String.

Mnemonic	Opcode	Operands and domain	Result in R[A]	Nominal cycles
CONCAT	1DH	RK(B), then RK(C); every language value kind has the conversion shown below	Combined String	1
CONCATN	1EH	B first register; C register count; neither B nor C is RK	String formed from `R[B]` through `R[B+C−1]`; empty String when C=0	2
NOT	20H	R[B], not RK(B); C unused	Boolean true for nil or false; boolean false for every other kind	1
LEN	21H	R[B], not RK(B); C unused; String or Table	Number equal to the String codepoint count or Table array length; other kinds fault	1

Value being concatenated	Resulting text
nil	`nil`
false	`false`
true	`true`
String	Its contents
Table	`table`
Thread	`thread`
Closure or BuiltinFunction	`function`
NaN	`nan`
Positive infinity	`inf`
Negative infinity	`-inf`
Finite Number	Default six-significant-digit conversion below; +0 becomes `0`, and −0 becomes `-0`

Concatenating a Table therefore does not serialize its entries, and concatenating a function does not print its contents or identity. Use concatenation to assemble readable text, but do not assume that its text distinguishes every language value. Finite Number formatting is shared numeric formatting, not a conversion to a 32-bit word.

1.5.6 Comparisons and conditional execution

EQ, LT, and LE do not write a result register. They evaluate a predicate and compare it with the expectation encoded by A: zero expects false, and any nonzero A expects true. If the predicate differs from that expectation, execution skips exactly the next decoded instruction. A WIDE prefix and its instruction body form one instruction for this skip. If the predicate matches the expectation, the next instruction executes normally.

To produce a Boolean value for `R1 < R2`, the program must also select a Boolean load. With R1 and R2 holding Numbers and R0 available for the result, the following five unprefixed instructions select the appropriate load. P is a four-byte-aligned instruction address:

Instruction address	Instruction	Operation
P	`LT 1,1,2`	Expect the comparison to be true; skip the instruction at P+4 when it is false
P+4	`JMP +2`	Continue at P+16
P+8	`KFALSE 0`	Write false to R0
P+12	`JMP +1`	Continue after the sequence at P+20
P+16	`KTRUE 0`	Write true to R0

The true path executes LT, the first JMP and KTRUE. The false path skips the first JMP and executes KFALSE followed by the second JMP. Both paths reach P+20 with a Boolean in R0 and have three base instruction cycles, assuming successful instruction reads. LT itself never writes R0; its A=1 is the expected truth, not register R1. The branch reference defines the word-relative offsets used by the two jumps.

Equality respects value kinds. Different kinds are unequal; there is no numeric-string or boolean-number equality conversion. Numbers use numeric equality, so NaN is unequal even to itself, while +0 and −0 are equal. Table, Closure, and Thread equality is reference identity rather than a comparison of their contents.

Matching operand kinds	EQ predicate
nil, false, or true	Equal to the same kind
Number	Numeric equality; NaN is unequal, +0 equals −0
String	Same interned string identity
BuiltinFunction	Same builtin identifier
Table, Closure, or Thread	Same reference identity
Different kinds	False

Ordering is defined for two Numbers or two Strings. Numbers use numeric less-than or less-than-or-equal comparison. Strings use lexical UTF-8 order. Mixed kinds and all other kind combinations produce a false predicate rather than a comparison

type fault. Equality of Tables can therefore test whether two values refer to the same Table, but LT and LE do not provide an ordering of Tables.

Mnemonic	Opcode	Operands	Predicate and control effect	Nominal cycles
EQ	`23H`	RK(B), RK(C); A is expected truth, not a destination	Equality by the value-kind rules above; skip the next decoded instruction if the predicate differs from `(A≠0)`	1
LT	`24H`	RK(B), RK(C); A is expected truth, not a destination	Number less-than or String lexical less-than; false otherwise; skip the next decoded instruction if the predicate differs from `(A≠0)`	1
LE	`25H`	RK(B), RK(C); A is expected truth, not a destination	Number less-than-or-equal or String lexical less-than-or-equal; false otherwise; skip the next decoded instruction if the predicate differs from `(A≠0)`	1

1.6 Numeric result details

1.6.1 POW: exponentiation

POW computes a Number base `x` raised to a Number exponent `y`, written here as `x^y`. Its result is a Number, including when the result is a signed zero, an infinity, or NaN. In this section, `+0` and `-0` are distinct zeros; `+infinity` and `-infinity` have their usual signs. NaN means a not-a-number result, without a promise to preserve an operand's NaN payload.

1.6.1.1 Order of special cases

Apply these rules in order. The order matters when more than one description appears to fit.

1. **Either zero exponent produces +1.** For every base, `x^(+0)` and `x^(-0)` are `+1`. This includes zero, infinity, and NaN bases.
2. **Otherwise, either NaN operand produces NaN.** Consequently, `NaN^0` is `+1`, but `1^NaN` is NaN.
3. **An infinite exponent uses the base's magnitude.** For a non-NaN base, the following table applies before the rules for finite exponents.

Base magnitude	Exponent `+infinity`	Exponent `-infinity`
`abs(x) < 1`, including either zero	`+0`	`+infinity`
`abs(x) = 1`, meaning `+1` or `-1`	NaN	NaN
`abs(x) > 1`, including either infinity	`+infinity`	`+0`

The base's sign does not change these infinite-exponent results. In particular, neither `1^(+infinity)` nor `(-1)^(-infinity)` is `1`. Every zero and infinity produced by this table is positive.

1.6.1.2 Finite, nonzero exponents

The next table covers all finite, nonzero exponents for the special bases. **Odd** and **even** mean integer exponents; **fractional** means nonintegral, not merely a decimal spelling. The exponent's sign selects a column group independently of its parity: `-3` is odd, while `-2` is even.

For a **positive finite exponent**:

Base	Odd integer	Even integer	Nonintegral
`+0`	`+0`	`+0`	`+0`
`-0`	`-0`	`+0`	`+0`
`+infinity`	`+infinity`	`+infinity`	`+infinity`
`-infinity`	`-infinity`	`+infinity`	`+infinity`
`+1`	`+1`	`+1`	`+1`
`-1`	`-1`	`+1`	NaN

For a **negative finite exponent**:

Base	Odd integer	Even integer	Nonintegral
`+0`	`+infinity`	`+infinity`	`+infinity`
`-0`	`-infinity`	`+infinity`	`+infinity`
`+infinity`	`+0`	`+0`	`+0`
`-infinity`	`-0`	`+0`	`+0`
`+1`	`+1`	`+1`	`+1`
`-1`	`-1`	`+1`	NaN

Thus a fractional exponent does not itself make a negative-zero or negative-infinity base invalid. For example, `(-0)^0.5` is `+0`, `(-0)^(-0.5)` is `+infinity`, `(-infinity)^0.5` is `+infinity`, and `(-infinity)^(-0.5)` is `+0`.

For other finite, nonzero bases:

- A positive base has a positive result magnitude, which may become `+0` or `+infinity` at the range limits.
- A negative base with a nonintegral exponent produces NaN. For example, `(-2)^0.5` is NaN.
- A negative base with an odd integer exponent has a negative result; an even integer exponent has a positive result. This sign rule also applies when the magnitude becomes zero or infinity. A negative exponent takes a reciprocal magnitude; it does not reverse the result's sign.

1.6.1.3 Specific finite exponents

Several exponents have particularly direct meanings. The earlier zero-exponent and NaN rules still take precedence.

Exponent	Result
`+1`	The base unchanged, including `-0` and `-infinity`.
`-1`	The reciprocal `1/x`, preserving the reciprocal's sign: `-0` becomes `-infinity`, and `-infinity` becomes `-0`.
`+2`	The Number product `x*x`. Either zero becomes `+0`; either infinity becomes `+infinity`. A finite product can overflow, become subnormal, or round to `+0`.
`+0.5`	For a positive finite base, the positive square root. The special-base table gives `+0` for either zero and `+infinity` for either infinity; a negative finite, nonzero base produces NaN.

In particular, `(-0)^0.5` does **not** preserve the base's negative-zero sign.

1.6.1.4 Represented exponent parity

Oddness and evenness refer to the exponent Number actually supplied. Numbers use binary64 precision. From `2^52` upward, every representable finite exponent is integral. From magnitude `2^53` upward, every representable finite exponent is even: no odd exponent exists at that precision.

The largest positive representable odd integer is `2^53 - 1`, or `9007199254740991`. Therefore `(-1)^(2^53 - 1)` is `-1`, whereas `(-1)^(2^53)` is `+1`. The same parity distinction applies to the corresponding negative exponents. Sign selection cannot recover an intended odd integer that the supplied Number cannot represent.

1.6.1.5 Range limits and subnormal results

POW accepts nonzero subnormal bases and can produce subnormal results; it does not discard all such values as zero. Overflow produces infinity, while sufficiently small magnitudes round to zero. For negative bases with odd integer exponents, these results retain a negative sign.

The following are exact result examples, not decimal display approximations:

Expression	Result
`2^(-1074)`	`2^(-1074)`, the smallest positive subnormal Number
`2^(-1075)`	`+0`
`2^1024`	`+infinity`
`(-2)^1025`	`-infinity`
`(-2)^(-1075)`	`-0`

These exact examples do not establish a universal correct-rounding or error guarantee for other finite powers.

1.6.2 Default Number-to-text conversion

Concatenating a Number with a String uses the default Number-to-text conversion. For example, `'' .. 1.234565` produces `1.23456`. Both `CONCAT` and `CONCATN` use this ordinary conversion; neither supplies a format string, precision, field width, padding, or notation choice. The rules below describe that default conversion, not a configurable formatting operation.

The precision target is **six significant decimal digits**, not six decimal places and not a requirement to display six digits. Fractional trailing zeros disappear. Thus `0.0001` has fewer visible digits, while a small value can have several zeros after the decimal point before its significant digits begin. Integer trailing zeros remain part of the printed integer.

1.6.2.1 Sign, zero, and the integer shortcut

Positive zero becomes `0`; negative zero becomes `-0`. Neither goes through normalization. For every other finite value, conversion works on the positive magnitude and prefixes one `-` if the input was negative. Positive inputs have no leading `+`.

An exact integer magnitude **strictly below 1000000** is printed directly as a decimal integer, without normalization or rounding. For example, `999999` becomes `999999`, and `-999999` becomes `-999999`. The strict limit matters: `1000000` takes the normalized path and becomes `1e+06`.

1.6.2.2 Normalization and rounding

For all remaining finite inputs, let *n* initially be the positive magnitude and let the decimal exponent *e* initially be zero. Perform the following operations in the stated order, using ordinary Number arithmetic at each multiplication. The powers in the table steps are fixed Number constants, not exponentiation operations requested by the program.

1. **Normalize the magnitude.** Use the descending stage order **256, 128, 64, 32, 16, 8, 4, 2, 1**.

   If the initial *n* is at least 10, visit each stage *k*. Whenever the current *n* is at least 10*k*, multiply *n* by the Number constant 10−*k* and add *k* to *e*. After the table stages, while *n* is still at least 10, multiply it by `0.1` and increment *e*.

   If the initial *n* is below 1, visit the same descending stages instead. Whenever the current *n* is below 10−*k*, multiply *n* by the Number constant 10*k* and subtract *k* from *e*. After the table stages, while *n* is still below 1, multiply it by `10` and decrement *e*.

An initial magnitude in [1, 10) needs neither branch. These operations normalize a computed Number; they are not exact real-number decimal normalization.

2. **Scale and round.** Multiply the computed *n* by `100000`. Round this computed result to the nearest integer: a fractional part below `0.5` rounds down, one above `0.5` rounds up, and exactly `0.5` selects the even integer. This produces the rounded digit integer used to construct the text.
3. **Apply a carry before choosing notation.** If the rounded integer is `1000000`, replace it with `100000` and increment *e*. Both the digit buffer and the exponent now include the rounding carry.

Nearest-even applies to the **computed normalized and scaled Number**, not directly to the original input's mathematical decimal expansion. This distinction is observable: `123456.5` prints `123457`, not `123456`. In contrast, `1.234565` prints `1.23456`, and `1.234575` prints `1.23458`. Do not substitute an ideal six-significant-digit decimal-rounding rule for the ordered Number operations above.

1.6.2.3 Choosing and spelling the notation

Use the exponent **after the carry**. Fixed notation is selected exactly when

$$-4 \le e < 6.$$

In fixed notation, the decimal point position is $e + 1$, counted from the start of the six-digit buffer. A positive position places that many digits before the point; insert a point only when the position is below six. A position of zero or less produces `0.`, followed by the required leading fractional zeros and then the six-digit buffer. Remove trailing fractional zeros, then remove the decimal point if no fractional digits remain. Do not remove integer trailing zeros.

All other exponents use scientific notation. Place the point after the first digit of the six-digit buffer, then trim fractional zeros and an empty decimal point in the same way. Append lowercase `e`, an explicit `+` or `-`, and the exponent magnitude. Exponent magnitudes below 100 use exactly two digits, including a leading zero when needed; larger magnitudes use three digits. Examples of exponent suffixes are `e+06`, `e-05`, and `e-324`.

The notation boundary is therefore not merely a comparison of the original magnitude with `0.0001` or `1000000`. Rounding can move the exponent across a boundary: `0.00009999995` prints `0.0001`, while `999999.5` prints `1e+06`.

1.6.2.4 Number-to-text examples

Each row gives a Number input and its resulting text. The same conversion is obtained by using that input in `'' .. input`.

Number input	Resulting text
positive zero / negative zero	`0` / `-0`
`0.0000999999`	`9.99999e-05`
`0.0001`	`0.0001`
`0.00001`	`1e-05`
`0.00009999995`	`0.0001`
`999999`	`999999`
`999999.5` / `1000000`	`1e+06` / `1e+06`
`1.234565` / `1.234575`	`1.23456` / `1.23458`
`123456.5` / `123457.5`	`123457` / `123458`
`2147483648`	`2.14748e+09`
`5e-324`	`4.94066e-324`
`1.7976931348623157e308`	`1.79769e+308`

The output is not a lossless serialization: distinct inputs such as `999999.5` and `1000000` produce the same text. Nonfinite Numbers bypass the finite conversion described above: NaN becomes `nan`, positive infinity becomes `inf`, and negative infinity becomes `-inf`.

1.7 Calls, closures, globals, and tables

1.7.1 Calling and returning values

Function calls use language value slots. A frame's register area follows its variable-argument area in the active thread's slot stack. The function record's maximum-stack value is a **slot count**, not a RAM address or byte count. These slots have no cartridge-addressable byte-RAM base; do not place an assumed stack at the top of main RAM.

`CALL A,B,C` takes its callee from `R(A)`. Arguments occupy consecutive registers starting at `R(A+1)`, and returned values replace consecutive registers starting at `R(A)`.

Only closure and builtin-function values are callable. A number is not callable merely because its numeric value is a function-record address. Calling any other value category raises `CALL_NON_FUNCTION`.

The three count operands in the call/return family deliberately do not use one common encoding:

Instruction operand	Zero	Positive value
`CALL B`: supplied arguments	Use `max(top-A-1, 0)` arguments.	Supply exactly `B-1` arguments. In particular, `B=1` supplies none.
`CALL C`: requested results	Accept all produced results.	Accept exactly `C` results.
`RET B`: produced results	Return `max(top-A, 0)` values.	Return exactly `B` values.
`VARARG B`: copied arguments	Copy all saved varargs.	Copy exactly `B` values, filling any shortage with nil.

For example, place a callee in R3 and two arguments in R4 and R5, then execute `CALL 3,3,2`. It supplies those two arguments and requests two results in R3 and R4. If the function produces only one result, R4 becomes nil; if it produces more than two, the excess is discarded. After result placement, `top` is exactly 5, so the earlier argument in R5 lies outside the active range.

An open-result call instead sets `top` to `A` plus the number of results actually produced. This exact-top rule makes a returned value range available to a following open-argument call or open return without treating every previously written register as an argument or result. It also means that result placement can lower `top`. Normal builtin completion follows the same result selection, nil padding, and exact-top rules.

On entry to a closure, the function record supplies the code address and parameter metadata. The frame starts with `top` equal to its fixed-parameter count, not its maximum stack allocation. Supplied arguments fill the fixed parameters in order, and missing fixed arguments become nil. Excess arguments are retained as varargs only for a function marked vararg; otherwise they are not retained as varargs.

`VARARG A,0` copies the saved excess arguments starting at `R(A)` and sets `top` exactly to the end of that copied range. If there are none, it sets `top` to `A`. A fixed-count `VARARG` pads with nil as needed and only raises `top`, to at least `A+B`.

`RET` closes the returning frame's open upvalues before routing its results. An ordinary nested return places results in the caller's saved destination range and releases the returning frame. A return need not have an ordinary caller: protected-call targets, completion, and coroutine resumption have distinct return routes. Returning from a root frame with no resumer records completion values.

1.7.2 Closures and captured variables

A closure is a callable language value associated with a function record and captured upvalues. `CLOSURE` constructs this value; it does not merely copy a numeric code address into a callable register.

The `CLOSURE` register-address flag chooses how to obtain the function-record address:

- Flag clear: the address is `Bx × 16`.
- Flag set: `Bx` selects the register supplying the unsigned address.

The function record is read before the destination is written. It identifies the function's code and frame metadata, including the fixed-parameter count, maximum stack allocation, vararg and static flags, and upvalue information. A failed record read hard-halts without writing `R(A)`. A bus-read fault while reading the upvalue descriptors likewise hard-halts without writing `R(A)`.

Each upvalue descriptor is a 32-bit word. Descriptor `i` is read at `upvalueTableAddress + 4i`:

Descriptor bits	Capture source
High bit set	A register slot in the parent frame; the low 31 bits select the slot.
High bit clear	An upvalue of the parent closure; the low 31 bits select that upvalue.

An open upvalue accesses its captured parent-frame register directly. Captures can share an already-linked open upvalue for that slot, so updates through the captured variable remain shared. Closing an upvalue retains the current language value separately from the frame; subsequent reads and writes access that retained storage. `GETUP` and `SETUP` use the same upvalue operand regardless of whether the upvalue is open or closed.

Repeated CLOSURE operations for a STATIC function record with no upvalues return the same function value for that record address. An ordinary call uses that value; a numeric function-record address is supplied to CLOSURE or EXEC under their separate operand rules.

1.7.3 Two global namespaces

Ordinary globals and system globals are separate namespaces. `GETGL` and `SETGL` access ordinary globals; `GETSYS` and `SETSYS` access system globals. A matching name in one namespace does not select the value stored under that name in the other.

For these instructions, `Bx` is an index into the corresponding execution-image name table. It is not a register operand, a constant-pool string operand, or a table key. Each image's ordinary and system name tables are independently mapped to their namespace slots. Registering a new name initializes its value to nil; registering an existing name reuses the slot in that namespace. The namespace distinction alone does not specify a privilege boundary.

1.7.4 Tables, keys, and method receivers

The table instruction families differ in where the key comes from:

- `GETT` and `SETT` take general keys through `RK` operands.
- `GETI` and `SETI` carry the integer key directly in an instruction operand.
- `GETFIELD`, `SETFIELD`, and `SELF` select a string key directly from the image constant pool. That constant operand is not an `RK` operand.

Reads accept a table base. A string base instead uses the CPU's string-index table; these instructions do not specify substring indexing. Other base value categories raise `INDEX_NON_TABLE`. Without a metatable, a table read is direct. The table-only inherited lookup and its exact chain bound are specified below.

Writes accept only table bases, including for integer and string-field forms. Assigning through any other value category, including a string, raises `ASSIGN_NON_TABLE`.

`SELF` prepares both a method value and its receiver. It preserves the original `R(B)` in `R(A+1)` before looking up the string-keyed field into `R(A)`. The receiver therefore remains available as the first argument when the method value is called. Both destination writes extend `top` as needed. Preserving the original receiver also defines the result when a destination overlaps the source register.

`NEWT` creates an empty table with an array-capacity request and a hash-size request. These operands request storage; they neither supply initial elements nor state the number of elements already present.

1.7.5 Table entries and inherited reads

A Table maps admissible keys to non-nil values. Numeric and nonnumeric keys may coexist in the same Table. Array storage and hash storage are internal placement choices, not separate kinds of Table. Moving an entry between them does not change its key category.

1.7.5.1 Keys and presence

Nil is not an admissible key. Reading or writing with a nil key raises `INDEX_NIL` (4), even when the Table is empty or the value being written is nil. This is distinct from reading a missing admissible key, which returns nil.

Key category	Matching rule
Boolean	`false` and `true` are distinct keys.
Number	Numeric matching, with the zero and NaN rules below.
String	String identifiers match; no conversion to a numeric key occurs.
Table, closure, thread	The same reference matches; separate objects do not match by contents.
Builtin function	The same builtin identity matches.

Numeric `1`, string `"1"`, and boolean `true` are separate keys. Positive zero and negative zero address the same key. A canonical numeric NaN is also an admissible key: repeated NaN keys address the same mapping, although ordinary numeric NaN equality is false. Positive and negative infinity are admissible numeric keys.

Only positive integral numeric keys from 1 through `0x7fffffff` are eligible for array storage. Zero, negative numbers, fractions, NaN, infinities, and larger numbers use hash storage. An eligible numeric key may also reside in hash storage when it lies beyond the allocated array capacity. Numeric key zero has no special indexing role.

A non-nil value establishes a mapping. In particular, `false` is present: it is not a missing entry and does not permit an inherited value to replace it. Assigning nil removes the Table's own mapping. It does not create a retrievable nil-valued entry. An allocated but empty array slot is likewise missing. Deletion does not erase other keys.

1.7.5.2 Stored length

The Table length operation, `#t`, reads a stored length maintained by Table mutations. It does not count all mappings, find the largest numeric key, traverse a metatable, or search for a boundary when the length is read. Initial capacity allocates storage only: an empty Table has length zero regardless of its array or hash capacity.

Let `L` denote the stored length. Deleting an array-eligible numeric key `k` shortens the length to `k - 1` when `k - 1 < L`. Otherwise that deletion does not shorten it. Entries above the new boundary remain mapped. Thus a Table can retain entries at keys greater than its reported length.

An in-capacity non-nil write at key `L + 1`, or a newly inserted hash entry for that array-eligible key, advances the length over consecutive non-nil own entries until the first missing numeric key. This scan continues into hash storage beyond array capacity. A `false` entry participates in that advance. Other in-capacity non-nil writes do not advance the boundary merely because their key is large. Replacing an existing non-nil hash mapping does not itself recompute the stored length. Storage redistribution can rebuild length through reinsertion, so not every apparently unrelated insertion is guaranteed to leave an already path-dependent length unchanged.

For example, with array capacity four and keys 1 through 5 populated, the initial length is 5. Deleting key 2 shortens it to 1 without removing key 5. Restoring key 2 advances the length to 5, including that hash-held entry.

1.7.5.3 Metatables and indexed reads

A Table may have a metatable. An ordinary indexed read first checks the queried Table's own mapping. A non-nil value wins immediately, including `false`. If the entry is missing, the read obtains the direct `"__index"` member of that Table's metatable. Only a Table-valued `__index` continues the lookup, using the original key on that next Table. Each subsequent Table is processed in the same way.

An absent or non-Table `__index` terminates the read with nil. Function-valued indexers are not called. The `__index` member itself is read directly; its retrieval does not start another inherited lookup. One chained read permits 32 visited Tables. Exhausting that bound raises `METATABLE_LOOP` (5); an actual cyclic reference is not required to exhaust the bound.

Writes always modify the named target Table directly. They do not redirect through `__newindex` and do not invoke a function stored there. Deleting an own mapping may therefore expose an inherited mapping on a later ordinary read.

The BIOS Table and metatable interface specifies `rawget`, `rawset`, `getmetatable` and `setmetatable`. Raw reads bypass the inherited lookup above; raw writes use the same maintained Table length.

1.7.6 Language allocation and lifetime

The CPU provides a **3.5 MiB language-allocation budget** for system and cartridge execution together. Installed library values and active execution state already consume part of it at cartridge entry. Further retained values and storage growth use the remaining capacity.

The budget covers Table storage, dynamic closures and captured-variable cells, Threads and their reserved value-slot capacity, and String storage charges. Capacity matters as well as visible contents: a Table can reserve more entries than it currently contains, and a Thread's reserved slot storage can exceed its active argument/local range. Setting one field to nil or returning from one call therefore does not imply an equal immediate increase in available budget.

Collection reclaims allocations that are no longer reachable from retained program values. Globals, active frames, reachable Tables and closures, and retained Threads can keep other objects alive; mutually referencing objects can be reclaimed when the whole group becomes unreachable. Weak Table behavior is defined in Weak references and collection. Allocation may invoke collection before it succeeds. If the requested growth still exceeds capacity, it raises the Lua out-of-memory error. If language error handling does not handle it, the CPU takes a Trap exception with LuaFaultReason `9` (`OUT_OF_MEMORY`).

Language allocations have value identity rather than cartridge-visible byte addresses. Plan DATA/BSS and hardware buffers using the separate byte-storage reservations in Book I, "Memory and bus access", under "Reserved and cartridge storage in main RAM". Use addressed byte storage for exact device records and payloads.

1.7.6.1 Weak references and collection

The collector reads a Table's metatable member `__mode` directly. A String containing lowercase `k` selects weak keys; lowercase `v` selects weak values; both select both. Other characters do not select a mode. A missing or non-String member selects ordinary strong entries. This lookup does not invoke `__index`.

Mode	Collection effect
Strong	Entries retain their keys and values.
`k`	An entry's value is retained only when its key is alive independently; chains of such dependencies are followed to convergence. A value cannot keep its own otherwise unreachable weak key alive through that entry.
`v`	Keys remain strong; values alone do not retain collectable objects.
`kv`	Neither side alone retains collectable objects.

Collectable weak entries can refer to Tables, threads and dynamically captured closures. Numbers, booleans, builtin functions and canonical static closures do not disappear through weak-entry collection. Strings also remain in surviving weak entries; they are not removed merely because their weak position is their only reference. A static function with zero upvalues has the canonical closure identity described above, not the lifetime of a newly allocated captured closure.

```
local owners = {}
setmetatable(owners, { __mode = 'k' })
local key = {}
owners[key] = { label = 'attached' }
key = nil
```

The entry does not keep the former key alive. Once collection determines that key unreachable, the mapping is removed; this sequence does not specify an immediate collection or reclamation deadline. Collection removes mappings, not their inherited replacements, and can shorten a weak-value array's stored length when it clears an earlier element.

1.7.7 Instruction reference

The following tables describe the decoded operands. `A`, `B`, `C`, and `Bx` name instruction fields, not additional language values. Opcode values use hexadecimal notation (`H`). All costs are base instruction cycles; see **Builtin call charges** below.

1.7.7.1 Calls and captured values

Instruction	Opcode	Operands	Result or effect	Base cycles
`CALL`	`2FH`	`A, B, C`	Call the closure or builtin in `R(A)` with arguments beginning at `R(A+1)`. `B=0`: use the open argument range; otherwise supply `B-1` arguments. `C=0`: accept all results; otherwise accept exactly `C`, truncating or nil-padding. Results begin at `R(A)`.	2
`RET`	`30H`	`A, B`	Close the frame's open upvalues and return values beginning at `R(A)`. `B=0`: produce the open range; otherwise produce exactly `B` values.	2
`VARARG`	`2EH`	`A, B`	Copy saved varargs beginning at `R(A)`. `B=0`: copy all; otherwise copy exactly `B`, padding with nil.	2
`CLOSURE`	`2BH`	`A, Bx`, register-address flag	Construct a closure in `R(A)`. Flag clear: function record at `Bx × 16`. Flag set: address supplied by `R(Bx)`. Capture according to the record's upvalue descriptors.	1
`GETUP`	`2CH`	`A, B`	`R(A) = upvalue(B)` of the current closure.	1
`SETUP`	`2DH`	`A, B`	`upvalue(B) = R(A)` in the current closure.	2

1.7.7.2 Global namespaces

Instruction	Opcode	Operands	Result or effect	Base cycles
GETGL	38H	A, Bx	Load into R(A) the ordinary global selected by ordinary-global image name index Bx.	1
SETGL	39H	A, Bx	Store R(A) into the ordinary global selected by ordinary-global image name index Bx.	2
GETSYS	36H	A, Bx	Load into R(A) the system global selected by system-global image name index Bx.	1
SETSYS	37H	A, Bx	Store R(A) into the system global selected by system-global image name index Bx.	2

1.7.7.3 Tables and field lookup

Instruction	Opcode	Operands	Result or effect	Base cycles
GETT	0EH	A, B, C	R(A) = index(R(B), RK(C)).	1
SETT	0FH	A, B, C	index(R(A), RK(B)) = RK(C).	2
GETI	3AH	A, B, C	R(A) = index(R(B), C). C is the integer key itself.	1
SETI	3BH	A, B, C	index(R(A), B) = RK(C). B is the integer key itself.	2
GETFIELD	3CH	A, B, C	R(A) = index(R(B), K(C)), where K(C) supplies the string key.	1
SETFIELD	3DH	A, B, C	index(R(A), K(B)) = RK(C), where K(B) supplies the string key.	2

Instruction	Opcode	Operands	Result or effect	Base cycles
`SELF`	`3EH`	`A, B, C`	Preserve original `R(B)` in `R(A+1)`, then set `R(A) = index(original R(B), K(C))`, where `K(C)` supplies the string key.	1
`NEWT`	`10H`	`A, B, C`	Put a new table in `R(A)`, with array-capacity request `B` and hash-size request `C`.	1

1.7.8 Builtin call charges

CALL costs 2 base cycles. A primitive target adds its own base charge of 1, 2 or 4 cycles; this surcharge has no per-argument or per-result term. Installed BIOS functions can execute ordinary instructions and call further primitives, so include the executed sequence when budgeting the whole service. The BIOS base, String and protected-call/coroutine references identify their primitive surcharges and error behavior.

1.8 Worked values: signed zero and inherited fields

Use runtime operands when the distinction between Number values matters:

```
local values = { -0, 3, -3, 0.5 }
local negative_zero = values[1] ^ values[2]
local positive_zero = values[1] ^ values[4]
local reciprocal = 1 / negative_zero

local parent = { answer = 42 }
local child = {}
setmetatable(child, { __index = parent })
local inherited = child.answer
child.answer = 9
local own = child.answer
child.answer = nil
local inherited_again = child.answer
```

The first group produces negative zero, positive zero and negative infinity. The reciprocal exposes the sign that an equality comparison cannot distinguish. The POW reference above covers infinities, NaNs, represented exponent parity and subnormal results; these rules also apply to runtime operands.

Additional runtime-operand examples follow directly from the POW reference:

Expression	Result
`(-0)^3`, `(-0)^0.5`	Negative zero, positive zero
`(-0)^(-3)`	Negative infinity
`(-infinity)^0.5`, `(-infinity)^(-3)`	Positive infinity, negative zero
`(-1)^infinity`, `(-2)^0.5`	NaN
`NaN^0`	1
`(-1)^9007199254740991`	−1
`(-1)^9007199254740992`	1
`2^(-1074)`	Smallest positive subnormal Number
`(-2)^(-1075)`	Negative zero

Here infinity and NaN denote Number operands, not additional source keywords.

The second group produces `42`, `9`, then `42`. The inherited read does not create an own mapping. Assigning `nil` removes the own mapping and exposes the parent again. `rawget(child, 'answer')` is then `nil`. A function-valued `__index` is not called by this indexed-read datapath; consult the table reference above rather than assuming another Lua version's metatable behavior.

1.9 Memory transfers

Memory instructions transfer numeric scalars between frame registers and byte-addressed storage. Word and U32LE loads produce unsigned 32-bit integer Numbers. Floating loads interpret the stored IEEE bit pattern. The format selector determines the byte width and conversion in the following table.

1.9.1 Scalar formats

The `C` operand of a scalar load or store selects one of six formats:

C	Format	Bytes	Loaded register value	Store conversion	Required address alignment
0	`Word`	4	Unsigned 32-bit integer Number	Convert to unsigned 32-bit integer	4 bytes
1	`U8`	1	Integer Number, 0–255	Store the low 8 integer bits	1 byte
2	`U16LE`	2	Integer Number, 0–65535	Store the low 16 integer bits	2 bytes
3	`U32LE`	4	Unsigned 32-bit integer Number	Convert to unsigned 32-bit integer	4 bytes
4	`F32LE`	4	Number decoded from IEEE single precision	Round the numeric value to IEEE single precision and store its bits	4 bytes
5	`F64LE`	8	Number decoded from IEEE double precision	Store the numeric value's IEEE double-precision bits	**4 bytes**

All multi-byte formats are little-endian: the least significant byte occupies the lowest address. `Word` and `U32LE` transfer the same numeric data, but identify the access differently in bus-fault metadata. Floating-point transfers decode or encode floating-point bits; they are not integer conversions of the floating-point number. No guarantee of preserving every NaN payload is made.

A scalar load or store with C greater than 5 is a reserved instruction. It raises `ReservedInstruction` (exception code 10, Cause `00000028H`) with EPC at the instruction start, before address conversion, alignment checks or a data access. BadAddress and the Memory fault record are unchanged. The normal two-cycle base charge still applies. In counted word stores, C is a word count, not a scalar-format selector.

Address conversion and integer-store conversion truncate toward zero and retain the unsigned low 32 bits. Non-numeric register values take the numeric-conversion path through NaN and become zero on this integer path. Numeric NaN and infinities also become zero on this integer path. Floating-point stores use numeric conversion instead, so a non-numeric register value becomes NaN. A fractional address is converted before its alignment is checked.

`F64LE` is two ordered 32-bit transfers, not one indivisible eight-byte transfer. The low half is accessed at the starting address, then the high half four bytes later. Four-byte alignment is sufficient; eight-byte alignment is not required.

1.9.2 Address operands

The ordinary forms, `LOAD_MEM`, `STORE_MEM`, and `STORE_MEM_WORDS`, obtain their starting byte address from `RK(B)`. A nonnegative `B` selects register `B`; a negative `B` selects constant `-1-B`. The selected value is converted to an unsigned 32-bit address.

The displacement forms, whose names end in `_D`, always use register `B` as the base:

```
address = (u32(R[B]) + 4 × disp) modulo 2^32
```

Here `disp` is the instruction's **unsigned eight-bit word displacement**, from 0 through 255. It reaches byte offsets 0 through 1020. It is neither signed nor extended by `WIDE`. A negative displacement therefore cannot be expressed by setting the top displacement bit. `WIDE` can extend the other operand fields without extending this displacement.

1.9.3 Scalar access and IO

Scalar loads check alignment before attempting a memory access. If a new bus fault occurs during the read, the destination register is not changed. This does not undo a read or a device side effect that has already occurred.

Scalar stores check alignment, then ask whether the mapped starting address is ready for a write. This readiness check precedes reading and converting the source register for the write. If the target is not ready, no store takes place: the instruction is left blocked on that raw target address. A matching ready event clears the blocked condition; the next attempt starts the instruction again. Readiness is checked only at the starting address, including for `F64LE`.

A readiness check is not a permission or fault preflight. An aligned but not-ready IO target can defer a store before a width or bus fault would be observed. Once a scalar store is attempted, a new bus fault raises `DataBusError`; effects already produced by the store remain.

The two halves of an `F64LE` access are fault-ordered. A new fault on the low half prevents the high-half access. A high-half fault leaves the low-half access or store already performed. A load updates its destination only if both halves succeed. The high half of an `F64LE` store is not covered by a separate readiness preflight.

Memory instructions are available in user and supervisor modes. Each mapped target applies its own access rules. In I/O space, an aligned U8 load reads a whole register and returns its low byte. U8 stores and U16LE accesses fault. Word, U32LE, F32LE and each half of F64LE use whole-word register operations, including that register's side effects.

1.9.4 Counted word stores

`STORE_MEM_WORDS` and `STORE_MEM_WORDS_D` use `C` as a **word count**, not a scalar-format selector. They write consecutive registers to consecutive words:

`R[A+i] → (start + 4 × i) modulo 2^32`, for `i = 0 … C-1`.

Each register is converted to an unsigned 32-bit integer. The starting address must be four-byte aligned, even when `C` is zero. A zero count otherwise performs no writes and adds no sequence surcharge.

Before writing anything, the instruction scans the counted span's IO overlap in increasing address order and waits on the first not-ready word. This preflight happens before the sequence surcharge as well as before any write. Consequently, a later not-ready IO word can defer the instruction before an earlier word would otherwise commit or fault. The readiness span is computed without 32-bit wrap; the actual write addresses wrap modulo `2^32`. Do not treat this as readiness preflight for IO reached only by wrapping into low addresses.

Once preflight succeeds, the instruction charges one additional cost unit per four counted words, rounded up: `(C + 3) >> 2`. It then attempts the words in increasing `i` order and checks for a new bus fault after every word. The first such fault stops the sequence and raises `DataBusError`. Earlier successful words remain written; later words are not attempted. The full sequence surcharge has already been charged even if a later word faults.

A blocked attempt writes nothing and retries from the instruction start. After writes begin, the first new bus fault terminates the sequence with its completed prefix retained. Arrange output and recovery around those ordered partial effects.

1.9.5 Memory instruction reference

`read_C(address)` below means a read in the scalar format selected by `C`. `u32` denotes the conversion described above, not serialization of a language value. The common alignment, readiness, fault, and partial-effect rules apply to every applicable row. Costs are nominal base charges plus the stated surcharge; execution-budget, prefix, and retry accounting follow the CPU execution rules.

Opcode	Instruction and operands	Effect	Nominal cost
`04H`	`LOAD_MEM_D A, B, C, disp`	Load `read_C((u32(R[B]) + 4×disp) mod 2^32)` into `R[A]`; leave `R[A]` unchanged on a new bus fault	2
`0CH`	`STORE_MEM_D A, B, C, disp`	Store `R[A]` in scalar format `C` at `(u32(R[B]) + 4×disp) mod 2^32`	2
`0DH`	`STORE_MEM_WORDS_D A, B, C, disp`	Starting at `(u32(R[B]) + 4×disp) mod 2^32`, store `C` consecutive unsigned words from `R[A]` onward	`2 + ((C+3)>>2)` after successful readiness preflight
`31H`	`LOAD_MEM A, B, C`	Load `read_C(u32(RK(B)))` into `R[A]`; leave `R[A]` unchanged on a new bus fault	2
`32H`	`STORE_MEM A, B, C`	Store `R[A]` in scalar format `C` at `u32(RK(B))`	2
`33H`	`STORE_MEM_WORDS A, B, C`	Starting at `u32(RK(B))`, store `C` consecutive unsigned words from `R[A]` onward	`2 + ((C+3)>>2)` after successful readiness preflight

`F64LE` uses the same listed scalar-memory cost as the other scalar formats; it adds no per-half transfer surcharge. Neither this cost nor the counted-store surcharge specifies a device's electrical bandwidth.

1.10 Branches

Branch conditions use value truth. Only Nil and False are false. Numeric zero, numeric NaN, the empty string, and all other language values are true.

The signed `BX` displacement is measured in four-byte instruction words. A branch target is:

```
target = (instruction-start PC + full instruction byte width + 4 × signedBX) modulo 2^32
```

The full instruction width includes a `WIDE` prefix when present. Thus the displacement is relative to the byte immediately after the complete branch instruction, not its first word. `BX` is signed 20-bit without `WIDE` and signed 26-bit with it. A conditional branch that is not taken continues at the already selected sequential PC.

The branch itself performs no target-memory read or target-alignment check. Fetching at the resulting PC is a separate step governed by the instruction-fetch rules.

Opcode	Instruction and operands	Effect	Nominal cost
`28H`	`JMP BX`	Assign the branch target to PC; `A` is ignored	1
`29H`	`JMPIF A, BX`	Assign the target to PC if `R[A]` is true; otherwise continue sequentially	1

Opcode	Instruction and operands	Effect	Nominal cost
2AH	JMPIFNOT A, BX	Assign the target to PC if R[A] is false; otherwise continue sequentially	1

1.11 Cycle accounting and execution boundaries

The instruction tables give **base charges**, not whole-program transfer rates. Before an admitted instruction executes, PC advances over its complete width and its base charge is deducted. Privilege, address alignment and write-ready handling occur after that base charge. There is no refund when a store blocks; a later retry incurs its base charge again.

An instruction's base charge is not subdivided at a device-event deadline. An admitted operation can carry system time past that deadline; due device work is then serviced against the advanced time. For example, a one-cycle LOADK followed by a successful two-cycle ordinary RAM LOAD_MEM consumes three cycles, even if a device deadline falls within that interval. A store whose destination is not ready follows the write-ready and retry rules. Eight-byte and counted accesses retain their specified ordered, potentially partial bus effects; the timing rule does not make them atomic.

Use the complete instruction charge when calculating a straight-line sequence. Use device status and interrupt protocols for synchronization, rather than assuming that a device request interrupts a partially executed instruction. Instruction costs do not specify one universal request-to-handler delay.

1.11.1 Selected instruction base charges

The CPU/system timebase is **33,868,800 cycles per second**. The following are selected instruction base charges, not a complete timing table:

Instruction	Base charge
Scalar LOAD_MEM and STORE_MEM	2 cycles
Scalar displaced forms LOAD_MEM_D and STORE_MEM_D	2 cycles
ADD, SUB, MUL	1 cycle
WIDE prefix	0 cycles

A WIDE instruction uses its body's base charge; the prefix does not add another base charge. These charges do not specify the cost of a source expression, total program execution time, or sustained memory bandwidth.

The accounting rules above define admission, retries and additional work; the memory instruction reference gives the complete transfer-instruction table. Book I, "Direct memory access", under "Transfer timing and sharing the bus" supplies endpoint rates, block and tail budgets, contention and GPU transport examples; CPU instruction charges are not DMA transfer rates.

1.11.2 Additional work and waiting

A scalar memory instruction costs 2 base cycles, including either four-byte or eight-byte numeric formats. A counted store adds (C+3)>>2 only after its write-ready preflight succeeds. These are instruction charges, not an electrical memory-bus bandwidth specification.

Counted words	Base	Sequence surcharge	Instruction total after ready preflight
0	2	0	2
1	2	1	3
4	2	1	3
5	2	2	4

The comparison skip, full-width branch and counted-store examples concern instruction charges, not a completed graphics or audio job. A builtin call has its additional builtin charge; device-specific command work and completion remain in the unit chapter.

A blocked store writes nothing while it waits for the owning device's matching ready edge. Waiting advances machine time toward scheduled events rather than repeatedly polling and re-executing the store. IRQ HALT and CPU-held intervals also let due device work advance. A HALT therefore has neither a fixed elapsed length nor the meaning of stopping every console device. The IRQ chapter supplies the pending/mask/acknowledgment and accepted-interrupt conditions.

1.12 CPU reset and program replacement

At CPU reset, Status is `00000001H`; Cause, EPC, BadAddress, LuaFaultReason and the exception-domain word are zero. Retained pre-NMI report words are also zero. Interrupt-wait, blocked-store, hard-halt and pending interrupt states are cleared. The CPU discards the active call stack and enters the system startup static function with no arguments.

The first PC comes from that startup function's record, not an assumed instruction address zero. Reset is not the same edge as `EXEC`: the latter replaces execution with a selected static function and sets the entry Status according to that function's execution domain. Neither statement implies that all RAM has been erased. The program startup reference supplies the reset vector, BIOS cartridge selection and section-initialization route; SYSTEM defines the separate software reset request and device-context transitions.

1.12.1 Hard halt and instruction exceptions

A reserved opcode, a WIDE body containing another WIDE, or an unreadable required function record causes a hard halt. EXEC also hard-halts when its record lacks STATIC. Execution and interrupt acceptance stop until reset. HALT, in contrast, is a resumable interrupt wait.

Misaligned instruction addresses and mapped instruction-read faults take the address-error or instruction-bus-error exception route. The scalar-memory reference separately defines ReservedInstruction for unsupported format fields.

1.13 CP0 and execution control

CP0 holds exception and privilege state. Current privilege comes from **Status bit 1**, not from the identity of the currently selected image. A clear bit means supervisor mode; a set bit means user mode. Status bit 0 is the current interrupt-enable bit. System entry initializes Status to 1; cartridge entry initializes it to 3.

`MFC0`, `MTC0`, and `RFE` require supervisor mode. In user mode they raise coprocessor-unusable before selecting a CP0 field, converting a source value, or checking whether `RFE` has an exception frame. `HALT`, branches, and the memory instructions have no corresponding current-mode check.

1.13.1 CP0 fields

For `MFC0` and `MTC0`, `A` names the value register and `B` is the raw CP0 field number, **not an RK operand**. Their handlers do not use `C`.

Field number	Name	`MFC0` result	`MTC0` effect in supervisor mode
8	`BadAddress`	Raw unsigned word as a Number	Ignored
9	`LuaFaultReason`	Raw unsigned word as a Number	Ignored
12	`Status`	Raw unsigned word as a Number	Replace the complete word, without masking
13	`Cause`	Raw unsigned word as a Number	Ignored
14	`EPC`	Raw unsigned word as a Number	Replace the complete word, without an alignment check
15	`EXEC`	Zero	Replace execution with the addressed static function

`MTC0` converts `R[A]` to an unsigned 32-bit word before applying the selected field's write behavior, including an ignored write. An unknown field number hard-halts in supervisor mode. This is not a coprocessor-unusable exception, and a rejected `MFC0` does not update its destination. A hard halt does not itself assign a new CP0 Cause.

1.13.2 Exception entry and cartridge dispatch

On an accepted maskable interrupt, Cause is `00000400H` and EPC is the interrupted instruction address. Entry shifts the low six Status bits left by two within that six-bit field, retaining higher Status bits. Current interrupt enable and current user mode are consequently clear. In word notation, the new Status is `(Status & ~0x3f) | ((Status << 2) & 0x3f)`.

The CPU enters the function selected by the ROM's IRQ or exception address as an exception frame with no supplied arguments. Lua-trap entry additionally places the error value in R0; other exception entries leave R0 nil.

The BLua control interface defines source access and the standard `irq` and `exception` callbacks. An ordinary callback returns to its entry routine, which issues RFE from the exception frame. A custom entry routine must provide its own dispatch and exception return. The BIOS monitor defines installed memory-fault service.

1.13.3 Exception return

In supervisor mode, `RFE` requires the current frame to be an exception frame; otherwise it hard-halts. It samples the active EPC, closes the exception frame's open upvalues, removes that frame, and restores the register-stack top to the removed frame's vararg base.

`RFE` replaces Status bits 0–3 with the former bits 2–5, retaining the higher bits. If a caller remains, its PC is assigned the sampled EPC. In particular, writing EPC before `RFE` changes this return target; `RFE` does not impose a separate alignment check on it.

Returning from an NMI-marked exception frame also restores the retained pre-NMI Cause, EPC, BadAddress, LuaFaultReason, and exception-domain words. The return PC still comes from the active EPC sampled before that restoration. The retained pre-NMI Status word is not restored wholesale.

1.13.4 EXEC

Writing `EXEC` supplies a **function-record address**, not an instruction-entry address. The record is read on the CPU bus and must identify a static function; otherwise execution hard-halts.

A successful `EXEC` clears the active call stack, selects the function's image, and starts the static function with zero arguments. It clears HALT, retained-interrupt-event, blocked-write and hard-halt state. The new Status is 3 if the selected execution-domain ID is nonnegative, or 1 otherwise. `EXEC` replaces execution: it is neither an ordinary function call with a saved return point nor an exception return.

1.13.5 HALT

`HALT` is available in user mode as well as supervisor mode. If an interrupt event is already retained, it consumes that event without creating a HALT condition. Otherwise it records the active thread and current frame depth; the HALT condition matches only while both still match. This is an IRQ-wait latch, not a program busy loop or a request for a fixed elapsed duration. Interrupt acceptance and wake scheduling follow the interrupt rules.

1.13.6 Control instruction reference

Opcode	Instruction and operands	Effect	Nominal cost
`26H`	`MFC0 A, B`	Supervisor only: copy CP0 field `B` into `R[A]` as an unsigned-word Number; EXEC reads zero; unknown fields hard-halt	1

Opcode	Instruction and operands	Effect	Nominal cost
27H	MTC0 A, B	Supervisor only: convert R[A] to an unsigned word and apply the CP0 field's write behavior; EXEC replaces execution; unknown fields hard-halt	1
34H	RFE	Supervisor only: return from the current exception frame using active EPC and the Status restoration above; a non-exception frame hard-halts	1
3FH	HALT	Consume a retained interrupt event, or record the current thread/frame-depth HALT condition	1

RFE and HALT do not use A, B, or C.

1.13.7 Read CP0 in a cartridge IRQ

Use this sequence after fresh BIOS cartridge entry, with the running beam unchanged and this program as the sole VBlank callback and IRQ-mask owner. There must be no supervisor request or device-context takeover. Entry is in user mode with interrupt enable set; main cannot read CP0 directly.

```
module<entry>
bss saved_status: word
bss saved_cause: word
bss saved_epc: word
bss services: word

function irq(flags)
    if (flags & 4) ~= 0 then
        mem[0x08000008] = 0
        *saved_status = cop0.status
        *saved_cause = cop0.cause
        *saved_epc = cop0.epc
        mem[0x08000004] = 4
        *services = *services + 1
    end
end

mem[0x08000008] = 0
mem[0x08000004] = 4
mem[0x08000008] = 4
while *services == 0 do
    halt_until_irq
end
```

Define the callback before enabling delivery. The initial mask and ACK discard old delivery and old pending VBlank; the final MASK write admits VBlank. In the callback, saved Status is 0000000CH, Cause is 00000400H, and EPC is the variable interrupted **instruction address**, not a function-record address. Full supplied flags contain VBlank and may contain other masked pending bits.

The callback returns normally. The standard IRQ entry performs RFE and main continues after the wait. Do not issue RFE from the callback's ordinary call frame. CPU supervisor privilege during IRQ does not by itself activate the SYSTEM supervisor device context; SYSTEM STATUS remains zero under these assumptions. Masking further delivery does not clear CPU interrupt enable or prevent later VBlank pending bits from being raised.

1.14 Memory faults and saved PC

A misaligned load raises `AddressErrorLoad`; a misaligned scalar or counted store raises `AddressErrorStore`. Both record the converted effective address in BadAddress before any mapped access. A newly raised bus fault produces `DataBusError`; that exception does not itself replace BadAddress.

For these synchronous instruction exceptions, EPC identifies the **start of the faulting instruction**, not its already advanced sequential PC. Common exception entry records Cause and the exception domain, shifts the low six Status bits left by two while retaining the higher bits, and creates an exception frame.

Bus-fault detail and CPU exception detection are distinct. The detailed bus-fault record retains the first unacknowledged fault, while each new fault event is still detectable by the CPU. A new `DataBusError` can therefore coexist with older latched detailed fault metadata until that record is acknowledged. A preexisting record alone does not make a later load or store fail.

2 BLua source programs and extensions

BLua provides the language facilities used to control the BMSX: named functions and modules, fixed byte layouts, typed address access, direct register access and interrupt waiting. This chapter specifies those forms and the source rules that differ from general Lua.

Use the hardware chapters for device register values and command sequences. Use the BIOS chapters for installed function calls. The source forms here select accesses and calls; the addressed device or called routine determines their effects.

Program-source extensions and BIOS runtime compilation are separate interfaces. The BIOS `load` service accepts the text language specified in BIOS runtime compilation, not every declaration and access form in this chapter. The CPU reference defines the instructions and execution behavior selected by these source forms.

2.1 BLua extensions

BLua provides explicit source forms for program linkage, static storage, typed address access, processor control, instrumentation, and augmented assignment. Hardware programming is a principal use, but not the limit of these extensions.

For general Lua syntax and introductory material, see the Lua documentation. The rules in this chapter apply where BLua differs. Local `<const>` has its familiar binding role; `module<const>` additionally selects the static namespace and call rules specified below.

Extension family	Source forms	Detailed reference
Module roles and static linkage	`module<entry>`, `module<const>`, literal `require`, static exports, `<init>`	Module modes, static imports, and `<init>`
Static storage and layouts	`struct`, `bss`, `data`, `rodata`, typed arrays	Typed storage, layouts, and address views
Typed address selection	`local p: *type`, `*p`, fields, indices, unary `&`, `sizeof`, `offsetof`, static-array #	Views, selection, and byte addresses
Six raw mapped-access forms	`mem`, `mem8`, `mem16le`, `mem32le`, `memf32le`, `memf64le`	Byte-addressed memory maps
Six control fields	`cop0.bad_address`, `cop0.lua_fault_reason`, `cop0.status`, `cop0.cause`, `cop0.epc`, `cop0.exec`	Control-register access
Three intrinsic operations	`blua32.closure`, `blua32.trace`, `blua32.trace_sink`	Intrinsic calls
Interrupt wait statement	`halt_until_irq`	Interrupt waiting
Six augmented assignments	`+=`, `-=`, `*=`, `/=`, `%=`, `^=`	Augmented assignment
Additional operator forms	Unary `&` on a String; named-member `->`	Unary `&` and named members

Memory-map names, `cop0` and `blua32` must be used in their direct source forms. Choose a mapped access for an explicit address and width, or a typed view when field offsets and element strides belong to a declared layout. Unary `*` reads a typed selection; the view itself occupies no new byte range.

The source-semantics reference records literal spellings, local binding, result adjustment and control-flow rules that must not be inferred from another Lua edition.

2.2 Typed storage, layouts, and address views

Storage declarations define byte ranges for scalar state, transfer buffers, lookup tables and structured records. A storage name supplies its byte address. Field selection and indexing apply the declared layout to that address; scalar dereference reads or writes the selected bytes.

Use addressed storage when an operation needs a byte address or a prescribed binary layout: for example, a transfer buffer, a record exchanged with a device, or a lookup table read by numeric offset. Ordinary locals and Tables hold language values instead. A Table's indexed values are not a declaration of consecutive bytes, and assigning a storage address to a local does not copy those bytes into the local.

2.2.1 Choosing a storage section

`bss` reserves writable RAM that is cleared to zero during section initialization. `data` reserves writable RAM whose initial bytes are copied from the program's ROM data. `rodata` places initialized bytes in ROM.

Choose BSS for scratch space and state that starts at zero, DATA for mutable state with an initial value, and RODATA for fixed lookup data. These are program storage sections: an ordinary function call does not allocate another copy or initialize them again.

```
bss scratch: u8[16]
bss total: word
data counter: word = 3
rodata levels: u16[] = { 10, 20, 30 }
rodata title: string = "ready"

*total = *counter
local previous = *counter
*counter = previous + 1
```

After these operations, `previous` and the contents of `total` are 3, while the contents of `counter` are 4. The Number `previous` is a loaded value; `counter` and `total` name the byte addresses of the two stored words. `scratch` reserves 16 bytes; `levels` holds three consecutive two-byte values.

A bare storage name, including `counter` or `total`, evaluates to its linked numeric address. Use `*counter` to load its scalar contents and `*counter = value` to store them. Assigning a new value directly to a `bss` or `data` name is rejected; the name is not a rebindable scalar variable.

Section initialization copies `data` bytes into RAM and then clears `bss`, using word accesses. It precedes static module initializers and the entry routine. Locate writable program sections outside the installed firmware's RAM reservation. The standard placement puts program DATA after firmware BSS, then program BSS after DATA. Access declarations by their resolved addresses; fixed offsets from the start of RAM can overwrite firmware state. See Book I, "Memory and bus access", under "Reserved and cartridge storage in main RAM" for the firmware footprint and placement rules.

Each declaration starts at its type's alignment, and the section cursor after a declaration is rounded to four bytes. Two separate `u8` declarations therefore need not occupy adjacent bytes. Use one array or struct when the relative spacing of bytes matters.

2.3 Worked static storage

Prepare two writable records from fixed input records, then update an initialized counter. Each record has a tag, a 32-bit value and two 16-bit samples. RODATA supplies the inputs, DATA supplies the counter's initial value, and BSS supplies the destination bytes. The example connects these choices to field addresses and to the resulting words in RAM.

Exporting a storage name likewise exports its numeric address, not a copy of its bytes.

2.3.1 Worked example: prepare and copy two records

This sequence prepares bytes without submitting a device job. Start after normal cartridge DATA/BSS initialization. The declared objects supply the source and destination addresses.

```
module<entry>
struct memory_sample
    tag: u8
    value: word
    samples: u16[2]
end
rodata inputs: memory_sample[2] = {
    { tag = 0x11, value = 0x12345678, samples = { 0x1122, 0x3344 } },
    { tag = 0x22, value = 0x9abcdef0, samples = { 0x5566, 0x7788 } },
}
data counter: word = 0x12345678
bss output: memory_sample[2]
local source<const>: *memory_sample = inputs
local destination<const>: *memory_sample = output

for index = 0, #inputs - 1 do
    destination[index].tag = source[index].tag
    destination[index].value = source[index].value
    destination[index].samples[0] = source[index].samples[0]
    destination[index].samples[1] = source[index].samples[1]
end
*counter = *counter + #inputs
```

2.3.1.1 Record indexing is not byte arithmetic

Part of `memory_sample`	Byte offset	Bytes
`tag`	0	1
Padding	1	3
`value`	4	4
`samples`	8	4

Record alignment is 4 and record size is 12 bytes. Each two-record array occupies 24 bytes. Typed indices start at zero: `&source[1]` is 12 bytes after `inputs`, `&source[0].value` is 4 bytes after it, and `&source[1].samples[1]` is 22 bytes after it. Advancing a byte view by one would instead select the next byte, here padding, not the next record.

2.3.1.2 Resulting bytes

BSS supplies initial zero output bytes. The copy writes scalar fields rather than assigning whole records, so their three padding bytes remain zero.

Output byte offset	Word	Contents
0	`00000011H`	First tag and three zero padding bytes
4	`12345678H`	First value
8	`33441122H`	First record’s two samples
12	`00000022H`	Second tag and three zero padding bytes
16	`9ABCDEF0H`	Second value
20	`77885566H`	Second record’s two samples

The final counter is `1234567AH`: `#inputs` counts **two records**, not bytes. No interrupt, rendering or buffer-release event follows this CPU copy. A hardware-facing use must still satisfy the receiving unit’s address, layout, count, completion and lifetime rules.

2.3.2 Typed byte storage and address views

Use byte-addressed storage for the records, command words and sample bytes that hardware consumes. BLua's `bss`, `data` and `rodata` declarations define addressed storage objects; a local typed view supplies an access layout for an existing address. Neither a local value nor an ordinary Table is a device buffer.

The declaration and layout reference defines declarations, initializers, sizes, alignment, arrays and structs, including `sizeof`, `offsetof`, address-of and zero-based indexing. Book I, "Memory and bus access" defines what happens when the resulting address reaches RAM, ROM or an I/O port.

For example, a word view of an existing DMA register bank accesses consecutive four-byte ports. It does not reserve a DMA channel or alter the port layout:

```
local dma0<const>: *word = 0x0800000c
local remaining = dma0[2]
```

The read reaches TRANSFER_COUNT at `08000014H`. By contrast, `dma0 + 2` is the byte address `0800000EH`, not that third port. Use an indexed typed access or an explicit byte offset that preserves the device's required alignment.

A storage address still carries the region's ownership and lifetime rules. Keep cartridge objects outside the reserved main-RAM ranges, use ROM as a read-only byte source, and keep a device's input bytes unchanged until its completion condition releases them. The record copy above only prepares bytes; Book I, "Memory and bus access", under "DMA buffer lifetime" and the unit references cover asynchronous use.

2.3.3 Language execution storage and byte RAM

The record-copy example's `output` is twenty-four addressed RAM bytes. A Table used to organize two records is a language value with its own identity; it is not that byte array. A local variable holding an address also differs from the storage reached through the address. Keep these roles separate when supplying hardware operands or budgeting writable storage.

Calls use the CPU's frame value slots, not a cartridge-addressable byte stack. The function record's maximum stack counts value slots rather than bytes.

Language values use a shared allocation budget, described under CPU — Language allocation and lifetime. That budget does not describe a mapped byte-RAM heap or report free RAM bytes. Mapped DATA/BSS placement and the language budget are separate constraints: placing a static byte buffer neither allocates a language Table nor yields a byte address for the CPU's value slots.

2.3.3.1 Values are not byte buffers

A language value can hold a Number, text, a Table, a function or a Thread. Copying a Table or function value preserves its identity; it does not copy its contents into main RAM. A device needs the numeric address of storage containing its prescribed bytes, not the Table that the program uses to organize them.

Use typed static storage and scalar writes to construct that byte layout. Loads return numeric scalars under the selected access format; they do not reconstruct arbitrary language values from words. Integer stores retain the specified low bits, while floating stores encode floating-point bits. The Book I, "Memory and bus access", under "Scalar memory accesses" give the alignment, conversion and partial-access behavior.

2.3.3.2 Function arguments, results and BIOS calls

Passing an address to a function passes its numeric value. It neither copies the pointed-to bytes nor extends their lifetime. A callee can establish a typed view of that address before accessing it, as in the example below. Ordinary calls use value slots, not a cartridge-addressable byte stack.

Use Calling and returning values for argument/result placement and closure capture, and BLua source semantics for source-level varargs and result expansion. Named static BIOS targets and installed global library values are distinct call routes; see Named BIOS function imports. Calling either kind of BIOS function is an ordinary call, not EXEC or a privilege transition. Its documented memory and device requirements still apply.

2.3.3.3 Passing an address to a function

After normal BIOS entry, typed storage supplies the destination bytes; passing its address to a function does not allocate a new destination:

```
bss written: word[2]
local write_pair = function(address, first, second)
    local destination<const>: *word = address
    destination[0] = first
    destination[1] = second
    return first + second, 2
end
local sum, count = write_pair(written, 17, 25)
```

`sum` is 42, `count` is 2, and the two stored words are 17 and 25. The function's address view and the declared array select the same storage. Neither a Table nor a String can be substituted for its byte address.

2.4 Declaration and layout reference

The following forms use lowercase placeholder names; source identifiers cannot contain uppercase letters.

Form	Meaning
`struct record ... end`	Define a layout with fields written as `name: type_name`.
`bss buffer: type_name`	Reserve zero-initialized writable storage; no initializer is accepted.
`data buffer: type_name = initializer`	Define writable storage with a compile-time initializer.
`rodata buffer: type_name = initializer`	Define image-resident storage with a compile-time initializer.
`type_name[n][m]`	Array type with explicit dimensions.
`type_name[]`	Infer the outer dimension of initialized `data` or `rodata`.
`local view: *type_name = address`	Associate a local address value with an access layout.
`local view<const>: *type_name = address`	The same view with an immutable local binding.
`*view` / `*view = value`	Read or write a selected scalar through a typed address.
`view.field` / `view[index]`	Select a field or element using its layout.
`&view.field` / `&view[index]`	Obtain a selected address without loading its value.
`sizeof(type_name)`	Obtain a type's size in bytes; array dimensions are allowed.
`offsetof(record.field)`	Obtain a field's byte offset; nested field paths are allowed.
`#buffer`	Obtain the outer element count of a declared static array.

Struct fields may be separated by commas or semicolons; a separator is not required between successive field declarations. Storage declarations accept an optional trailing semicolon. The declaration spellings are contextual forms, not additions to the lexical reserved-keyword list.

2.4.1 Primitive storage types

Every size and alignment below is in **bytes**. An alignment of four means that the address must be divisible by four.

Type names	Size	Alignment	Stored representation and access
`u8`, `i8`	1	1	Unsigned byte read; stores keep the low 8 bits.
`u16`, `i16`	2	2	Unsigned little-endian read; stores keep the low 16 bits.

Type names	Size	Alignment	Stored representation and access
`u32`, `i32`	4	4	Unsigned little-endian read; stores keep the low 32 bits.
`f32`	4	4	IEEE float32; stores round the numeric value to float32.
`f64`	8	4	IEEE float64 accessed as two little-endian 32-bit words.
`addr`	4	4	Raw unsigned 32-bit address value, accessed like `u32`.
`word`	4	4	Raw unsigned 32-bit word exposed as a numeric value.
`string`	4	4	Image constant-pool reference; reads the selected constant value.

`i8`, `i16` and `i32` have the same **unsigned load** behavior as their `u` counterparts. Sign extension must be explicit when a stored field represents a signed quantity. `word` holds a numeric register word, and `addr` holds a numeric address; select the target layout with an annotated view.

To recover an eight-bit signed quantity, shift the loaded word left by 24 bits, then arithmetically right by 24. Use shifts of 16 for a sixteen-bit quantity; `raw | 0` gives the signed interpretation of a complete thirty-two-bit word. See bitwise operations and shifts.

```
bss delta: i8
*delta = -3
local raw = *delta                -- 253
local signed = (raw << 24) >> 24  -- -3
```

A `string` field holds a constant reference. Its RODATA initializer selects a String constant, and a read returns the referenced value. It is separate from inline text bytes; use byte storage and an explicit decoder for encoded text.

2.4.2 Initializer rules

Initializer table syntax describes bytes to encode at compile time, not a table allocated for later use. Integer storage initializers must evaluate to compile-time integers. A static initializer is not an arbitrary runtime expression.

```
struct settings
    enabled: u8
    period: u16
end

data presets: settings[2] = {
    { enabled = 1, period = 60 },
    { enabled = 0, period = 120 },
}
data matrix: u8[2][3] = {
    { 1, 2, 3 },
    { 4, 5, 6 },
}
```

- Arrays require positional elements and exactly the declared outer element count. Each further array dimension requires another nested initializer.
- Struct initializers require identifier-named fields. Missing, duplicate, or unknown fields are compilation errors.

- Padding bytes in encoded initializers start at zero.
- An explicit dimension must be a positive compile-time integer.
- Only the outer dimension of initialized `data` or `rodata` may be inferred from the initializer's field count, and that count must be nonzero. There is no inference source for `bss`, struct-field, inner-dimension, or local-view `[]`.
- Static `string` initializers must be compile-time strings and are supported only in `rodata`, including string fields inside aggregates. A `string` initializer in `data` is rejected.
- `f32` and `f64` initializers in `data` and `rodata` must be compile-time numbers. They encode IEEE 754 binary32 or binary64 in little-endian order, including inside arrays and structs. `f32` rounds the computed value to binary32; `f64` retains its binary64 value. Both preserve negative zero.

2.4.3 Computing struct and array layouts

Fields are placed in declaration order. Before placing each field, its offset is rounded up to that field's alignment. The struct's alignment is the greatest field alignment; its final size is rounded up to that alignment. Nested structs use their own resolved sizes and alignments. An array retains its element type's alignment and multiplies its size by every declared dimension. There is no packed-layout declaration form.

The following record combines a byte tag, a 32-bit value and a floating-point weight. Its field offsets determine both the record stride and the addresses selected by a typed view:

```
struct sample
    tag: u8
    value: u32
    weight: f64
end

bss samples: sample[2]
local current<const>: *sample = samples
current[0].tag = 3
current[0].value = 7
current[0].weight = 1.5
local value = current[0].value
local second_weight = &current[1].weight
local sample_bytes = sizeof(sample)       -- 16
local array_bytes = sizeof(sample[2])     -- 32
local weight_offset = offsetof(sample.weight) -- 8
local count = #samples                    -- 2
```

Part	Byte offset	Size	Alignment
`tag`	0	1	1
Padding	1	3	—
`value`	4	4	4
`weight`	8	8	4
Complete `sample`	0	16	4

The second record starts 16 bytes after `samples`. Its `weight` address is therefore `samples + 24`, not `samples + 8` or `samples + 2`. The `f64` field requires four-byte, not eight-byte, alignment.

Unknown types, unknown fields, duplicate field declarations, and recursive layouts are compilation errors. Type declarations have block-wide forward visibility, with lookup through enclosing scopes and then globals. Storage value references remain ordered; forward visibility of a type does not make every storage value available early.

2.4.4 Views, selection, and byte addresses

A typed local view supplies a layout for an existing address. Declare it with `: *type_name`, then select fields or elements in the same way as for declared storage. The addressed bytes keep their existing lifetime and access permissions; creating the view reserves no storage and copies no record.

In the section-choice example, `local current<const>: *word = counter` selects word accesses at that address. `*current` reads the stored 4; `&current` obtains its address without a read. The annotation selects how an access is made, while `<const>` prevents rebinding `current` to another address. It still permits `*current = 5`. The address, its stored value and the view's binding are three separate things.

Indexing is **zero-based**. This is different from the one-based sequence keys used by the Table library. A typed array describes consecutive bytes; an ordinary Table is a collection of language values keyed by values. For a view of `u16`, `view[1]` selects the address two bytes after `view`. For a view of `sample`, `view[1]` selects the address `sizeof(sample)` bytes after it. For a multidimensional array, each index removes one dimension and uses the size of the remaining array as the stride. Thus `matrix[1][2]` selects byte offset five in the `u8[2][3]` example.

Numeric address addition is different: `view + 1` adds **one byte**, regardless of the view's type. Selection offsets, `sizeof`, and `offsetof` all use bytes. A device register's explicitly specified word count is a device unit, not an automatic consequence of pointer typing.

Storage names and typed local roots permit pointer-style indexing even when their declared type is a scalar or struct. After indexing, field selection, or dereference, that permission does not continue unless the selected type is an array. Dereference retains the selected type: it neither removes an array dimension nor automatically follows an address stored in a field. Whole arrays and structs are address ranges, not scalar values that can be loaded or assigned as complete aggregates. Select a scalar field or element, or use `&` to obtain the selected address.

An `addr` field loads a numeric address only. Create an explicitly annotated local view before selecting the target layout. The same rule applies to scalar address storage:

```
bss target_address: addr
bss payload: u8[4]
*target_address = payload
local target: *u8 = *target_address
target[0] = 65
```

Unary `&` on typed storage or a typed selection computes its address without reading its contents. A bare storage name already provides its base address. This is not general address-of for arbitrary ordinary locals, and unary `*` requires resolved typed storage or a typed view rather than an arbitrary numeric expression.

2.4.5 Sizes, offsets, and counts

`sizeof` takes a **type**, not a storage value: use `sizeof(sample[2])`, not `sizeof(samples)`. `offsetof` takes a struct type followed by named fields, such as `offsetof(outer.inner.field)`. It adds the selected byte offsets; intermediate fields must be non-array structs. An indexed path such as `offsetof(outer.items[1].field)` is not accepted.

`#samples` is the declared array's **outer element count**, not its byte size. For `matrix: u8[2][3]`, the count is two, not six. This static-array form also recognizes resolved static storage exports, but does not provide capacity for arbitrary typed locals or selected fields. Do not use `#view` as a pointer-capacity query.

2.4.6 Access hazards

Typed indexing emits no array bounds check or runtime integer-index check. A statically known noninteger index is rejected. For computed indices, the programmer must keep the resulting byte address within the intended range.

Effective access addresses are converted to unsigned 32-bit addresses. Misaligned accesses cause a synchronous address exception before the mapped access: two-byte alignment for `u16`/`i16`, four-byte alignment for `u32`/`i32`, `word`, `addr`, `f32`, and `f64`. Byte accesses have no additional alignment requirement. A bus fault causes a synchronous data-bus exception; a failed load leaves its destination register unchanged. A store can wait for the mapped destination to become ready before writing.

Direct writes through `rodata` selections are rejected, as are static string-reference assignments and whole-array or whole-struct assignments. A local view of a RODATA address permits store syntax, so the mapped memory permissions still govern a write through that view. `<const>` fixes the local address binding while allowing stores through it.

Typed accesses to memory-mapped I/O invoke the mapped device rather than behaving like inert RAM. Word accesses are not interchangeable with narrow accesses: an aligned I/O byte read reads the slot and keeps its low byte, other byte positions fault, byte writes fault, and 16-bit reads and writes fault. Use the device's specified access width; writes to read-only slots fault.

An `f64` access performs its low-word access before its high-word access at byte offset four. If the first access faults, the second is skipped. The pair is not atomic: the first write may already have occurred when the second faults.

2.5 Operator spellings and source conventions

2.5.1 Augmented assignment

BLua accepts six augmented assignments:

Assignment	Arithmetic operator
`+=`	`+`
`-=`	`-`
`*=`	`*`
`/=`	`/`
`%=`	`%`
`^=`	`^`

Each requires exactly one target and one right-hand expression. There is no `//=`, bitwise or shift assignment, concatenation assignment, or increment or decrement form. In particular, `--` begins a comment, not a decrement.

Evaluation prepares the target once, evaluates the right-hand expression, reads the target, performs the arithmetic and writes the result. This is a read followed by a write. Device read effects and write effects both apply; use explicit register values for commands whose fields are not suitable for read-modify-write operation.

2.5.2 Unary `&`: storage address or String

For a typed storage selection, unary `&` obtains the selected storage address without reading its contents. A record field or indexed element therefore selects an address using its declared layout.

For a String operand, unary `&` returns the String value unchanged. Use String operations on that result. Only the typed-storage form yields a byte address; a String operand supplies neither a character-buffer address nor a numeric identifier.

Postfix field selection, indexing and calls bind before unary `*` and `&`. These unary forms have the same precedence as unary `-`, `not`, `#` and `~`: exponentiation binds more tightly, while multiplication and the other binary arithmetic operators bind less tightly. Thus `*current + 4` adds four to the loaded value, and `&samples[0].weight` selects a field before taking its address. Use `(*current)^2` to square a loaded scalar rather than making the power expression the operand of the dereference.

2.5.3 Named members with `->`

`base->member` is another named-member spelling alongside `base.member`. It is not C's automatic extra pointer dereference.

For a typed field selection, the field's address is the base's typed address plus the declared field offset. The arrow does not first read an address-valued field and turn its contents into a new typed pointer. Choose the intended address view explicitly rather than expecting C pointer-member behavior.

2.5.4 Names and a line-leading `*`

Identifiers begin with lowercase ASCII `a`–`z`, `_`, or `$`; following characters may also include decimal digits. Uppercase ASCII letters in identifiers are rejected, and non-ASCII letters are not identifier characters. Uppercase hexadecimal digits inside numeric literals are a separate matter.

Statements generally need no semicolon, but newlines are not universally insignificant. When `*` begins a new line, it does not continue the preceding expression as multiplication: it can begin a dereference statement instead. Keep a continuation multiplication operator on the existing expression line. Parentheses do not remove this particular newline rule; do not generalize it to every operator.

2.6 Module modes, static imports, and `<init>`

A module publishes either an ordinary value or static exports. Use an ordinary value for mutable state and replaceable function fields. Use static exports for named constants, storage addresses and directly resolved calls.

This choice determines what an importer receives. An ordinary export can be retained, passed to other functions and changed through its runtime fields. A static namespace instead resolves named selections: a storage export gives its address, a constant gives its value, and a static function export supplies a call target. Importing a namespace does not create an object containing those exports. The import path names that interface, independently of the local name used for it.

2.6.1 Choose the module mode

Put an optional module marker before the chunk's statements. The spellings are case-sensitive; an optional semicolon may follow the marker.

Beginning of chunk	Meaning
No marker	An ordinary module; a final export declaration supplies its runtime export.
`module<entry>`	The program's entry chunk, entered after its ordinary dependencies are initialized.
`module<const>`	A compile-time export namespace or root static function, not an allocated module table.

A program has one entry chunk. `module<entry>` declares that role. Each module also has a distinct path using `/`; dots remain literal path characters. Use that path in imports. The Book I, "Memory and bus access" and Book III, "Cartridge sockets and mapped card devices", under "Selection and execution" are separate from module naming. Program images and startup specifies program entry and initialization.

2.6.2 Import a literal module path

Use the built-in `require` with exactly one literal string argument:

```
local state = require("lib/state")
local settings <const> = require("lib/settings")
```

The first form is suitable for an ordinary runtime export; the second can bind a compile-time namespace. `require "lib/state"` is also accepted. A computed path, an extra argument, or a method-style built-in import is a compile error. A lexical declaration named `require` shadows the built-in.

Imports name supplied modules or installed named function imports. An unavailable module is a compile error, not a missing-file error during play. Module names use `/`; dots are not converted to `/`. An import variable's name does not change the imported module's path.

Literal imports select dependencies for the program, including imports written inside functions. They are not deferred file-loading instructions. Ordinary dependencies initialize before their importers; a dependency cycle is a compile error. Do not rely on filename ordering between unrelated dependencies.

2.6.3 Worked module linkage

An ordinary module named `counter` can publish a mutable runtime object:

```
local counter = { value = 10 }
function counter.advance(amount)
    counter.value += amount
    return counter.value
end
return counter
```

Its importer begins after that top-level initialization:

```
module<entry>
local first = require('counter')
local second = require('counter')
local before = first.advance(2)
local after = second.advance(3)
```

`before` is 12 and `after` is 15. Both imports select the same export, not two instances. Neither written import reruns the initializer. A dependency written inside a function is still initialized before entry; imports express dependency order rather than deferred loading.

2.6.4 Ordinary exports stay runtime values

An ordinary module declares its export with a final top-level `return` containing exactly one expression. Its result may be a table, function, scalar, `false`, or `nil`. For `return make_value()`, only the call's first result is exported. An earlier or nested return, bare return, or final multiple-expression return does not declare that export contract.

The module's top-level code is initialized before entry; an ordinary `require` expression reads its retained export rather than rerunning that code at the written import line. Successful initialization is recorded separately from the export, so exporting `nil` or `false` does not cause another initialization. An ordinary module with no final single-expression export supplies `true`; additional requested import results are nil. An earlier return does not select an alternative published export.

Imports of an ordinary exported table read the same published root. Its fields remain runtime fields: replacing a function field changes what a later ordinary `module.field()` lookup calls. A const local naming that table does not turn the table into a compile-time namespace or freeze its fields.

2.6.5 A const namespace is not a const local

`local name <const>` makes a local binding immutable. `module<const>` instead declares compile-time exports: named constant leaves and static function targets, or a root static function. These are different features. A recognized const module has no ordinary runtime initializer.

For example, a const module named `lib/settings` can declare:

```
module<const>
return {
    step = 8,
    scale = function(value)
        return value * 8
    end,
}
```

Its named exports can be selected without producing a runtime module table:

```
local settings <const> = require("lib/settings")
local step = settings.step
local scaled = settings.scale(step)
```

Named fields and literal string keys form export paths; nested fields form nested namespaces. `settings["scale"](step)` is also a static selection. Computed indices and positional array entries are not named namespace export paths.

Constant leaves may be `nil`, booleans, numbers, strings, or static storage addresses. Numeric negation and supported numeric constant expressions can be folded; arbitrary runtime expressions are not thereby constant expressions. A named leaf that is neither a compile-time constant nor a static function target is a compile error.

2.6.6 Static functions are call targets, not function values

Const modules may export function literals, top-level local function declarations, or top-level const locals initialized with function literals. A root export can be written as `return function(value) return value * 8 end`. Bind such a root with `local operation <const> = require("lib/operation")`, then call `operation(value)`. A named target can have an alias such as `local scale <const> = settings.scale`.

A compile-time namespace cannot be used as a runtime table. Select its exports instead. A static function target cannot be stored in a runtime table, passed as an ordinary callback, or returned as a runtime function value. A constant value export cannot be called. These are compile errors, not runtime conversions. Installed named function imports use the same call-target distinction. Exported static functions cannot capture runtime outer locals or use dynamic call targets. Their operations are restricted to scalar/static work: numeric and boolean constants, scalar locals, branches, static calls, and scalar memory access. Runtime global access, table allocation or dispatch, object length, runtime closure creation, vararg dispatch, and dynamic string concatenation are excluded. Ordinary string literals in such function bodies are also excluded; already interned constants read through immutable typed rodata are permitted.

2.6.7 Preparation participants: `<init>`

A program can collect local preparation functions into one callable sequence. Mark each function with `<init>` so a later sequence call can run them in dependency order while retaining access to their captured local state.

The declaration form is a top-level local function with no parameters or varargs:

```
module<entry>
local count = 0
local function prepare <init>()
    count = 0
end
prepare()
```

The explicit `prepare()` call runs the body at that source position. `<init>` marks participation in the separate preparation sequence. The participant is an ordinary local closure and may capture local state. Declarations inside `if`, loops, `do`, or other functions are not top-level participants. `<init>` is forbidden in a const module and is not an attribute on global or method declarations. A non-entry participant module must be statically required.

Normal startup establishes DATA and BSS and initializes ordinary dependencies before entering the entry chunk, as specified in Program images and startup. It does not invoke the participant sequence. Use an explicit call when the body is needed during ordinary startup.

When the separate participant sequence is invoked, it first completes any pending ordinary-module initialization, then calls participants with zero arguments. Dependency-module participants precede importer-module participants; entry participants are last, with declaration order retained within each module. Each invocation runs that participant order again. DATA, BSS and the entry chunk keep their existing state during the sequence. Use the explicit startup call for preparation required at normal entry; the annotation alone supplies no startup call or once-only latch.

The executable header publishes the preparation sequence's absolute function-record address at byte offset `0x5c`. Zero means that no sequence is present. A sequence can contain only module initialization when the program has dependencies but no annotated participants. For a nonzero address, construct a closure with `blua32.closure` and call it with no arguments. This is an ordinary call that returns to its caller; it does not require EXEC or a privilege change. It returns one nil value; participant return values are discarded. A language error escaping a participant propagates through the call and prevents later participants in that invocation from running.

Continuing after the declaration above, a cartridge can invoke the complete sequence as follows. The ordinary data selection must select that cartridge when reading its ROM header:

```
local image<const> = 0x10000000 + mem32le[0x10000020]
local participants<const> = blua32.closure(mem32le[image + 0x5c])
count = 5
participants()
-- prepare runs through the sequence; count is now 0.
```

Declare the entry's participants before making this call so their closures have been established. Dependency modules are initialized before the entry chunk. The sequence address designates this program's participants; it does not select another cartridge's or the BIOS's participants.

2.7 Named BIOS function imports

The named BIOS function imports are:

Function-table slot	Module path	Arguments and returned values	Routine reference
0	`math/sin`	One turn-phase word; returns one signed integer Q16 sine.	Turn-phase sine
1	`string/float/decode`	High word, low word and byte count; returns one Number decoded from the supplied floating-point words.	Floating-point word decoder
2	`gpu/system_vram_region`	No arguments; returns packed origin and extent words for the BIOS VRAM reservation.	Book I, "Memory and bus access", under "Firmware reservation in VRAM"

Each path names a root static function. Import and call them as follows:

```
local sin_turn32<const> = require('math/sin')
local decode_float<const> = require('string/float/decode')
local system_vram_region<const> = require('gpu/system_vram_region')

local sine = sin_turn32(0x40000000)                -- 65536: +1 in Q16
local decoded = decode_float(0x3fa00000, 0, 4)    -- 1.25
local origin<const>, size<const> = system_vram_region()
```

`math.sin(angle)` is not the turn-phase import: the ordinary Math-library function takes radians and returns an unscaled Number. Likewise, the floating word decoder does not read a String or a memory address. Follow the routine references for complete units, field packing and result contracts.

These paths are reserved published BIOS names. Importing one selects its static call target; it leaves ordinary program-module initialization unchanged. A program module cannot redefine one of these paths. Other BIOS libraries use their installed globals unless a named import is listed.

The program-image reference specifies the published slots and the ROM fields used to locate these function records. The source import selects that public target; it does not supply a private copy of the BIOS or its global bindings.

2.8 Direct access forms and intrinsics

BLua provides direct memory access, control-register access, closure construction, selectable tracing, and an interrupt-wait statement. These are language forms, not a library of replaceable tables and functions.

The six memory roots are `mem`, `mem8`, `mem16le`, `mem32le`, `memf32le`, and `memf64le`. The two intrinsic roots are `cop0` and `blua32`. None of these eight names may be declared as a local or parameter. They are not first-class values: do not assign a root to another variable, pass it as an argument, or replace it by assignment. Their supported forms use the root name directly, as described below.

2.8.1 Byte-addressed memory maps

Each memory map uses `map[byte_address]` for a read or an assignment target. The index is a byte address, regardless of the access width. For example, adjacent 32-bit words are four address units apart, not one. Addresses are consumed as unsigned 32-bit addresses.

Map	Value transferred	Size in bytes	Required byte alignment
`mem[address]`	Raw unsigned 32-bit word, little-endian	4	4
`mem8[address]`	Unsigned byte	1	1
`mem16le[address]`	Unsigned 16-bit integer, little-endian	2	2
`mem32le[address]`	Unsigned 32-bit integer, little-endian	4	4
`memf32le[address]`	Numeric IEEE float32, little-endian	4	4
`memf64le[address]`	Numeric IEEE float64, little-endian	8	4

`mem` and `mem32le` both transfer a little-endian unsigned 32-bit value. They remain distinct access kinds, including their fault-access identity; `mem32le` is not a source alias for `mem`. The floating-point maps convert between numeric values and their stored floating representation. They do not return the integer bit pattern of that representation. Use an integer map when the required result is raw bits.

The following fragments show access shapes. Their address variables must already refer to suitable memory locations; the fragments are not complete programs.

```
local raw_word = mem[word_address]
mem[word_address] = 0x00000001
mem8[byte_address] = 0x7f
local halfword = mem16le[halfword_address]
local bits = mem32le[word_address]
memf32le[float_address] = 1.5
local wide = memf64le[double_address]
```

Misalignment raises an address-error-load or address-error-store exception before the access. In particular, `memf64le` requires four-byte, not eight-byte, alignment. It transfers the low word first and the high word at the address plus four. A fault stops the sequence; the two-word transfer is not an atomic transaction. A fault on the second word of a write does not undo the first word.

A mapped device retains its own access rules. An access width is not permission to slice a device register as though it were ordinary byte storage. For example, a `mem8` I/O read must address an I/O slot and returns that slot's low eight bits; a byte address between slots faults. `mem16le` I/O accesses fault even when their two-byte alignment is correct. Word and unsigned 32-bit I/O accesses use the addressed device slot. Mapped bus faults raise synchronous data-bus-error exceptions. A device that holds a store pending completes the access when it becomes ready; the processor waits for that access rather than requiring repeated writes.

2.8.2 Typed dereference is a separate form

Unary `*` reads a resolved typed pointer, and `*pointer = value` writes its resolved scalar storage. The pointer's scalar type selects the access kind; the resolved byte offset and read-only status remain part of the access. Whole-structure scalar reads and assignments are not supported by this form, and directly resolved RODATA selections cannot be assigned through it. A local typed view does not retain that RODATA protection; the storage section above distinguishes these cases.

An untyped numeric address is not a dereferenceable pointer. Do not replace `mem[address]` with `*address` when `address` has no compiled pointer type, and do not use `*0x08000000` as a raw memory access. Typed-pointer declarations, storage layout and address views are specified under Typed storage, layouts, and address views.

2.8.3 Compound assignment performs a read and a write

The augmented-assignment spellings also apply to mapped-memory targets. Each performs both accesses through the selected map.

For a mapped-memory target, evaluation proceeds in this order:

1. Evaluate and retain the target address.
2. Evaluate the right-hand expression.
3. Read the target through the selected memory map.
4. Apply the arithmetic operation and write through that map.

Thus `mem[address] += amount` performs an actual mapped read followed by an actual mapped write. It is not a device-atomic operation. Read-side effects, write readiness, and access faults still apply separately. For device registers, use the device-defined write value rather than assuming that reading and rewriting its previous value is harmless.

2.8.4 Control registers: `cop0`

Use a direct field read or assignment, such as `cop0.status` or `cop0.epc = resume_address`. The complete source-visible field set is:

Field	Register number	Permitted source access
`cop0.bad_address`	8	Read only
`cop0.lua_fault_reason`	9	Read only
`cop0.status`	12	Read and write; compound assignment permitted
`cop0.cause`	13	Read only
`cop0.epc`	14	Read and write; compound assignment permitted
`cop0.exec`	15	Write only; compound assignment rejected

Other field names are rejected. Reading `cop0.exec` or assigning a read-only field is also rejected, rather than providing another control-register operation. Every permitted read and write still requires supervisor mode. In user mode it raises a coprocessor-unusable synchronous exception; ordinary program code does not gain privileged access by using this syntax.

`cop0.status` and `cop0.epc` writes supply raw unsigned 32-bit words. In `status`, bit 0 is current interrupt enable and bit 1 is current user mode. Writing `cop0.exec` requests execution of the supplied function-record address; it is not an ordinary field update or a substitute for a normal function call. The execution-transfer and exception contracts belong to the CPU reference.

2.8.4.1 Source access and control instructions

A permitted read emits `MFC0`; an assignment emits `MTC0`. Source access restrictions differ from the raw privileged instructions: the latter ignore writes to read-only fields and read zero from EXEC. The source restrictions do not introduce another hardware permission fault.

A supervisor-only source fragment can read exception state and set a resume address:

```
local status = cop0.status
local cause = cop0.cause
cop0.epc = resume_address
cop0.status = status | 1
```

Compound assignment to `status` or `epc` evaluates the right-hand expression, then reads the control register, applies the operation, and writes the result. These are separate control-register operations, not an atomic update.

The installed monitor can save `cop0.epc` in a local and restore it before exception return. This requires supervisor context; ordinary user-mode entry cannot read CP0. At boot, BIOS writes `cop0.exec = startup`, where `startup` is the static function-record address supplied by the ROM header, not an instruction address or an ordinary function value.

The standard BLua IRQ entry reads FLAGS, calls `irq(flags)` when it is nonzero, and executes RFE. The standard exception entry calls `exception(error_value)` and executes RFE after the callback returns. Return normally from either callback so its entry routine can perform exception return. A custom entry routine must implement its own dispatch and issue RFE from its exception frame. The CPU control reference defines saved state and entry privilege.

2.8.5 Intrinsic calls: `blua32`

The complete intrinsic operation set is `closure`, `trace`, and `trace_sink`. Use a direct dotted call on `blua32`; alias calls and colon-method calls do not provide intrinsic access. Unknown operation names are rejected.

Use `closure` when a callable value must be formed from an existing function record. The two trace forms serve a different purpose: optional observation of program state. A subject identifies the observed object, a named channel selects the observation, and a bound sink receives the payload values. A compilation can retain selected channels or erase their statements and operand evaluation. Keep required program work outside these optional observations.

2.8.5.1 `blua32.closure(function_address)`

`closure` constructs one closure from the supplied function-record address. Supply exactly one argument. The form is valid in an expression or a call statement; requesting further results supplies nil in those positions.

The address is consumed as an unsigned 32-bit function-record address in the active execution image. The function record is read before closure construction; an unsuccessful record read hard-halts execution. Function-record acquisition, capture context and closure execution follow the CPU reference.

```
local callback = blua32.closure(function_record_address)
```

Public program records supply addresses for such uses as the preparation sequence and BIOS call-target lookup. Use the address and capture context required by the selected interface; `closure` does not discover them. For source text or a constructed syntax tree, the BIOS compilation interfaces instead supply a callable root closure with its constant-pool capture context. Ordinary function expressions also need no programmer-supplied record address.

2.8.5.2 `blua32.trace(subject, channel, ...)`

`trace` is a statement-only form with at least two source arguments. The channel must be a String literal, not a variable or computed String. Each payload expression after the channel supplies one result. `trace` cannot be used as a value-producing expression.

Instrumentation selection is a compilation choice: erase all trace statements, select all channels, or select a named set of channels. Erasure is the default. Argument-count and literal-channel requirements still apply to erased statements. An erased statement evaluates neither its subject nor its payload expressions. Do not put required program effects in those expressions.

For a selected channel, `trace` evaluates the subject and reads its channel sink. If that sink is falsey, no payload expression is evaluated and no record is made. Otherwise it evaluates the payload and calls the sink's `record` method with the sink as receiver, followed by the payload values. A trace is therefore a call to a bound sink, not automatic output to a log.

2.8.5.3 `blua32.trace_sink(subject, channel, sink)`

`trace_sink` is also statement-only and requires exactly three source arguments. Its channel must be a String literal. It obeys the same compilation selection: an erased binding evaluates neither the subject nor the sink expression. When selected, it evaluates the subject, then the sink, and binds that sink to the subject's channel. Passing nil clears the binding. Use the intrinsic rather than accessing its channel-storage field directly.

This fragment shows a receiver-aware sink and matching channel literals:

```
local subject = {}
local sink = {}
function sink:record(value)
    self.last = value
end
blua32.trace_sink(subject, 'movement.step', sink)
blua32.trace(subject, 'movement.step', 4)
blua32.trace_sink(subject, 'movement.step', nil)
```

2.8.6 Interrupt waiting: `halt_until_irq`

`halt_until_irq` is a keyword statement, not a function or replaceable name. It takes no parentheses or arguments; a following semicolon is optional. Unlike `cop0` access, it has no supervisor-mode requirement.

If an interrupt-event-pending latch is already set, the statement consumes it without establishing a new halt. Otherwise it halts the active thread at its current physical frame depth. An occurrence need not wait for a future event. Accepted interrupts, not arbitrary device activity, define the interrupt boundary. Maskable interrupt acceptance requires current interrupt enable and an asserted maskable interrupt line. Full interrupt-entry, return, and scheduling rules are specified in the CPU reference.

Wait for the required condition in a predicate loop, for example by comparing an interrupt-handler-maintained completion sequence with its previous value:

```
while completion_sequence == previous_sequence do
    halt_until_irq
end
```

The interrupt handler and sequence belong to the operation being awaited. An interrupt alone does not establish that the desired transfer or frame completed.

2.9 BLua source semantics

This section records source conventions and behavior that must not be inferred from another Lua edition. General Lua syntax and introductory programming are outside this reference. Runtime values, arithmetic and instruction costs are specified by the CPU reference.

2.9.1 Number spellings

Decimal literals begin with a digit, or with `.` followed by a digit. A fractional dot after integer digits is part of the number only when another digit follows it. An exponent uses `e` or `E`, an optional sign, and one or more decimal digits.

Form	Examples
Decimal integer	`0`, `12`, `480`
Decimal fraction	`12.5`, `.5`
Decimal exponent	`1e3`, `1E-3`, `1.5e+2`
Hexadecimal integer	`0xff`, `0XFF`
Hexadecimal fraction	`0x1.8`, `0x.8`
Hexadecimal binary exponent	`0x1.8p+2`, `0x.8p-1`

Hexadecimal literals begin with `0x` or `0X`. Their digits may use either case, independently of the lowercase-only identifier rule. They require at least one hexadecimal digit across the integer and fractional parts. A `p` or `P` exponent has decimal digits and scales the hexadecimal value by a power of two.

A literal whose parsed numeric value is non-finite is rejected. Unary `-` is a separate operator, so `-12` is an expression rather than one signed numeric token. Use `1.0` rather than `1.` for a decimal fraction. Digit separators, binary or octal prefixes, and numeric suffixes are not supplied by these literal forms.

The expressions `-0` and `-0.0` produce negative zero. Constant binding and loading do not replace either expression with positive zero.

Hexadecimal spelling does not create a distinct address or register-word literal type. It is another spelling of a Number literal. The typed-storage reference defines declarations and address views; Book I, “Memory and bus access” in Book I defines what happens when the resulting address reaches the bus.

2.9.2 String literal differences

Quoted Strings accept `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, `\\`, `\"`, `\'`, `\xHH`, one-to-three-digit decimal `\ddd` (0–255), and `\z` followed by discarded whitespace. Exactly two hexadecimal digits follow `\x`. Unsupported escapes, `\u{...}`, backslash-newline continuation, a literal line feed and an unterminated quoted String are rejected.

Escapes select codepoints, not UTF-8 input bytes. For example, `'\xc3\xa9'` contains two codepoints, U+00C3 and U+00A9. Decode mapped UTF-8 bytes explicitly with the String BIOS interface. Long bracket Strings skip one initial CR, CR/LF pair or LF; subsequent content is literal through the matching delimiter. Bracket delimiters do not nest.

2.9.3 Local binding differences

`<const>` makes the local binding immutable, including through captures, not the contents of a Table it names. It requires an initializer position. `<close>` and other local attributes are not supported. A typed address annotation follows the attribute: `local cursor<const>: *word = address`.

Initializers normally resolve before the new locals become visible. Two forms make their own name available recursively: `local function name(...) ... end` and a single-name, single-value `local name<const> = function(...) ... end`. This exception does not extend to other or multi-name initializers.

2.9.4 Known-call argument diagnostics

The compiler can diagnose too few written arguments when it resolves a callee signature. This minimum check counts argument expressions as written: an explicit `nil` counts as one argument. Surplus expressions are not rejected by this check. The diagnostic concerns source use, not a runtime fault. If execution reaches a source-function call, missing fixed parameters are filled with `nil`; that runtime behavior does not make every source spelling acceptable to the compiler. See Calls, closures, globals and tables for the reached-call contract.

2.9.5 Variable arguments and result positions

`...` reads the variable arguments available to the **current call**. A nested function does not inherit an enclosing call's variable arguments.

BLua accepts `...` as an expression even when the function's written parameter list contains no `...`, and in the entry chunk. This does not make a bare `...` a statement. A function without `...` in its parameter list has **zero available variable arguments**, even if its caller supplies extra arguments; the entry chunk likewise has zero available variable arguments.

Zero available arguments are not the same as one `nil` result. With zero available variable arguments, an open-result use of `...` supplies zero values. When a context requests a fixed number of result positions, unavailable positions are filled with `nil`; selecting one result therefore gives `nil` in this case. For argument and result transfer at the machine-call boundary, see CPU calls.

2.9.6 Logical expressions

Only `nil` and `false` are false-valued. Every other value is true-valued, including all numbers (zero and NaN included), the empty string, and object references.

Expression	Result	Evaluation
`a and b`	`a` if it is false-valued; otherwise `b`	Evaluate `a` first; evaluate `b` only if `a` is true-valued.
`a or b`	`a` if it is true-valued; otherwise `b`	Evaluate `a` first; evaluate `b` only if `a` is false-valued.
`not a`	Boolean `true` if `a` is false-valued; otherwise Boolean `false`	Evaluate `a` and negate its truth value.

`and` and `or` select an operand value without converting it to a Boolean. Each operand is adjusted to one result, so the expression produces one selected value, not multiple results. An unneeded right operand is not evaluated, including its side effects or runtime faults; short-circuit evaluation also applies when the expression is used as a condition.

Thus `condition and 1 or 0` yields numeric `1` for a true-valued condition and numeric `0` otherwise: the intermediate `1` is true-valued. However, `condition and a or b` is not a general ternary expression: even when `condition` is true-valued, a selected `a` of `nil` or `false` causes `or` to select `b`.

2.9.7 Return boundaries and grouped results

A newline alone does not terminate `return`. An expressionless return is recognized only before `end`, `else`, `elseif`, `until`, or end of source; `return;` is not its accepted spelling. An expressionless source return supplies one nil, not zero results.

A parenthesized call or variable-argument expression supplies exactly one result. Extra results are discarded; a call returning no values supplies nil. Thus `local a, b = (pair())` assigns the first result to `a` and nil to `b`, even when `pair` returns two values. The same call adjustment applies in the BIOS runtime-loaded language.

2.9.8 Numeric `for`

```
for name = start, limit [, step] do
    block
end
```

Square brackets mark the optional step; they are not written in the source.

The header evaluates `start`, then `limit`, then the explicit `step`, once before the first test on each ordinary entry into the statement. Each expression supplies one result, including a call expression. Omitting `step` supplies Number `1`; supplying `0` does not omit it. The start result initializes the loop variable. The limit and step results are retained for subsequent tests and updates: assigning to an outer variable used by a header expression does not reevaluate that expression or replace its retained result.

The header expressions use the enclosing scope. Only the body has the new local binding named by `name`, so a same-named reference in the header still refers to its enclosing binding. The loop binding is not visible after the loop. A distinct local declared inside the body can shadow it in the usual lexical way.

Before each body entry, let `i` denote the current loop variable, `limit` the retained limit, and `step` the retained step. The selector and exit tests are:

Selector	Exit before the body when	Finite-Number reading
`0 < step` is true	`limit < i`	The body is allowed while `i <= limit`.
`0 < step` is false	`i < limit`	The body is allowed while `i >= limit`.

The exit predicates are the strict comparisons shown, not universal <=/>= tests. The inclusive paraphrases apply to finite Numbers; they do not establish termination for every Number value. For Number operands, the loop uses ordinary Number arithmetic rather than an integer-only counter.

Ordinary fallthrough from the body adds the retained step to the loop variable's **current** value, then repeats the selector and test. Assignment to the unshadowed loop variable is permitted and changes that same control variable, not a hidden copy. It affects the tail addition and next test; it does not trigger a new range check halfway through the body. A `break` leaves the loop without performing the tail addition.

An explicit zero step is accepted, not replaced by one or rejected as a zero-step error. It takes the second test above. With finite Numbers, a start below the limit skips the body, while a start equal to or above the limit allows initial entry. If the body leaves the variable unchanged, adding zero makes no counter progress. Changing the loop variable can make the next test exit; `break` can leave immediately.

The control path uses ordinary Number addition and comparison; header and body calls have the usual call and result-adjustment costs. Use the instruction-cost guide and cycle-accounting boundaries for the selected instructions and executed path, rather than assigning the source loop a fixed per-iteration cycle charge.

2.9.9 Generic `for`

A generic `for` obtains each set of loop-variable values by calling an iterator:

```
for name [, name ...] in expression [, expression ...] do
    block
end
```

There must be at least one loop name and one expression after `in`.

Before the first iteration, the expression list is evaluated once, in source order, and adjusted to three values: **iterator, state, control**. Each expression before the last supplies one value. Only the final expression can supply multiple results: a final call or `...` can fill the remaining entries. Missing entries become `nil`; surplus values are discarded. With three or more expressions, each supplies one value and only the first three values are retained. **Every expression is still evaluated**, including expressions whose results are discarded. Their effects and errors occur before the first iterator call.

A final parenthesized `(factory())` supplies one value to this list. Use an unparenthesized final call when the iterator factory returns iterator, state and control together.

The three retained values are cached, not recomputed on subsequent iterations. Rebinding a source variable does not replace its cached value. Caching a mutable value does not copy its contents: changes to shared contents remain visible.

Each attempted iteration calls exactly:

```
iterator(state, control)
```

The call requests as many results as there are loop names. Extra results are discarded and missing results become `nil`.

- If the first adjusted result is `nil`, the loop ends without assigning that call's results to the loop variables or executing the body.
- Otherwise, all adjusted results are assigned to the visible loop variables. The first result also becomes the hidden control value **before** the body starts. After the body, the next iterator call uses that control.

Only `nil` stops iteration: a first result of `false` continues and becomes the next control value. A later result of `nil` does not stop the loop. See values.

The visible loop variables are writable locals. Reassigning them in the body does not change the already saved hidden control; the next successful iteration replaces their values with new results. Their lexical scope is the loop body. The new locals are not in scope in the `in` expressions or after `end`; a loop name in its own initializer does not refer to the new local.

A reached `break` exits the nearest enclosing loop, skipping the remaining body and subsequent iterator calls.

Iteration uses ordinary calls, passing state and control unchanged. A non-callable iterator causes an error at the attempted call, not an empty loop. Other errors occur at the expression, call or body operation reached. The ordinary execution costs and boundaries apply to the loop operations, iterator calls and body.

See the BIOS iteration routines.

2.9.10 Labels and goto

Labels use `::name::` and branches use `goto name`.

Label names have **function-wide scope**, not block scope. A `goto` can name a label written before or after it, including a label in another lexical block of the same function. A nested function has its own label names: a label in another function cannot satisfy the jump. Each chunk also has its own label scope.

A second label with the same name in one function is rejected as `Duplicate label 'name'.`, even if the labels occupy different blocks. A target name not defined in that function is reported as `Missing label(s): ...`. A same-spelling label in a separate function is independent.

BLua does not reject a jump merely because it crosses a local declaration or enters or leaves a lexical block. The jump must still have a defined target and an encodable displacement. These are not checks on local scope or initialization.

A forward jump can skip local initialization; a backward jump resumes at its earlier target with existing values. Jumps insert neither initialization nor capture-close operations at crossed block boundaries. Keep required initialization and state changes on the executed path.

2.10 Global names and source resolution

The BIOS base reference defines the installed functions and library Tables. The rules here describe how a source name selects a shared ordinary-global binding.

Ordinary global values belong to a shared bank indexed by String names. This bank is separate from the system-global bank. It is not an ordinary Table, and accessing it is not Table indexing.

Source-name resolution and the value in that bank are distinct. A bare global read is resolved before execution; an unknown bare name is a compilation error. Installed bindings and the program's global assignments or function declarations make names known for this resolution. A declaration can make the name known before its assignment executes; it does not supply the assigned value early. A String argument to `setglobal` writes the bank without declaring a source identifier. Use `getglobal` and `setglobal` for names chosen during execution.

A previously unseen ordinary name begins with `nil`. When another program image registers the same ordinary name, it uses the existing shared binding; registering the name does not clear its value. Do not assume that moving between program images gives every name a new `nil` value.

For names chosen during execution, use the BIOS `getglobal` and `setglobal` calls. A computed name does not declare a bare source identifier. The program startup reference describes which global bindings survive execution transfer.

2.10.1 Runtime compilation

Use BIOS `load` to compile supported text during execution. On success it returns a closure; a protected compilation failure returns `nil, errorValue`. The runtime-compilation reference specifies the accepted text language, environments, external bindings and errors.

The BIOS monitor maintains its own session environment. Monitor variables and loaded-chunk environments are separate from the ordinary global bank; use the appropriate interface for the names being read or written.

3 Program images and startup

The ROM boot header identifies the console's entry functions and the BLua32 program header. The program header locates the constants, names and sections used by the program. Each function has a separate record describing its code, parameters, frame capacity and captured variables.

Startup establishes writable static contents before calling the application. An ordinary function call, an EXEC transfer and a fresh startup therefore have different effects. This chapter follows those effects from reset to cartridge entry, then specifies the ROM records and BIOS call-target lookup. Book I, "Memory and bus access" in Book I supplies the physical address map and the storage reservations that every placement must respect.

3.1 Addresses used to enter a program

The boot header connects three kinds of information. The **program header** describes the image shared by its functions. A **function record** describes one callable function. That record's **code address** selects the instruction bytes at which the function begins. Selecting the image establishes its constant and name tables; entering a function establishes a frame and begins fetching its instructions.

For example, consider these address fields in a cartridge's ROM. The chosen record and code locations are illustrative; the field offsets are those specified in the reference below.

Read or calculation	Example value	Meaning
ROM base	`10000000H`	Start of the selected socket's ROM window
Word at ROM base + `20H`	`00000100H`	Program-header offset relative to ROM base
ROM base + program-header offset	`10000100H`	Absolute program-header address
Word at ROM base + `28H`	`10001230H`	Absolute startup-function-record address
Word at function record + `00H`	`10002000H`	Absolute initial instruction address
Word at program header + `18H`	`10000300H`	Absolute constant-table address

For a startup record with STATIC set, EXEC receives `10001230H`, reads the record and starts its code at `10002000H`. It does not add the program-header address to either value. The constant table belongs to the selected image and is located independently of the function's code. Its entries are selected by constant indices in the instructions.

The same distinction applies to ordinary calls. CLOSURE constructs a callable value from a function-record address, and CALL invokes that value with arguments. EXEC instead replaces the active call chain. The CPU linkage examples use the same `10001230H` record address to show the instruction encoding.

3.2 ROM contents, RAM sections, and startup

The ROM layout described here stores program instructions, read-only data and the initial contents of writable data. Mutable static objects have separately assigned run addresses in main RAM. The section names distinguish their storage and initialization rules:

Section	Contents in the ROM image	Address used during execution
TEXT	Executable code	Instruction byte address in ROM
RODATA	Read-only data bytes	Read-only byte address in ROM
DATA	Initializer bytes	DATA run address in main RAM, after startup copies the initializers

Section	Contents in the ROM image	Address used during execution
BSS	No section byte payload	BSS address in main RAM, after startup clears the section

DATA consequently has two distinct addresses:

- Its **load address** identifies the initializer bytes in the ROM image.
- Its **run address** identifies the writable storage in main RAM.

Instructions access a DATA object's RAM run address. Startup reads the initializer at its ROM load address and copies it to that run address. RODATA is used in place in ROM. BSS reserves a RAM address range, whose initial zero contents are written by startup; the ROM needs only its address and size.

3.2.1 Follow the records, counter and output storage

The record-copy example uses three addressed objects: two read-only input records, an initialized counter, and space for two output records. Each record occupies twelve bytes. Their storage requirements are:

Object	Bytes in ROM	Bytes in main RAM	Preparation
`inputs`: two read-only records	24 record bytes	None	Use the ROM bytes in place
`counter`: one initialized word	4 initializer bytes	4 working bytes	Copy `12345678H` into the working word
`output`: two zero-initialized records	No BSS payload	24 working bytes	Clear all 24 bytes

The counter's initializer occupies bytes `78 56 34 12` in increasing ROM addresses. Startup copies those bytes to the counter's RAM run address. The program subsequently reads and changes the RAM word; its ROM initializer stays unchanged. Likewise, filling `output` changes its RAM bytes, not `inputs`.

The object extents above do not choose their addresses. Place the RAM objects outside the Book I, "Memory and bus access", under "Reserved and cartridge storage in main RAM", and describe the chosen section addresses consistently in startup and the program image. A local pointer view selects an existing object's bytes; it does not allocate another copy of that object.

3.2.2 When initialization occurs

A program's startup routine establishes its static storage before entering application code. The standard BLua startup sequence is:

Order	Operation
1	Copy the complete DATA section from its ROM load address to its RAM run address.
2	Write zero throughout the BSS section in RAM.
3	Run static module initialization.
4	Call the program's entry function with no arguments.

The DATA copy and BSS clear use four-byte word accesses over their complete padded byte counts. System startup initializes the firmware sections; cartridge startup initializes the cartridge program's sections.

EXEC starts the addressed function. DATA initialization and BSS clearing occur only when the instructions performing them execute. Therefore the ROM startup address must select the initialization routine when that initialization is required. Entering another function leaves the sections as they stand. Re-entering startup repeats the copy and clear, replacing the program's writable static contents with their initial values.

3.3 Reset and cartridge startup

3.3.1 From reset to the system program

At reset the CPU selects system ROM and enters the startup function specified in its ROM header. The call stack and transient execution state are cleared, there are no supplied arguments, and Status starts at `00000001H`: supervisor mode with current interrupt enable set.

The firmware startup routine initializes system DATA and BSS, runs module initializers, then enters the BIOS boot program. That program selects an executable cartridge and performs the handoff described below.

3.3.2 BIOS cartridge selection

The BIOS scans cartridge slot 0 before slot 1, considering slots whose presence bits are set in the sampled cartridge-status word. For each candidate it selects that slot and checks:

- ROM magic is `0x58534d42`;
- the header-size word is at least 60;
- the executable-image-offset word is nonzero;
- the startup-function word is nonzero.

The first candidate passing all four checks supplies the startup-function address. The CPU then applies the function-record and program rules specified in the record reference below.

The boot screen updates at VBlank. BIOS hands over when an executable cartridge is ready and its boot-frame counter has reached at least 50. A change between the missing, invalid and ready cartridge states resets that counter; changing the selected card while its classification remains ready does not by itself reset it. At handoff BIOS clears and flushes terminal status, masks central IRQ delivery, and writes the selected startup-function address to CP0 EXEC.

3.3.3 EXEC and cartridge entry

A startup function address identifies a **function record**, not the address of its first instruction. EXEC reads the target record and requires its STATIC flag. An unreadable record or one without that flag causes a hard halt.

Successful EXEC replaces the active call chain, selects the target image and execution domain, and starts the selected static function with no arguments. For a cartridge image, CPU Status has interrupt enable and user mode set (bits 0 and 1) from that function's first instruction. This privilege change occurs before the later application-entry call. The BIOS's zero IRQ-mask write masks source delivery without clearing CPU interrupt enable. The full EXEC reference specifies execution-state changes.

The selected cartridge startup routine performs the DATA copy, BSS clear, module initialization and entry call described under When initialization occurs. Those instructions establish the cartridge's writable sections. Selecting another static function enters that function directly, leaving DATA and BSS as they stand; selecting startup again repeats initialization.

3.3.4 Fresh system boot and execution transfer

Fresh system boot clears both global-name banks, then installs the BIOS environment before normal program entry. `EXEC` keeps the existing banks and BIOS bindings while replacing execution. Initialize program-owned globals explicitly when transferring between programs. See CPU execution control for the complete transfer behavior.

3.3.5 Taking ownership after the BIOS

At normal BIOS handoff, video timing is running and the display contains BIOS drawing. Cartridge code may retain that display or establish its own drawing and display configuration. Program each unit before use; its reset-value table applies to reset, while its inherited boot state follows the rules here. Book I, “System control and timing”, under “System time and frame timing” describes the live readouts, and CPU control describes entry privilege and interrupt enable.

The BIOS masks central IRQ delivery before handoff. Pending flags remain, and CPU interrupt enable is set at cartridge entry. Before enabling a source, install its handler and prepare the device: a previously pending source can become deliverable immediately. Service the device’s local condition and the controller’s pending bit separately. The complete dispatch and acknowledgement sequence is given in Book I, “Interrupt controller”.

Execution handoff also does not release the Book I, “Memory and bus access”, under “Firmware reservation in VRAM”; see Book I, “Memory and bus access”, under “Handoff is not release”. Later supervisor service preserves selected device context, not overwritten VRAM pixels. Its boundaries are described in Book I, “System control and timing”, under “Device boundaries during supervisor service”.

INPUT likewise carries BIOS sampling state into cartridge entry rather than starting from its reset words. Follow Book I, “Input controller”, under “BIOS handoff and supervisor sampling” to establish a fresh cartridge sample, including after supervisor return.

3.3.5.1 Firmware work at normal boot handoff

On the normal reset-to-cartridge boot path, BIOS has supplied its final GPU command batch and reached that batch’s completion marker. Firmware DMA and VBlank handlers have stopped submitting work. The following unit state is then available to the cartridge. Entry through another EXEC route must establish its own device state.

Surface	Firmware work at cartridge entry
DMA0 and DMA1	No outstanding firmware transfer or admitted transfer block. DMA0’s final transfer has finished; this boot path never starts DMA1.
GP0 input and readback	No active firmware writer, unfinished firmware packet, or outstanding firmware VRAM-to-CPU readback.
GEO	No outstanding firmware geometry job.
IMGDEC	No outstanding firmware decode, input, or output job.
APU	No queued firmware command, active playback slot, or outstanding sample-transfer batch. This does not imply that the APU clock or output activity has stopped.

Submitted BIOS drawing may still be pending after its final marker, although the firmware has stopped producing commands. Before changing resources used by that work, use the GPU completion protocol. Display publication remains a separate PCRTC event. EXEC leaves the device registers and submitted work in place.

3.4 ROM header fields

The ROM header begins at the ROM base. Provide at least its first **60 bytes**, through offset `0x3b`, for CPU program selection. All offsets in the following table are byte offsets from that base. Header words are unsigned 32-bit little-endian. The CPU uses the program and vector fields; BIOS also checks the identification and startup fields when selecting a cartridge.

Byte offset	Field	CPU program selection	BIOS cartridge selection
`0x00`	Magic	Not read.	Requires `0x58534d42`; bytes `42 4d 53 58` spell `BMSX`.
`0x04`	Header size	Not read.	Requires at least 60 bytes.
`0x08–0x1f`	Other prefix words	Not read.	Not read.
`0x20`	BLua32 image offset	Byte offset relative to the ROM base; zero means no executable image.	Requires a nonzero word.

Byte offset	Field	CPU program selection	BIOS cartridge selection
`0x24`	Declared image byte count	Not read.	Not read.
`0x28`	Startup function	Absolute function-record address.	Requires a nonzero word; supplies the EXEC target unchanged.
`0x2c`	IRQ function	Absolute function-record address.	Not read.
`0x30`	Exception function	Absolute function-record address.	Not read.
`0x34`, `0x38`	Unused by the boot interface	Not read.	Not read.

System ROM begins at `00000000H` and has execution-domain ID −1. Either cartridge socket uses ROM base `10000000H`, with domain ID 0 or 1 selecting the socket. Add the program-offset word to this base and retain the low 32 bits to obtain the BLua32 program-header address. The result must address installed ROM. The startup and vector fields are already absolute function-record addresses.

All 60 prefix bytes must be present in ROM. A zero program offset marks a ROM without a CPU program; an absent prefix likewise provides no program. System ROM must supply the reset program. CPU selection uses the fields shown in its column; identification checks belong to BIOS cartridge selection. The exception address's use follows the exception-routing rules, including the system route for synchronous cartridge faults.

The boot interface assigns no interpretation to the unused prefix words. Their contents do not select a program, change startup initialization or establish a memory reservation.

The next ROM-header word, at byte offset `0x3c`, is a ROM-relative offset to optional BIOS diagnostic records; zero means absent. It is outside the CPU's 60-byte selection prefix. BIOS diagnostics defines its source-location and frame-scope tables separately from executable selection and instruction operation.

3.5 BLua32 executable image

The BLua32 program header occupies **96 bytes** in ROM. Its offsets are relative to the program-header address obtained above. The CPU uses these six unsigned 32-bit little-endian fields to establish the program's constants and name banks:

Image byte offset	Field	Unit and interpretation
`0x18`	Constant-table address	Absolute machine byte address in installed ROM.
`0x1c`	Constant count	Number of 16-byte constant records.
`0x20`	Global-name-table address	Absolute machine byte address in installed ROM.
`0x24`	Global-name count	Number of 8-byte name records.
`0x28`	System-global-name-table address	Absolute machine byte address in installed ROM.
`0x2c`	System-global-name count	Number of 8-byte name records.

The other four-byte header slots describe the program's sections and linkage. They retain the following format; CPU program selection does not use them to locate the constant or name tables.

Image byte offset	Descriptive field
`0x00`	Magic; declared constant `0x32334c42`.
`0x04`	Version; declared constant 1.
`0x08`, `0x0c`	Image byte count; flags.
`0x10`, `0x14`	Function-table address; function count.
`0x30`, `0x34`	String address; string byte count.
`0x38`, `0x3c`	RODATA address; RODATA byte count.
`0x40`, `0x44`, `0x48`	DATA load address; DATA byte count; DATA run address.
`0x4c`, `0x50`	BSS address; BSS byte count.
`0x54`, `0x58`	TEXT address; TEXT byte count.

Image byte offset	Descriptive field
`0x5c`	Preparation-sequence function-record address; zero when no sequence is present.

Provide the complete 96-byte header, including its descriptive slots. The CPU reads the explicit constant/name addresses and counts. Magic, version, function-table enumeration, section order and declared section extents do not alter those reads or bound instruction fetching. DATA and BSS are initialized by the startup instructions.

The preparation address selects a separate ordinary function, not another startup vector. Calling it completes pending module initialization and runs the registered `<init>` participants without recopying DATA, clearing BSS or restarting the entry function. See BLua participant sequences for declaration order and invocation.

Table addresses are **absolute machine byte addresses**. Tables and their text must occupy installed ROM; their placement is independent of table order and of the declared image extent. Cartridge ROM reads select the program's execution socket, while system-ROM addresses select system ROM. A zero table count leaves its address unused.

3.5.1 Constant and name records

Constant record `i` begins at `constantTableAddress + 16i`, with zero-based `i`. The tag is a four-byte unsigned little-endian word at record offset `0x00`:

Tag	Value	Payload consumed
0	Nil	None.
1	False	None.
2	True	None.
3	Number	Eight-byte IEEE-754 binary64, little-endian, at `0x04`–`0x0b`.
4	String	Absolute ROM text byte address at `0x04`; text byte count at `0x08`, both u32 little-endian.

Constant indices select records in table order. Tags 0–2 encode their value without a payload; bytes `0x0c`–`0x0f` are unused. A record with another tag is invalid and prevents program activation.

Both name tables use the same layout. Name record `i` begins at `tableAddress + 8i`:

Record byte offset	Field
`0x00`	Absolute ROM text byte address, u32 little-endian.
`0x04`	Text byte count, u32 little-endian.

Text lengths count **bytes**, not codepoints. Reads consume the specified span, not a NUL-terminated sequence. A zero byte within the span is part of the text. A zero byte count produces empty text without reading the address.

Constant text and names use complete, shortest-form UTF-8 encodings of Unicode scalar values. Invalid leading or continuation bytes, truncated sequences, overlong encodings, surrogate values D800H–DFFFH and values above 10FFFFH make image activation fail. An initial `EF BB BF` encodes the ordinary character U+FEFF; it is retained rather than removed as a byte-order marker. A string beginning with U+FEFF therefore differs from the string without that character.

The two name tables establish separate ordinary-global and system-global namespaces. Within each namespace an existing name retains its value and a new name begins nil. A name-table index selects that name's slot; initial values are established by executed instructions.

3.5.2 Function and upvalue records

A function address identifies a **32-byte function record**, not its first instruction. Record fields consumed below are unsigned 32-bit little-endian words:

Record byte offset	Field	Interpretation
`0x00`	Code address	Absolute machine byte address of the initial instruction.
`0x04`	Declared code byte count	Not read.
`0x08`	Parameter count	Number of fixed parameter slots.
`0x0c`	Maximum stack	Number of register-value slots, not bytes or a RAM address.
`0x10`	Flags	Bit 0 VARARG; bit 1 STATIC.
`0x14`	Upvalue-table address	Absolute machine byte address.
`0x18`	Upvalue count	Number of four-byte descriptors.
`0x1c`	Reserved	Not read.

VARARG retains arguments beyond the fixed parameters. STATIC permits direct entry through EXEC; the closure-construction rules are described in CPU — Closures and captured variables. The immediate-address form of `CLOSURE` encodes a function-record address in **16-byte units**; its register-address form supplies the full address. Function-record reads themselves do not impose a 16-byte alignment check. The declared code byte count does not bound instruction fetches.

Upvalue descriptor `i` is a four-byte unsigned little-endian word at `upvalueTableAddress + 4i`. Bit 31 set captures a parent-frame register; bit 31 clear captures a parent-closure upvalue. Bits 0–30 contain the zero-based index. Function and upvalue records use ordinary mapped-bus reads and may therefore occupy mapped memory. Constant and name-table records and their text require ROM.

Function-record words are little-endian; instruction words are big-endian. Neither a function-record address nor a record's code address is relative to the image header.

3.6 Locating BIOS call targets

The published BIOS import names bind to fixed, zero-based slots in the system executable's function table. The ROM header at system address zero supplies the executable's offset at `0x20`. Its executable header supplies the absolute function-table address at offset `0x10`. Each function record occupies 32 bytes, so slot `s` has function-record address `table_address + 32*s`. These addresses select function records, not their instruction bytes.

An independent linker resolves the published names through these fields and slots. It uses that function-record address for a CLOSURE operand, with the address divided by sixteen as specified in the instruction reference. Only the three published slots bind these named imports. Startup, vector and preparation entry points are identified separately by their header fields; other table positions do not themselves define additional BIOS interfaces.

The same lookup is available to a running program. This sequence calls slot zero without a named import:

```
local image<const> = mem32le[0x20]
local function_table<const> = mem32le[image + 0x10]
local sin_turn32<const> = blua32.closure(function_table)
local sine = sin_turn32(0x40000000) -- 65536
```

Use the installed system ROM's own fields, rather than assuming a fixed instruction address or substituting a record from the cartridge image. For static RAM placement, the system executable's BSS address at `0x4c` plus its padded byte count at `0x50` gives the first byte after firmware static storage. The executable-image reference above defines these section and record fields.

4 Protected calls and coroutines

Protected calls let a program receive a language error and continue at a chosen call boundary. Coroutines let one procedure suspend and later continue with its local values intact. The BIOS supplies `error`, `pcall`, `xpcall` and the `coroutine` Table before normal program entry.

These interfaces carry language values and result tuples. Device status, interrupt acceptance and hardware exception return remain governed by the CPU, Book I, “Interrupt controller” and Book I, “System control and timing” interfaces.

4.1 Control flow and hardware progress

Protected calls, coroutines and interrupt waits solve different problems:

Mechanism	What it changes	What resumes the caller
`pcall` or `xpcall`	Adds a language-error boundary around a call.	The target returns or an eligible language error reaches that boundary.
`coroutine.resume(thread, ...)`	Transfers language execution to the selected suspended thread.	That thread yields, returns or fails with an unhandled language error.
`coroutine.yield(...)`	Suspends the current coroutine and returns values to its resumer.	A later accepted `resume` supplies the yield call’s results.
`halt_until_irq`	Waits at the current call depth unless an interrupt event is already pending.	An accepted interrupt event; the CPU reference defines the latch and return rules.

Coroutine execution is cooperative: the resumer waits while its selected thread runs. Device operations continue according to their own hardware rules. Establish completion from the device’s status or interrupt condition before consuming its result. A coroutine transfer or a protected error result is not a completion indication from a device.

4.2 Protected calls

Call `pcall` to receive a function’s success flag and results or its error value. Choose `xpcall` when a handler should transform that error before it returns to the caller. Test the success flag: an error value can be nil, false, a Number or an object as well as a String.

4.2.1 Signatures

Function	Signature	Purpose
`error`	`error(message [, level])`	Raise the supplied value; do not return normally.
`pcall`	`pcall(f, ...)`	Call `f` with protection.
`xpcall`	`xpcall(f, msgh, ...)`	Call `f` with protection and transform its error through `msgh`.

Brackets mark the optional `level` argument. The ellipses denote arguments forwarded to the target. Supply the leading arguments shown in each signature. If an indirect call omits a fixed argument, that argument arrives as nil: a missing first argument to `error` raises nil, and a missing protected target is nil. No maximum argument or result count is specified here.

4.2.2 Raising a value

`error` raises its first value unchanged. Every language kind is permitted, including nil, false and a Table reference. A raised Table retains its identity. The optional `level` and surplus arguments are ignored; the value carries no automatically added source prefix or traceback.

An error payload of `nil` is still an error. Inspect the success flag, not the payload's truth value, to distinguish failure from success.

4.2.3 Calling and receiving results

`pcall` forwards the arguments after `f`. `xpcall` forwards the arguments after `f` and `msgh`; the handler is not one of the body's arguments. The target must be a function: either a BLua closure or a built-in function. A value of another kind is not accepted as a protected target.

When the surrounding expression receives all results, the layouts are:

Outcome	Results, in order
Target returns no values	`true`
Target returns values	`true`, followed by every target result
`pcall` catches an error	`false, errorValue`
`xpcall` handles an error	`false, handlerValue`

Result count and `nil` positions matter. The arithmetic example below produces `true, 10, nil, 4`, not a compressed three-value result. An ordinary BLua function that falls through, or uses a bare return, returns one `nil`; its protected success therefore gives `true, nil`, not just `true`. A caught `nil` error produces exactly two results, `false, nil`. If the caller requests a fixed number of results instead, extra results are discarded and missing positions are filled with `nil`.

A protected target can itself be `pcall` or `xpcall`. Its own success flag and remaining results are then ordinary results of the outer target; an outer success flag does not mean that every nested operation succeeded.

4.2.4 Transforming an error

`xpcall` requires `msgh` to be a function before it establishes its new protected call. An absent or non-function handler therefore raises an error outside that new boundary. An already active outer protected call can catch it.

A body error unwinds the body's calls before the saved handler runs. The handler receives exactly one argument: the incoming error value. If it returns normally, only its first result becomes `handlerValue`; further results are discarded. A handler returning no results supplies `nil`. The example below transforms its error into `144` and discards the handler's extra result.

A language error reaching the same protected boundary while its handler is active replaces the payload with the exact string `error in error handling`. It does not return the handler's newly thrown value.

For a directly called built-in handler, explicit raised values and language execution errors produce that handler-error result. Memory exhaustion in that handler follows its out-of-memory path instead.

4.2.5 Which failures are protected?

The protected language-error path handles explicitly raised values, language execution errors, and classified out-of-memory errors. Explicit values remain unchanged; the latter two categories supply reason strings.

Condition	Error payload
Non-function protected target	`Attempted to call a non-function value.`
Non-function `xpcall` handler	`xpcall error handler must be a function.`
Nil Table key	`Table index is nil.`
Classified memory exhaustion	`Out of memory.`

The target check occurs inside the new protected boundary. For `pcall`, a non-function target therefore gives `false` and its reason string. For `xpcall`, the valid handler receives that target error. The handler check, by contrast, occurs before the new boundary, as described above. For example, `pcall(rawget, {}, nil)` returns `false, 'Table index is nil.'`.

Bus, address and coprocessor faults enter the hardware exception route. An entered hardware exception or interrupt also blocks unwinding to an older protected call across that boundary. A handler may establish a new protected call for language operations on its own side of the boundary. Use the CPU exception interface for hardware recovery.

Protected calls belong to individual threads. The innermost active protected call in the current thread receives an eligible error. Installing protection in one thread does not install it in another. Protection in a coroutine is considered before that coroutine is marked failed and its error is returned to its resumer.

A protected body is not made non-yieldable merely by using `pcall` or `xpcall`. An active `xpcall` handler is non-yieldable. A protected body can yield and resume before raising an error. Attempting to yield from its active handler instead produces the handler-error result rather than suspending there.

Protected unwinding closes captured locals of the removed calls and releases those calls. Writes already completed remain visible: the example below leaves the earlier assignment of `77` in place.

4.2.6 Incremental cycle charges

`error` adds 2 cycles; `pcall` and `xpcall` each add 4 cycles, with no additional per-argument or per-result charge. A protected built-in target or handler also incurs its own base charge. These are incremental charges, not total timings: target and handler execution, call and return instructions, and other machine work still contribute. Do not budget an entire protected operation as a fixed 4-cycle service.

4.3 Coroutines

The BIOS installs the `coroutine` table before cartridge execution. Its eight functions are available through that table; no cartridge import is needed.

A coroutine keeps its entry function, call chain, local values, protected calls and suspension point. The main thread starts active. Creating another thread leaves it suspended; an accepted `resume` starts or continues it. Yield, final return or an unhandled language error transfers control back to its resumer. At each point only one language thread is active.

4.3.1 Argument and result transfer

The figure follows one producer from its first resume to its final return. `resume` and `yield` abbreviate `coroutine.resume` and `coroutine.yield`. Arrows show language control and value transfers; vertical distance is not elapsed machine time.

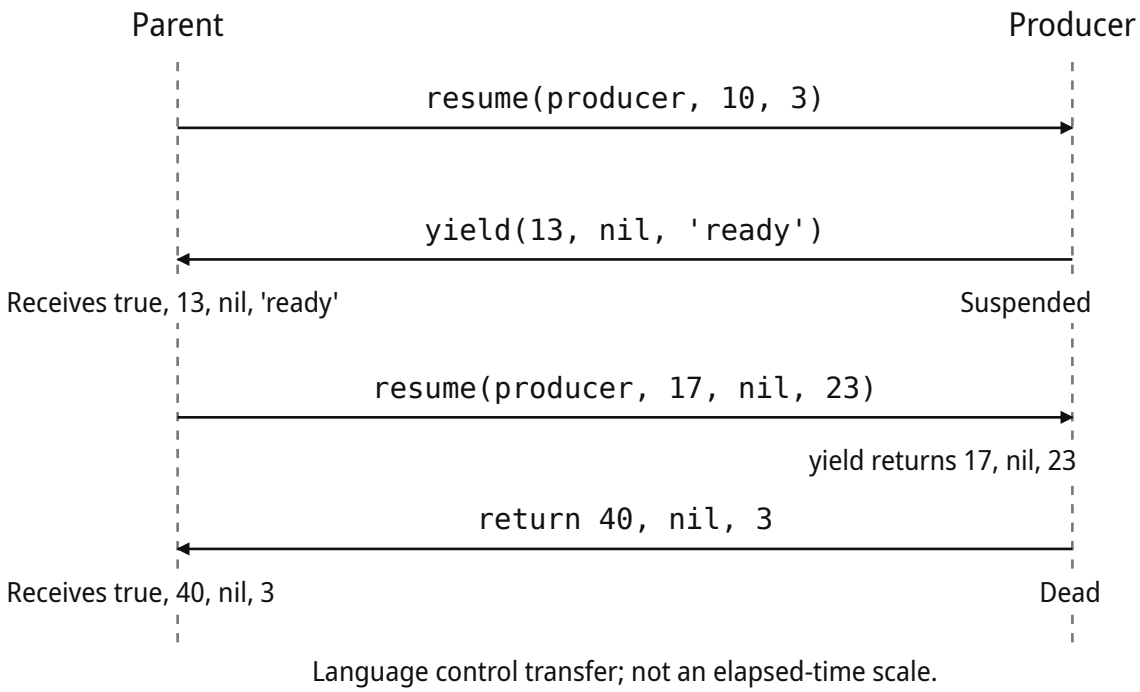


Figure 4.1: The producer yields values to its parent, then receives the next resume arguments as yield results.

4.3.1.1 Yield and resume

This producer follows the transfers in the figure. Its first arguments are entry arguments; the next resume supplies the results of its suspended yield.

```lua
local producer = coroutine.create(function(a, b)
    local x, y, z = coroutine.yield(a + b, nil, 'ready')
    return x + z, nil, b
end)
-- coroutine.status(producer) is 'suspended'.
local first_ok, first, gap, label = coroutine.resume(producer, 10, 3)
-- true, 13, nil, 'ready'; producer is suspended again.
local final_ok, final, middle, original =
    coroutine.resume(producer, 17, nil, 23)
-- true, 40, nil, 3; producer is now dead.
local dead_ok, message = coroutine.resume(producer)
-- false, 'cannot resume non-suspended coroutine'.
```

While the producer runs, `coroutine.running()` returns its Thread with false as the second result. Its resumer has status `normal`; when the producer yields or returns, the resumer becomes `running` again.

4.3.2 Creating and identifying threads

`coroutine.create(fn)` → `thread` returns exactly one thread value. The entry `fn` must be a language function closure. A built-in function value is not accepted merely because it is callable. Omitting `fn`, passing nil, or passing another value raises `Invalid argument`. Creation leaves the thread unstarted and publicly `"suspended"`.

`coroutine.running()` → `thread, is_main` returns exactly two values: the active thread and a boolean. The boolean is true in the main thread and false in a created coroutine. Keep thread values as language values; the operations below require a thread, not an address or numeric identifier.

`coroutine.status(thread)` → `string` returns exactly one of these strings:

Result	Meaning
`"suspended"`	Created but not started, or stopped at a yield.
`"running"`	Currently active.
`"normal"`	Waiting inside an accepted resume of another thread.
`"dead"`	Finished, closed, or failed with an unhandled language error.

A missing or non-thread argument raises `Invalid argument.` The public status does not distinguish an unstarted thread from a yielded thread, or a normally completed thread from a failed one.

4.3.3 Transferring execution and values

`coroutine.resume(thread, ...)` → `true, ...` or `false, error` resumes an unstarted or yielded thread. Its first argument must be a thread. A missing or incorrectly typed first argument raises `Invalid argument.` in the caller; it is not converted into an ordinary `false, error` result.

On the first accepted resume, all arguments after `thread` become the entry function's arguments. The caller becomes `"normal"`, and the target becomes `"running"`. If that target resumes another coroutine, it becomes normal while its own target runs. When control returns to a resumer, that resumer becomes running again.

An accepted resume returns a true prefix followed by either the values supplied to `yield` or the entry function's final return values. Yield leaves the target suspended; a final return leaves it dead. An unhandled language error instead returns exactly `false` and the original error value, subject to the hardware boundaries below. The target then has failed state, publicly reported as dead.

Attempting to resume a running, normal, completed, closed, or failed thread returns exactly `false, "cannot resume non-suspended coroutine"`. This rejection does not close or otherwise change the target. Hardware-exception rejection is a separate case described below.

`coroutine.yield(...)` → `...`, when later resumed, returns the next resume's arguments. Yield first suspends the active coroutine and returns its supplied values to the resumer, after the resumer's true prefix. Later, the values after `thread` in the next resume become the results of that suspended yield call. They do not restart the entry function and do not become another set of entry arguments. Calling yield where yielding is forbidden raises `Invalid argument.`; yield does not return a false prefix.

The exchange carries a value tuple, not an automatically packed table. Its count and nil positions are preserved. In Yield and resume, the first resume supplies 10 and 3; the child yields 13, nil, and `"ready"`. The caller receives true followed by all three values. The next resume supplies 17, nil, and 23; these are the three results of the child's yield call. The child finally returns 40, nil, and 3, so the caller receives true followed by those three values.

Ordinary call-result adjustment still applies. A fixed number of receiving variables truncates excess results or supplies nil for missing results. An expanding result position carries the actual tuple count. Preserve that count when nil values matter; nil is not an end marker. An explicit bare `return`, and an ordinary BLua function that reaches its end without a return value, each return **one nil value**, not a zero-result tuple. Accepting all results of a successful resume of that function therefore gives `true, nil`.

4.3.4 Querying yieldability

`coroutine.isyieldable([thread])` → `boolean` returns one boolean. With no argument it examines the active thread. With an argument it examines that thread; explicit nil is invalid and raises `Invalid argument.` Extra arguments are unused.

The main thread is not yieldable. A selected thread carrying a hardware exception frame or an active `xpcall` error-handler continuation is also not yieldable. Ordinary `pcall` execution and the body protected by `xpcall` do not themselves forbid yielding. Executing the `xpcall` error handler does. Merely having an error-handler function stored somewhere does not affect the answer.

Yieldability does not check status and is not permission to resume. An unstarted thread can report true. A normally completed or closed thread can also report true, and a failed thread may report true. None of those answers changes resume's requirement that the target be unstarted or suspended. Ask status when deciding whether a target can be resumed; ask yieldability about the selected context's yield restrictions.

4.3.5 Failure and closing

Language errors are first offered to protected calls in the active thread. If none consumes an error and the coroutine has a resumer, ordinary coroutine error transfer returns `false, error` to that resumer. The error remains its original value, including nil or the identity of a table; it is not necessarily a string. Failed state retains the error and language frames until closing. A public dead status alone does not indicate whether this retained failure exists.

A false result returned by resume is not itself an error thrown in the caller. Consequently, protecting resume with `pcall` adds the protected call's success boolean outside resume's own result tuple. A protected invocation can therefore succeed while resume reports a target failure. By contrast, an invalid argument raised in the caller is a caller-side error.

`coroutine.close(thread)` → `true` or `false, original_error` closes an unstarted, suspended, dead, or failed thread. A missing or non-thread argument raises `Invalid argument.` Closing a running or normal thread also raises `Invalid argument.`

Closing releases the thread's language frames, pending protected-call state, entry function, and register-stack state, and leaves it dead. It does not resume the function to execute remaining statements. Open upvalues are closed while preserving their current captured values. A surviving closure can therefore still read its captured local after the owning coroutine is closed. In the closing example below, closing after yielding the captured value 19 prevents the later assignment of 20; the surviving reader still returns 19.

An unstarted, suspended, or normally dead target returns exactly true when closed. A failed target returns exactly false and its retained original error, **but is still closed**. That close clears the retained failure; closing the same thread again returns true. The false prefix in this case reports the earlier failure, not a refusal to close.

4.3.6 Wrapped coroutines

`coroutine.wrap(fn)` → `wrapped_function` returns exactly one variadic function, backed by one newly created coroutine. The entry requirement is the same as for create. Each call to the returned function resumes that coroutine with the supplied arguments. Successful calls remove resume's true prefix and return the yielded or final tuple unchanged, including nil positions and result count.

On a false resume result, the wrapped function raises the first error value returned by that resume instead of returning a false prefix. If the retained coroutine is in failed state, the wrapper closes it before raising that value. A retained handle to that failed thread subsequently closes successfully because the wrapper has already cleared its retained failure. This failed-state cleanup also applies if resume rejected an already failed thread. In that case the wrapper raises the rejection string, not the earlier retained body error. Rejection in other non-suspended states does not trigger failed-thread cleanup. A finished wrapper cannot be used to restart its entry function. Replacing the public `coroutine.close` table entry does not replace the wrapper's captured closing operation.

4.3.7 Call arity

Create, status, close, and wrap each declare one fixed parameter; omitted fixed parameters arrive as nil, and excess arguments are not forwarded. Running declares no parameters. Resume has one fixed parameter and a variadic tail; yield and the function returned by wrap are variadic. Isyieldable also preserves its argument list, which is why omission differs from explicit nil.

4.3.8 Hardware IRQ and exception boundaries

If the active caller has a hardware exception frame, `resume` returns `false, "cannot resume coroutine across a hardware exception"`. This check precedes the target-status check and leaves the target unchanged.

Yield is also forbidden across an entered exception boundary. A language error that cannot reach protection on its own side enters the Lua fault exception route. It does not unwind through an older hardware handler to return a coroutine result. See Book I, "System control and timing" for device-context switching and the CPU HALT interface for interrupt waits.

4.3.9 Cycle increments

The underlying create, resume, yield, status, running, close, and isyieldable operations each add four cycles, with no additional per-argument or per-result charge. This is not their service time or the total cost of calling the installed function. Ordinary function instructions add work; wrap also creates and resumes a coroutine and may inspect, close, and raise an error. Resumed cartridge code contributes its own execution. No total per-call cycle bound is specified here.

4.4 Language fault reasons

If a classified language error reaches the CPU fault-exception route, its Lua fault reason identifies the condition below. A protected failure result does not itself mean that this route was entered. Arbitrary values raised by `error` keep their own value and identity; their text does not select a reason code.

Reason	Condition
1	Attempt to call a non-function value
4	Nil Table key
7	Non-function `xpcall` handler
8	Explicitly raised value
9	Classified out-of-memory condition
10	Invalid argument

Book I, “System control and timing” describes retained fault words and publication. Book I, “Interrupt controller” describes interrupt handling. Protected language calls are not substitutes for those hardware interfaces.

4.5 Reset and startup boundary

A newly created coroutine has not run its entry and is reported as `suspended`. CPU reset is not an ordinary `coroutine.close` call. It clears the active call and resumer chain, including protected-call ownership, and starts the selected system startup; it does not deliver a close-result tuple to the interrupted caller. CPU reference and Book I, “System control and timing” describe CPU startup and the system-wide reset request separately.

4.6 Programming examples

4.6.1 Protected results and errors

A protected call keeps the target’s result count and nil positions. A caught error keeps the original value, even when that value is nil or a Table.

```
local ok, sum, gap, difference = pcall(function(a, b)
    return a + b, nil, a - b
end, 7, 3)
-- true, 10, nil, 4.
local failure = {code = 72}
local state = {value = 0}
local caught_ok, object = pcall(function()
    state.value = 77
    error(failure)
end)
-- caught_ok is false; object is failure; state.value remains 77.
local nil_ok, nil_error = pcall(error, nil) -- false, nil
local index_ok, index_error = pcall(rawget, {}, nil)
-- false, 'Table index is nil.'.
```

Only the handler’s first result is retained. A language error in the handler instead produces the handler-error String.

```
local failure = {code = 72}
local ok, code = xpcall(function() error(failure) end,
    function(value) return value.code * 2, 'discarded' end)
-- false, 144.
local handled, message = xpcall(function() error('body') end,
    function() error('handler') end)
-- false, 'error in error handling'.
```

4.6.2 Failure and explicit close

Closing a failed coroutine reports its retained error but still completes the close. A second close therefore succeeds.

```
local failure = {code = 72}
local failed = coroutine.create(function() error(failure) end)
local ok, value = coroutine.resume(failed)       -- false, failure
local closed, original = coroutine.close(failed) -- false, failure
local closed_again = coroutine.close(failed)    -- true
```

Closing a suspended coroutine does not run its remaining statements. A closure that outlives the coroutine keeps the current value of its captured local:

```
local read
local abandoned = coroutine.create(function()
    local value = 19
    read = function() return value end
    coroutine.yield(value)
    value = 20
end)
local ok, yielded = coroutine.resume(abandoned) -- true, 19
coroutine.close(abandoned)
local retained = read() -- 19: the assignment of 20 never ran.
```

4.6.3 Wrapped calls

A wrapped coroutine exchanges the same tuples without resume's Boolean prefix. An error is raised in the caller; a failed coroutine is closed before that error is propagated.

```
local wrapped = coroutine.wrap(function()
    local next_value = coroutine.yield(12)
    return next_value
end)
local first = wrapped()   -- 12
local final = wrapped(34) -- 34

local failure = {code = 72}
local failed_thread
local invoke = coroutine.wrap(function()
    failed_thread = coroutine.running()
    error(failure)
end)
local ok, caught = pcall(invoke) -- false, failure
local closed = coroutine.close(failed_thread) -- true: already closed by wrap.
```

5 BIOS base services

The BIOS supplies functions for value checks, numeric and text conversion, Table traversal, argument selection, global-name access and console output. Use the installed global names directly after normal BIOS startup.

This chapter covers sixteen base functions, including direct Table access and metatable selection. Protected calls and coroutines specifies `error`, `pcall` and `xpcall`. Device registers and retained fault words are described in Book I, “System control and timing”.

Choose the operation according to the representation being used:

Task	Interface	Important distinction
Convert numeric text	`tonumber(value [, base])`	Conversion can return nil. Arithmetic does not implicitly parse a numeric String.
Produce diagnostic text	`tostring(value)`, `print(...)`	`tostring` returns text; `print` sends a line to the BIOS console.
Traverse own Table entries	`next(target [, key])`, `pairs(target)`	Entry traversal is different from a numeric sequence ending at its first missing value.
Traverse a numeric sequence	`ipairs(target)`	Starts at key 1 and stops at nil; use an explicit count when nil is part of a tuple.
Count or select call arguments	`select(selector, ...)`	Supplied nil positions still count. Table packing preserves that count in a Table.
Access a global by a computed name	`getglobal(name)`, `setglobal(name, value)`	These select ordinary global bindings, not keys in a global-environment Table.
Read or write a Table’s own mapping	`rawget`, `rawset`	Bypasses inherited reads; see Table and metatable access.
Inspect or set a Table’s metatable	`getmetatable`, `setmetatable`	The supported inherited-read and weak-reference rules are in the CPU reference.

5.1 Installed program environment

Before normal cartridge entry, the BIOS installs the base functions and publishes the library names `table`, `string`, `coroutine`, `lua_compiler`, `load`, `os`, and `math`. A cartridge entering through this path can use those installed bindings without installing the libraries itself.

These are ordinary public bindings. A program can change them; their initial presence does not make their values immutable. Nor does installation provide a private copy of the global environment for each cartridge.

The BIOS does not install `_G` or `_ENV` as global-environment Tables. Use `getglobal` and `setglobal` for name-based access to the ordinary global bank; this is separate from the environments accepted by runtime compilation.

5.1.1 Installed names and library Tables

The base installation provides these public names:

- `next`, `type`, `setmetatable`, `getmetatable`, `rawget`, `rawset`;
- `getglobal`, `setglobal`, `select`;
- `error`, `pcall`, `xpcall`, `assert`;
- `rawequal`, `tostring`, `tonumber`, `print`, `ipairs`, `pairs`.

This is the base installation inventory, not a restriction on names that a program or module may introduce. The entries below give the base-function contracts; name-based global access selects bindings during execution.

Several other installed globals hold ordinary library Tables. The distinction matters: `string` is a global binding, while the library Table initially held by that binding is a separate value.

Installed name	Interface navigation
`table`	The Table library. See the Table reference for its operations.
`string`	The completed String library Table, also registered for String indexing. See the String reference for indexing and library behavior.
`coroutine`	The coroutine library Table. See Protected calls and coroutines.
`math`	The mathematics library, including numeric domains, radian trigonometry and random-number state.
`os`	The time and date library: cycle-derived elapsed time and deterministic civil-time conversion, not filesystem or process services.
`lua_compiler`	A Table containing `compile_syntax`, `syntax_factory`, and `load`, for the BIOS runtime-compilation interface.
`load`	Initially the same function value as `lua_compiler.load`; see runtime compilation.

Use those routine chapters for argument domains, precision, errors, random-number behavior and timing conventions.

5.1.2 Public bindings and library references

Public library bindings can be reassigned. A saved Table or function reference continues to refer to its original value after the global name changes. The BIOS also retains some dependencies at installation: `assert` keeps its selection and error operations, `print` keeps its console destination, and String operations retain their listed helper functions. `pairs` instead reads the current global `next`; `print` reads current `select` and `tostring`; `tonumber` reads current `type` and `error`.

String method lookup uses the installed String-index Table. Reassigning the `string` global leaves that method Table in place. The routine references describe any further dependency relevant to replacing an installed binding.

5.2 Value checks and conversion

Use a conversion when a different value kind is required. A String of digits remains text until conversion succeeds; numeric zero remains truthy.

A missing fixed argument arrives as nil. Variadic interfaces count the supplied positions, including explicit nils. A fixed receiving list discards excess results and fills missing positions with nil; an expanding result position carries the returned count. These rules matter for assertions and argument tuples as well as for ordinary function calls.

5.2.1 `assert(condition, ...)`

`assert` tests the condition using BMSX truth: only `nil` and `false` are falsey. Every other value is truthy, including numeric zero and an empty String. An omitted condition is `nil` and therefore fails.

It uses captured error and argument-selection primitives. Replacing the public `error` or `select` binding does not replace those operations inside `assert`.

On success, `assert` returns the condition followed by every remaining argument, unchanged and in order. Its result count is exactly one plus the number of tail arguments. Explicit `nil` positions are retained, including trailing ones; this is not a function that returns only a Boolean or a single checked value.

```
local condition, message, absent, code = assert(0, 'kept', nil, 7)
```

This call succeeds and returns exactly four values: `0`, `'kept'`, `nil`, and `7`. For counting or selecting a multiple-result sequence, see Traversal, argument selection, globals, and output.

On failure, the presence of a tail argument determines the error value:

Arguments after the condition	Raised value
None	The String `assertion failed!`
One or more	The first tail value, unchanged

Consequently, `assert(false, nil)` raises `nil`, whereas `assert(false)` raises `assertion failed!`. A supplied `false`, Number, Table, or other non-String value is also raised as that value, not converted to a message. Later tail arguments are not additional error values. Error interception and propagation are described in Protected calls and coroutines.

5.2.2 `rawequal(left, right)`

Returns one Boolean using BMSX equality. Missing operands are nil and surplus operands are ignored. Values are compared in their existing kinds; `__eq` is not called.

Operand categories	Equality rule
Different categories	Unequal
`nil` and `nil`	Equal
Booleans	Equal when they have the same Boolean value
Numbers	Numeric equality; NaN is unequal, and positive and negative zero are equal
Strings	Equal String values
Built-in functions	Equal when they designate the same built-in routine
Tables, closures, and Threads	Compare reference identity within the same category

Closures and built-in routines both have type name `function`, but remain distinct value categories for equality. A Number and its textual spelling compare unequal. Two Table references compare equal only when they refer to the same Table, even if separate Tables have matching entries.

5.2.3 `type(value)`

`type` returns exactly one String naming the value's category. Omitting the argument is equivalent to passing `nil`; further arguments are ignored. The complete set of returned labels is:

Value category	Result
Nil	`'nil'`
False or True	`'boolean'`
Number	`'number'`
String	`'string'`
Thread	`'thread'`
Table	`'table'`
Closure or built-in function	`'function'`

These are exact, lowercase labels. In particular, Boolean values do not have separate `'true'` and `'false'` type labels, and the result does not distinguish the two callable categories. The function classifies the value itself rather than attempting to parse or convert it.

5.2.4 `tostring(value)`

Returns one String using the same value conversion as `'' .. value`. An omitted argument converts as nil; surplus arguments are ignored. `__tostring` does not participate in this conversion.

Value	String contents
`nil`	`nil`
`false`	`false`
`true`	`true`
String	Its contents, unchanged
Thread	`thread`
Table	`table`
Closure or built-in function	`function`
Number NaN	`nan`
Negative infinity	`-inf`
Positive infinity	`inf`

The category words are literal text, not addresses, names, or structural dumps. Number conversion uses the CPU chapter's **default Number-to-String formatter**. Positive and negative zero are preserved as `0` and `-0`; exact integer magnitudes strictly below `1000000` use the formatter's integer shortcut. Consult that formatter's contract for finite normalization and rounding. `tostring` has no formatting argument and does not offer a precision or notation selector.

5.2.5 `tonumber(value [, base])`

`tonumber` returns exactly one Number on a successful conversion, or exactly one `nil` when a supported parse fails. Some explicit-base argument errors raise a value instead of returning normally. The omitted-base and explicit-`nil`-base forms follow the same rules:

`value` category, with no nonnil base	Result
Number	The original Number, unchanged
String	The parsed Number, or `nil` if parsing fails
Any other category, including an omitted value	`nil`

An explicit nonnil base selects a different contract: the value must be a String, and the base must pass the checks below. A Number value is not accepted by this form even when it could be returned unchanged without a base. Arguments after `base` are not parameters.

5.2.5.1 Character and token rules

Both parsing forms recognize whitespace only as character code `32` or codes `9` through `13`, inclusive. Leading and trailing whitespace are permitted. Whitespace inside the numeral is not skipped. A sign, when allowed, is one leading `+` or `-` after leading whitespace.

The digit alphabet is ASCII `0` through `9`, then `A` through `Z` or `a` through `z`. Decimal digits have values zero through nine; either letter case maps to values ten through thirty-five. A digit is usable only when its value is less than the selected base. Other characters are not digits.

A successful parse must consume the entire String apart from the permitted leading and trailing whitespace. A missing required digit, an invalid digit, or any remaining non-whitespace character makes parsing return `nil`. There is no successful-prefix result that silently discards the rest of a String.

5.2.5.2 Default String form

After leading whitespace and an optional sign, an initial `0x` or `0X` selects an integer hexadecimal form. It requires at least one hexadecimal digit, accumulates those digits in base sixteen, and then accepts only trailing whitespace and the end of the String. This form has no hexadecimal fractional part or hexadecimal exponent.

Without that prefix, the form is decimal. It contains decimal integer digits, an optional decimal point and decimal fractional digits, or a decimal point followed by fractional digits. At least one digit must occur across the integer and fractional parts: `.25`, `12.`, and `12.25` are accepted forms, but a bare decimal point is not. A leading zero without the hexadecimal prefix does not select another base.

The decimal form may end with an exponent introduced by `e` or `E`. The exponent may have one `+` or `-` sign and must have at least one decimal digit. Only trailing whitespace may follow it. Thus neither an exponent marker alone nor an exponent sign without digits completes a numeral.

Part	Default decimal	Default prefixed hexadecimal
Leading whitespace	Permitted	Permitted
Leading sign	Optional	Optional, before the prefix
Prefix	None	`0x` or `0X`
Required digits	At least one across integer and fractional parts	At least one hexadecimal digit
Fraction	Optional decimal point and decimal digits	Not supported
Exponent	Optional `e` or `E`, optional sign, required decimal digits	Not supported
End	Trailing whitespace, then end of String	Trailing whitespace, then end of String

5.2.5.3 Explicit-base String form

The supported bases are integer Numbers from `2` through `36`, inclusive. After leading whitespace and an optional sign, this form reads one or more digits valid for that base, then permits only trailing whitespace and the end of the String. It does not accept a fractional point or provide a separate exponent syntax.

No prefix is stripped in the explicit-base form. In particular, selecting base sixteen does not give `0x` or `0X` special treatment: their characters are processed by the same digit rules as every other character. Letters such as `e` are digits when their values fit the chosen base, not exponent markers. Use the default form, rather than an explicit base, when requesting decimal fractions or decimal exponents.

5.2.5.4 Argument-check order and errors

For a nonnil base, the installed function performs these operations in order:

1. Check that `value` is a String. Otherwise, raise the exact String `bad argument #1 to tonumber (string expected)`.
2. Compute `integer_base = base // 1`. If `integer_base` differs from `base`, raise the exact String `bad argument #2 to tonumber (integer expected)`.
3. Require `integer_base` to be between `2` and `36`, inclusive. Otherwise, raise the exact String `bad argument #2 to tonumber (base out of range)`.
4. Parse the String using that base, returning a Number or `nil`.

Supply the base as a Number. The String-value check precedes its arithmetic and range checks; a numeric String supplied as the base fails the integer comparison. Error reporting uses the current global `error`, and kind checks use current `type`. Protected calls receive a raised value.

5.2.5.5 Arithmetic and execution cost

The explicit-base parser accumulates each digit as `value * base + digit`, using Number arithmetic. Decimal integer accumulation starts at `0` and processes digits left to right as `value = value * 10 + digit`. Fractional scanning starts with divisor `10`; each digit contributes `digit / divisor` to the current value, after which the divisor is multiplied by `10`. It does not collect all fractional digits for one final division. Exponent magnitude likewise starts at `0` and accumulates left to right as `exponent = exponent * 10 + digit`. Its signed magnitude controls repeated multiplication or division by ten, changing the exponent by one step each time. After scaling and successful trailing-whitespace/end validation, the decimal parser applies the leading sign as `sign * value`. These successive operations are not a single ideal decimal conversion: rounding and range behavior follow the arithmetic actually performed. Exponent scaling has no iteration cap or finite-value rejection. Very large parsed exponents do not carry a termination or bounded-running-time guarantee.

There is no fixed total cycle count for `assert`, `rawequal`, `tostring`, or `tonumber`: their executed operations, result handling, conversions, and parsing work contribute to the cost. `type` adds one cycle, separate from call and return instruction costs, with no additional per-argument or per-result charge.

5.3 Table and metatable access

Use raw access to inspect or change a Table's own mapping, or metatable access to inspect or replace its associated metatable. The CPU Table rules define inherited reads, direct writes and weak references.

All four functions below require a Table as their first argument; otherwise they raise `INVALID_ARGUMENT` (10). Ordinary BLua source must supply every listed argument. Pass explicit nil to delete a mapping or clear a metatable.

Signature	Operation and result
`setmetatable(t, mt)`	Sets the metatable to Table `mt`; nil clears it. Returns `t`. Any other `mt` raises `INVALID_ARGUMENT` (10).
`getmetatable(t)`	Returns the actual metatable Table, or nil. No protected-metatable substitution is performed.
`rawget(t, key)`	Returns only the own mapping, or nil when missing. Ignores the metatable.
`rawset(t, key, value)`	Writes only the own mapping and returns `t`. Nil `value` deletes it.

For `rawget` and `rawset`, a nil key raises `INDEX_NIL` (4). `rawset` uses the ordinary stored-length mutation rules.

5.4 Traversal, argument selection, globals, and output

Choose entry traversal for all own Table mappings, or indexed iteration for a sequence ending at nil. Use `select` when the supplied argument count, including nil positions, must be preserved. Name-based global access and console output use the same ordinary call and result-adjustment rules.

5.4.1 `next(table [, key])`

`next` traverses entries stored in a Table. Its first argument must be a Table; an omitted argument, nil, or another value faults with `Attempted to iterate a non-table value.` The optional second argument is the continuation key. Omitting it is the same as supplying nil. The key is used in its existing value representation, without a conversion by this function. Arguments after the second are unused.

A successful step returns exactly two results, `(key, value)`. Array keys are Numbers, and the returned value retains its stored representation. Exhaustion returns exactly **one nil**, not zero results and not a pair of nils. Starting with a nil key visits live array slots in ascending order, followed by live hash slots, according to the Table's current storage. This is not a guarantee of insertion order or a sorted ordering of all keys.

Use the returned key to continue traversal of the same Table. An array key within the current capacity resumes after that slot even if its value has become nil. An unrecognized non-array continuation key ends traversal. Removal can retain a

recognizable continuation position, so a missing mapping alone does not decide that outcome. Changes that reorganize Table storage can change traversal; keep the Table's entry structure stable when a complete single traversal is required.

`next` visits stored entries, not values supplied by inherited ordinary indexing. Its returned key is also the continuation control for a subsequent step. Test that key for nil when detecting exhaustion: false is not an exhaustion marker.

5.4.2 `pairs(target)`

Returns the three values `next, target, nil`, selecting the current global `next`. The first iterator call checks the target through that function. `__pairs` is ignored. Replacing `next` therefore changes the iterator returned by subsequent `pairs` calls.

A generic `for` obtains three values from its iterator expression: iterator, state, and initial control. Each step calls the iterator with `(state, control)`. The loop ends only when the first result is nil; false does not end it. Otherwise that first result becomes the next control. With the installed `next`, this mechanism traverses the stored entries described above. `pairs` adds no separate ordering or mutation guarantee.

5.4.3 `ipairs(target)`

Returns `ipairs_iterator, target, 0`. The iterator begins at key 1 and uses ordinary indexed reads. It is the BIOS sequence iterator, independent of the current global `next`; `__ipairs` is ignored. The target is first accessed when the iterator runs.

Each iterator step computes `next_index = index + 1` and reads `target[next_index]` using ordinary indexing. If the read produces nil, the step returns exactly one nil. Otherwise it returns exactly two results, `(next_index, value)`. A false value is therefore an ordinary sequence element and is returned, not treated as a stopping condition. A nil encountered at an index stops this traversal even when later indexes contain values.

Ordinary indexing is important here: the step is not a raw Table-entry lookup. A Table can supply a value through its ordinary metatable index resolution, and such a value can keep `ipairs` running where a stored entry alone would not. By contrast, `next` traverses actual storage. Ordinary indexing also accepts a String through its String-index table; that acceptance is not a promise that numeric String indexes return characters or substrings. A value that cannot be indexed faults when the iterator step reaches the read, rather than during construction of the `ipairs` triple.

`ipairs` never reads Table length. Its limit is the first nil returned by successive ordinary reads, not a length captured before iteration or recomputed at each step. Changes affecting those reads can affect subsequent steps; this is not a separate mutation-safe sequence traversal contract.

5.4.4 `select(selector, ...)`

`select` operates on the arguments after its selector. The exact String selector `'#'` returns exactly one Number: the number of those arguments, including every nil position. An omitted selector, nil, any other String, or any other non-Number selector faults with `Invalid argument.` Numeric selectors take a different path; they are not required to be integer-valued before conversion.

A numeric selector is converted to a signed 32-bit register word: discard its fractional part toward zero, reduce the resulting integer modulo 2^32, then interpret bit 31 as the sign bit. NaN and either infinity convert to zero. Let `s` be that converted value and `count` the number of remaining arguments. If `s` is negative, its starting position is `count + s + 1`; otherwise its starting position is `s`. The selected suffix is then restricted to argument positions `1` through `count`. Conversion precedes both the negative-offset calculation and the bounds filtering.

Converted selector	Results
`1` through `count`	That argument and every following argument
`-1`	The final argument, when one exists
`-count`, for a nonempty argument list	All arguments
`0`, or a value more negative than `-count`	All arguments
A positive value greater than `count`	Zero results

Results keep their original values and nil positions. With an empty tail, a numeric selector returns zero results while `'#'` returns the Number zero. A zero or overly negative selector returns the complete tail. An overly large positive selector returns zero results; a fixed receiving list then fills its variables with nil through ordinary result adjustment.

5.4.5 `getglobal(name)` and `setglobal(name, value)`

`getglobal` requires a String name without coercion. An omitted name or a non-String name faults with `Invalid argument.` It returns exactly one value: the current value of the named ordinary global, or nil if the name is unknown. Arguments after the name are unused. This is a live read, not a captured copy of a global's value from an earlier point in execution.

`setglobal` requires the same uncoerced String name and uses the same argument fault for an omitted or non-String name. It writes its second argument unchanged to that ordinary global. Omitting the second argument writes nil; later arguments are unused. A previously unknown name receives an ordinary global slot, including when the value written is nil. The function itself returns **zero results**. A caller requesting one result nevertheless receives nil through normal result padding.

Use an explicit nil as the second argument to `setglobal` in BLua source to meet the known-call minimum. Writing nil changes that one binding; it does not clear the global bank.

These calls select the ordinary global bank. Lexical locals and captured variables with the same spelling remain separate, as does the system-global namespace. Values retain their identity, so storing a Table stores a reference to that Table. `setglobal` writes when called, and that write remains visible after a later error.

5.4.6 `print(...)`

`print` writes one console field for every argument position, with one tab between adjacent positions and no leading or trailing separator. Nil arguments are counted and converted rather than omitted; a list consisting entirely of nils still has one field per argument. After the fields it ends the line. Calling `print` with no arguments also ends a line.

Argument selection uses current `select`; conversion uses current `tostring`. The destination remains the BIOS console when either of those global bindings is replaced. The conversion rules appear under Value checks and conversion.

Successful `print` returns **one nil**. In contrast, `setglobal` returns zero results. A single-result assignment receives nil in either case.

5.4.6.1 Console history and display

The BIOS console sends each codepoint to SYSTEM's PRINT_CHAR port and retains a separate one-byte history entry. Codes through 255 are retained unchanged; larger codes are recorded as 63 (`?`). The original codepoint still goes to PRINT_CHAR. Ending the line appends code 10 to the history and writes 1 to PRINT_FLUSH. The history holds 8192 entries; once full, each new entry replaces the oldest unread entry.

The console draws its retained entries separately. `print` appends text and ends the diagnostic line, but does not itself update the console display. Direct PRINT_CHAR and PRINT_FLUSH writes use only the diagnostic channel: they neither add BIOS history nor draw glyphs. The channel has its own 8192-byte queue and overflow rules, specified in Book I, "System control and timing", under "Diagnostic text output".

For a comparison of the resulting text and monitor display, see BIOS diagnostics — Diagnostic text and the BIOS console.

5.4.7 Faults and incremental cycle charges

Keep iterator construction separate from iterator execution when handling errors. `next` checks its Table argument when called. `pairs` and `ipairs` construct their triples without checking the target; faults arise when their iterators attempt their respective traversal or indexing operations. The primitive argument faults used here have the messages `Attempted to iterate a non-table value.` and `Invalid argument.` as specified above. Ordinary indexing and calls retain their own failure behavior. See Protected calls and coroutines for receiving failures.

Each invocation of `next`, `select`, `getglobal`, or `setglobal` adds **one cycle**, with no additional per-argument or per-result charge. This increment is separate from call and return instructions and the surrounding operation. `pairs`, `ipairs`, and `print` execute ordinary instructions, including their calls, branches, indexing, conversion, and console work. The one-cycle increment is not their total call cost. See Book I, "System control and timing" for the execution and timing model.

5.5 Programming examples

The following sequences use the installed globals directly. They illustrate conversion, traversal, argument tuples and console output.

5.5.1 Assertions and numeric text

A successful assertion preserves its full argument tuple. A failed assertion with an explicit nil message is still a failure: use the protected-call flag, not the error value, to distinguish it from success.

```
local first, label, gap, last = assert(0, 'kept', nil, 7)
-- first = 0, label = 'kept', gap = nil, last = 7.
local count = select('#', assert(0, 'kept', nil, 7)) -- 4
local ok, value = pcall(assert, false, nil)          -- false, nil
local default_ok, message = pcall(assert, false)
-- default_ok = false; message = 'assertion failed!'.
```

Numeric text is accepted only when the entire numeral and permitted trailing whitespace are consumed. A malformed numeral returns nil; an invalid explicit base raises an error instead.

```
local unchanged = tonumber(12.5)      -- 12.5
local decimal = tonumber(' -12.5e1 ') -- -125
local fraction = tonumber('.25')     -- 0.25
local hex = tonumber('0x10')          -- 16
local binary = tonumber('-101', 2)    -- -5
local base36 = tonumber('z', 36)      -- 35
local rejected = tonumber('12x')     -- nil
local ok, message = pcall(tonumber, '10', 37)
-- false, 'bad argument #2 to tonumber (base out of range)'.
```

5.5.2 Identity and traversal

Equal-looking Tables remain distinct objects. Their text labels describe the category, not their contents or identity.

```
local inventory = {apples = 11, pears = 22, plums = 33}
local same = inventory
local other = {apples = 11, pears = 22, plums = 33}
local shared = rawequal(inventory, same)   -- true
local distinct = rawequal(inventory, other) -- false
local category = type(inventory)          -- 'table'
local label = tostring(inventory)         -- 'table'
local total = 0
for name, quantity in pairs(inventory) do
    total = total + quantity
end
-- total is 66, independently of the order of named keys.
```

`ipairs` stops at the first nil read; it does not search ahead for another populated key. `next` instead visits stored mappings and returns one nil when exhausted.

```
local sequence = {10, 20, nil, 90}
local total = 0
for index, quantity in ipairs(sequence) do
    total = total + quantity
end
-- total is 30; sequence[4] remains 90.
local singleton = {[4] = 41}
local key, value = next(singleton)       -- 4, 41
local finished = next(singleton, key)    -- nil
local result_count = select('#', next(singleton, key)) -- 1
```

5.5.3 Tuples and globals

Count tuple positions with `select`, not Table length. An empty selected suffix has no results; a suffix beginning with nil still contains that position.

```
local count = select('#', 'skip', nil, 9)             -- 3
local first, last = select(2, 'skip', nil, 9)           -- nil, 9
local suffix_count = select('#', select(2, 'skip', nil, 9)) -- 2
local empty_count = select('#', select(4, 'skip', nil, 9))  -- 0
```

For a global owned by the cartridge, both a global-name expression and `getglobal` see writes made with `setglobal`. Protection does not reverse an earlier write.

```
setglobal('score_counter', 11)
local before = score_counter -- 11
local ok, message = pcall(function()
    score_counter = 22
    error('counter changed')
end)
local after = getglobal('score_counter') -- 22, despite the error
setglobal('score_counter', nil)
```

5.5.4 Console output

```
print('Score', nil, 7)
```

This produces three fields, `Score`, `nil` and `7`, separated by tabs, then ends the line. `select('#', print())` ends an empty line and returns 1: the successful `print` result is one nil, not an empty tuple.

6 BIOS runtime compilation

The BIOS compiles procedures during program execution. Supply a text block or construct its syntax with the installed factory, then retain the returned closure and call it when the procedure is needed. The same closure can be called repeatedly, and the procedure can create child closures of its own.

Choose the input representation according to how the procedure is supplied:

Input	BIOS interface	Preparation
Source text	`load` or `lua_compiler.load`	Supply a String containing the runtime source language. The BIOS parses and compiles it.
Constructed syntax	`lua_compiler.compile_syntax`	Build a chunk with `lua_compiler.syntax_factory`. The BIOS compiles its supplied operations and bindings directly.

Both routes produce ordinary callable closures. Compilation determines the code and name-access routes; invocation performs the procedure's statements. An environment or external binding can supply the values that its unresolved names will access. Keeping the closure alive retains its generated code.

Runtime source has the language forms specified below. It supports executable blocks rather than cartridge module and storage declarations. To use existing Tables, functions or hardware access routines, supply them through an environment or external bindings. The BLua program reference defines program-image source and module declarations separately.

6.1 Programming runtime compilation

6.1.1 Compile first, invoke explicitly

The installed environment supplies `load`. Compiling the first chunk below leaves `environment.total` at zero; only its invocations change the field.

```
local environment = { total = 0, step = 3 }
local chunk, message = load(
    'total = total + step return total', 'counter', 't', environment)
if chunk == nil then
    error(message)
end
local first = chunk()
environment.step = 5
local second = chunk()
```

`first` is 3 and `second` is 8. The same closure accesses the same environment; compilation did not copy its fields. Compare the first result with `nil` to recognize failure: the second result can be any error payload, including nil. Calling the closure is a separate operation with its own error boundary.

6.1.2 Construct syntax without parsing text

The expression `6 * 7` has two Number operands and a multiplication operation. To return its value, place that expression in a return statement, place the statement in a block, and make that block the chunk's body. The constructors describe these operations; they do not execute the multiplication.

Use the installed factory so each syntax occurrence has its own node:

```
local factory = lua_compiler.syntax_factory
local syntax = factory.syntax
local chunk = lua_compiler.compile_syntax(
    factory.chunk(factory.block({
        factory.return_statement({
            factory.binary_expression(syntax.binary_multiply,
                factory.number_literal(6), factory.number_literal(7)),
        }),
    })), 'syntax')
local result = chunk()
```

`result` is 42. `compile_syntax` has no load-style returned-error wrapper; use `pcall(lua_compiler.compile_syntax, ...)` when a protected result is required. A generated symbol can be shared as a key, but its declarations and each use must be separate syntax nodes. Compilation annotates those occurrences in place, so construct a fresh tree for each attempt. The factory reference specifies constructors, node ownership, result counts and diagnostics.

6.1.3 Names, environments and captured bindings

Local, parameter, and upvalue bindings take precedence. For a name not resolved there, compilation selects external-scope metadata first, then the environment route when `environment ~= nil`, otherwise the global-name route.

Environment-name accesses use the supplied value when the procedure runs, so an environment Table remains shared with its caller. Captures instead refer to bindings in the compiled procedure's enclosing functions.

A name's spelling and the lifetime of its value are separate questions:

Name route	What the compiled procedure uses when invoked
Local or parameter	Its current invocation's value slot
Captured enclosing local or parameter	The binding retained by the child closure
External name	The selected descriptor and its retained read/write callback
Environment name	An indexed read or write on the supplied environment value
Ordinary global	A name-based read or write through the retained global accessor

The enclosing bindings here belong to the procedure being compiled. They are not arbitrary locals in the BLua function that calls the compiler. The first example supplies `total` and `step` through its environment, so changing `environment.step` changes the next invocation's read.

6.1.3.1 Bind an external location

An external descriptor takes precedence over an environment field of the same name. Its callbacks access the supplied location rather than a captured copy:

```
local binding = { available = true, is_type = false, is_const = false,
    value = 10 }
local environment = { external_value = 1000, step = 2 }
local chunk, message = load(
    'external_value = external_value + step return external_value',
    'external', 't', environment, {
        names = { external_value = binding },
        read = function(location) return location.value end,
```

```
        write = function(location, value) location.value = value end,
    })
if chunk == nil then
    error(message)
end
local first, second = chunk(), chunk()
```

The calls return 12 and 14; `binding.value` becomes 14 while `environment.external_value` remains 1000. `step` has no external descriptor and uses the environment. These descriptors own ordinary storage. Accessors borrowing a selected frame instead expire when that evaluation scope closes, even if a returned closure retains its code.

6.1.4 Runtime-compiled code: storage, ownership and lifetime

Runtime-compiled procedures occupy the BIOS code arena in main RAM. One compilation places its root procedure, nested procedures, capture descriptors and instructions in one block. Closures retain the block and its associated constant Table while the code remains in use.

6.1.4.1 What keeps compiled code alive

The root closure retains the compilation's constant-pool table, which owns the code allocation. That reference remains valid after the loader returns.

Every child closure also retains the allocation owner, in addition to its required captures. An escaping child therefore keeps the entire compilation's code block alive, even if the root closure is no longer reachable or the child needs only some of the compilation's constants. Individual functions within that block are not reclaimed separately.

Keep a closure reachable while its compiled code is needed. Once all references retaining that compilation become unreachable, collection makes its block reusable. A raw function address does not retain the block. Release language references rather than attempting to free a code block explicitly; reclamation occurs during collection, not at the assignment that drops a reference.

6.1.4.2 Retain an escaping child

```
local environment = { offset = 10 }
local factory, message = load(
    'return function(value) return value + offset end',
    'capture', 't', environment)
if factory == nil then
    error(message)
end
local child = factory()
factory = nil
local first = child(5)
environment.offset = 20
local second = child(5)
```

The results are 15 and 25. Keeping the child retains the compilation's code allocation after the factory reference is dropped. It does not freeze the environment or promise when an unreachable allocation will be reclaimed.

6.1.4.3 Environments and borrowed bindings

When a compiled identifier requires an environment lookup, the constant pool retains the actual supplied environment value and the key. This is not a private copy of the environment. Supplying an environment that is never added to the constant pool does not guarantee that compilation will retain it.

An external binding retains its location and read/write accessors, not a copied binding value. Keeping those accessors alive does not extend the lifetime of a borrowed frame.

For evaluation against a selected BIOS monitor frame, the binding accessors borrow that frame. The evaluator closes the binding scope before returning its result, including when the result contains an escaping closure. A later read or write through a retired accessor raises:

```
Selected frame evaluation has ended.
```

Closing a coroutine also retires its borrowed frame scopes. Code retention and binding validity are separate: a surviving closure can retain its compiled code while access to a borrowed binding has already ended.

6.2 Runtime compilation interface

The BIOS installs `lua_compiler` and assigns its `load` function to the global `load` binding. Compilation produces a callable closure; it does not execute the compiled chunk.

Entry	Interface	Result
`load` or `lua_compiler.load`	`load(source, chunk_name, mode, environment, external_scope)`	Protected text compilation.
`lua_compiler.compile_syntax`	`compile_syntax(chunk, chunk_name, environment, external_scope)`	Direct compilation of a syntax chunk, without text parsing.
`lua_compiler.syntax_factory`	Syntax-construction interface	Supplies the syntax factory; its construction contract is separate from loading.

6.2.1 Text loading

`load` has five fixed parameters, in this order:

Position	Parameter	Omission or default	Use
1	`source`	Missing fixed argument becomes `nil`; no source default is supplied.	Text input passed to the parser.
2	`chunk_name`	`nil` or `false` selects `"=(load)"`.	Chunk label passed to parsing and compilation. Other values, including `""` and `0`, remain supplied values.
3	`mode`	Only `nil` bypasses the mode scan and accepts text.	A supplied String must contain lowercase `t`.
4	`environment`	`nil` selects ordinary global-name routing.	A non-`nil` language value supplies the environment-name route.
5	`external_scope`	An omitted argument becomes `nil`.	Forwarded to compilation for external-name binding.

Mode acceptance happens first, before the chunk-name default or text parsing. For a supplied String, the entire acceptance test is whether a lowercase `t` occurs. This is not an exact-mode-name or allowed-character check.

Mode	Effect
`nil`	Accept text without scanning.
`"t"`, `"bt"`, or another String containing `t`	Accept and parse text.
`""` or `"b"`	Reject before parsing.

Acceptance never selects a binary decoder. The scan uses one-based String codepoint positions, not UTF-8 byte offsets. Non-String values encounter the same length and String-codepoint operations, not a separate mode-type validator. A Table

supplies its maintained length to the length operation. In particular, `false` reaches a length failure before parsing, with the reason String `"Attempted to get length of an unsupported value."`.

A rejected mode calls the current global `error` with `"attempt to load a text chunk (mode is '" .. mode .. "')"`. This describes the error value passed to that call, not a display prefix or stack-trace format.

6.2.2 Results and execution

Outcome	Exact returned values
Successful compilation	One value: the compiled closure.
Failure returned through the protected boundary	Two values: `nil, errorValue`.

`load` retains only the protected call's first result after its success flag. It does not normalize every failure into a String. An explicitly thrown language value is preserved, so `errorValue` can itself be `false` or `nil`. Classified execution and out-of-memory failures supply a reason String. Use the first result to distinguish a closure from failure; do not require the error value to be truthy or non-`nil`.

The protected result convention applies to language errors, not hardware exceptions. It leaves earlier writes in place. An intervening hardware exception or interrupt also prevents protection from unwinding across that boundary; hardware-access and opcode faults do not become a `nil, errorValue` pair.

Program in this order:

1. Select the text source, chunk label, mode, and any environment or external scope.
2. Call `load` with those arguments and retain its first and second results.
3. If the first result is `nil`, handle the second value as the failure payload, including a possible `nil` payload.
4. Otherwise, invoke the returned closure explicitly when execution is wanted.

6.2.3 Retained loader dependencies

Generated environment-name accesses retain the supplied language value, not a copied set of globals. Environment-name reads and writes perform indexed operations directly against that value. `false` is a supplied environment, not an omission: it selects this route too. The loader does not add an environment-type check. Without an environment, generated global accesses use retained global-accessor function values and textual keys. Reassigning public `getglobal` or `setglobal` does not replace those retained dependencies.

The initial global `load` and `lua_compiler.load` bindings contain the same function value. Later assignment to one does not reassign the other. The loader also retains its protected-call primitive, so replacing public `pcall` does not replace that dependency. Each call looks up the compile entry on the compiler Table retained by the loader; it does not dispatch through the current global `load` or public `lua_compiler.load` binding. Mode scanning uses a retained String-codepoint primitive, while mode rejection uses the current global `error` binding.

6.2.4 Direct syntax compilation

`lua_compiler.compile_syntax(chunk, chunk_name, environment, external_scope)` is a direct function-value alias for syntax-chunk compilation. It produces a closure without invoking it. Unlike `load`, this route does not perform a mode check, parse text, supply the `"=(load)"` chunk-name default, or convert a protected failure into `nil, errorValue`. It has four fixed parameters; omitted fixed arguments become `nil`.

6.2.5 Written calls and reached calls

The published source-call signatures and the reached functions differ in one specific respect:

Published source-call signature	Required written expressions	Actual fixed parameters
`load(chunk, chunkname?, mode?, env?)`	1	5, including `external_scope`

Published source-call signature	Required written expressions	Actual fixed parameters
`lua_compiler.compile_syntax(chunk, chunkname, env?)`	2	4, including `external_scope`

For recognized calls, this source-call check rejects a written expression count below the required count; it does not impose a maximum. The omitted `external_scope` entry is not a runtime argument limit. A reached call fills missing fixed parameters with `nil` and does not retain surplus arguments in these non-vararg functions. That runtime behavior does not establish permission for every shortened source call or general source-input acceptance.

6.2.6 Compilation work

The retained protected-call primitive has a base charge of **4 cycles**, with zero additional primitive charge per argument or return value. This is an incremental protected-call charge, not the duration of `load`. It excludes the loader's own execution, mode scanning, parsing, semantic analysis, code generation, linking, allocations, callees, instruction reads and writes, and caller call/return work. No fixed total compilation time follows from it.

6.3 Runtime-loaded source language

`load` accepts the source forms in this section. Its input is an executable block, not a list of declarations: statements and a final `return` may appear at the top level. Compiling the block produces a function; it does not execute the block.

This language is narrower than BLua. A source file accepted for linking is not necessarily accepted by `load`.

6.3.1 Names, spacing, and comments

Element	Runtime source form
Name	First character: ASCII `a`–`z`, `_`, or `$`. Later characters may also be decimal digits. Uppercase letters are not permitted in names.
Whitespace	Space and character codes 9 through 13.
Line comment	`--` through the next LF or the end of the source.
Statement separator	`;` is optional and may also appear as an empty separator.

There are no long comments or long quoted strings. LF advances the source line; other whitespace does not. String positions and lengths use codepoints, not UTF-8 byte offsets.

6.3.2 Numeric literals

Form	Examples and requirements
Decimal integer	`0`, `27`, `1234`
Decimal fraction	`12.5`, `.5`; a fractional dot must be followed by a digit.
Decimal exponent	`1e3`, `2.5E-2`; `e` or `E` may be followed by a sign, then must have decimal digits.
Hexadecimal integer	`0x2a`, `0X2A`; at least one hexadecimal digit is required.

A leading sign is an operator, not part of the numeric literal. These spellings do not specify additional numeric range or rounding guarantees.

6.3.3 Quoted strings

Use single or double quotes. An unescaped CR, LF, or end of source before the closing quote is an unfinished string.

Escape	Meaning or accepted form
`\'`, `\"`, `\\`	Quote or backslash.
`\a`, `\b`, `\f`	Alert, backspace, form feed.
`\n`, `\r`, `\t`, `\v`	Newline, carriage return, tab, vertical tab.
Decimal escape	Backslash followed by up to three decimal digits; the value must not exceed 255.
Hexadecimal escape	`\x` followed by exactly two hexadecimal digits.
`\z`	Skip subsequent whitespace.

6.3.4 Expressions and suffixes

Primary expression	Form
Name	`name`
Literal	Number, quoted string, `false`, `true`, or `nil`
Anonymous function	`function(parameter, ...) ... end`; the displayed ellipsis means more named parameters, not a vararg token.
Parenthesized expression	`(expression)`

Function parameters are zero or more comma-separated names. Duplicate names within one function's parameter list are rejected. There is no vararg parameter or vararg expression.

Suffixes may be chained after a primary expression:

Suffix	Form
Named member	`.name`
Indexed member	`[expression]`
Ordinary call	`(argument, ...)`
Method call	`:name(argument, ...)`

Calls require parentheses, including calls with no arguments. The commas in a call separate zero or more expressions.

A member/index path must begin at a visible name. Thus `items[index].value` has the required shape, but `make_items().value` does not: a member or index path cannot begin at a call result, literal, or function expression. This restriction does not by itself exclude `make_items():method()`, whose receiver belongs to a method call rather than a member/index path.

6.3.5 Operators

Unary operators bind more tightly than the binary operators below.

Unary form	Operation
`-expression`	Negation
`not expression`	Logical negation
`#expression`	Length
`&expression`	String-ID form. Evaluate the operand once as one value and retain its String value unchanged. The result is not a numeric address or pointer; literals, variables, indexed values and calls can supply the String.

Binary precedence is listed from highest to lowest. Every row is left-associative.

Precedence	Operators
1	`*`, `/`, `//`, `%`
2	`+`, `-`
3	`&`
4	`==`, `~=`, `<`, `<=`, `>`, `>=`
5	`and`
6	`or`

Parentheses override binary precedence. The unary String-ID form and binary `&` occupy different expression positions.

6.3.6 Statements

The following forms are available in a block. In these forms, `block` means a sequence of statements, and bracketed text denotes optional source.

Statement	Form
Local	`local name [<const>] [= expression]`
Assignment	`target = expression`
Call statement	An ordinary call or method call.
Return	`return` or `return expression [, expression ...]`

An assignment has exactly one target and one value expression. Its target is a name or a member/index path. A bare expression that is neither an assignment nor a call is not a statement.

Control statement	Form
Conditional	`if expression then block [elseif expression then block ...] [else block] end`
While loop	`while expression do block end`
Numeric loop	`for name = start, limit [, step] do block end`
Loop exit	`break`

The numeric loop evaluates its start, limit, and supplied step expressions into loop locals. An omitted step is `1`. `break` must be inside a loop in the current function; a function nested inside a loop does not inherit permission to break that loop.

A `return` must be the final non-semicolon statement of its immediate block. It need not be the final statement of an enclosing block: for example, an `if` branch may end in `return` while the containing block continues after the `if`.

6.3.7 Locals, closures, and names

Rule	Effect
Uninitialized local	`local name` initializes the local to `nil`.
Constant local	`local name <const> = expression` requires an initializer. `const` is the only accepted local attribute.
Constant assignment	Direct reassignment to a constant binding is rejected, including a captured constant or constant supplied binding. Member/index writes through that binding are not prohibited by this rule.
Initializer scope	The initializer is bound before the new local becomes visible.
Block scope	Leaving a block restores any previous same-name local binding.
Closure capture	An anonymous function captures visible enclosing local or parameter bindings.

For recursive local functions, declare the local before assigning the function:

```
local f
f = function()
  return f()
end
```

By contrast, in `local f = function() return f() end`, the `f` inside the function does not denote the local being introduced by that declaration.

Names resolve in this order:

Priority	Binding
1	Active local or parameter.
2	Captured enclosing local or parameter.
3	A supplied external binding.
4	A key in the supplied environment, or a global name when no environment is supplied.

An absent external name continues to environment/global lookup; it is not automatically an unknown-name error. A supplied external entry that represents a struct type is not a runtime value, and an unavailable external entry has no live location. Ordinary source names such as `_env` do not independently change this lookup order.

6.3.8 Calls and results

An ordinary call evaluates its callee before evaluating its arguments. A method call evaluates its receiver once, uses it as `self`, selects the named method, and then evaluates the explicit arguments in order.

Context	Result handling
Ordinary value expression	A call supplies one result.
Final argument expression	An unparenthesized call expands its result tuple into arguments.
Earlier argument expression	A call supplies one result.
Final return expression	An unparenthesized call forwards its result tuple after any preceding return values.
Earlier return expression	A call supplies one result.
Parenthesized call	Does not expand its result tuple.
Bare `return`	Returns one `nil` result.
Function fallthrough	Returns one `nil` result.

For example, `return a, f()` forwards the results of `f` after `a`; `return a, (f())` uses one result from `f`. Likewise, `g(a, f())` expands the final call, while `g(a, (f()))` does not.

6.3.9 BLua program forms are not runtime-loaded forms

BLua program form	Runtime-loaded distinction
`module<const>`, `module<entry>`	No module-directive form. These spellings are ordinary names and comparison tokens, not directives.
`struct`, `bss`, `data`, `rodata` declarations	No corresponding declaration or type-reference syntax.
Pointer annotations such as `:*type`	Not part of the local declaration form.
Multiple local names or assignment targets	One local name, or one assignment target and one value expression.
`local function name(...) ... end`	Use a local declaration and an anonymous function expression.
`function name(...) ... end`	No named-function statement; `function` must be followed by `(`.

BLua program form	Runtime-loaded distinction
Local-function `<init>`	Not an accepted local-function form.

Other BLua program form	Runtime-loaded distinction
Generic `for`	Only the numeric `for name = ...` form is available.
`repeat` loop	No repeat-loop statement.
Standalone `do ... end`	No standalone block statement.
`goto`	No goto statement.
Halt-until-interrupt statement	No corresponding runtime-loaded statement.
Table constructor	No `{ ... }` expression.
Varargs	No `...` parameter or expression; dots are separate punctuation.

The absence of a statement form does not reserve every associated spelling as a keyword. For example, `repeat` may be used as an ordinary name in runtime-loaded source.

6.3.10 Source diagnostics

Lexical and parse diagnostics use `[load:<chunk_name>] <message> at <line>:<column>`. Representative messages include `upper-case identifiers are not allowed`, `unfinished string`, `invalid numeric literal exponent`, `expected '<token name>'`, `unsupported expression`, and `expected assignment or function call`.

The whole block is parsed before binding and compilation. Binding also rejects a misplaced `return`, `break` outside a loop, missing constant initializers, direct assignment to constants, duplicate function parameters, and member/index paths without a visible-name root. A function or expression requiring too many registers is rejected with `[load:<chunk_name>] function or expression needs too many registers`, without a source position appended by that diagnostic.

6.4 Constructing and compiling syntax trees

The installed `lua_compiler.syntax_factory` table constructs runtime syntax nodes. Its `syntax` member contains the numeric node tags and operator constants used by those nodes. There is no separate installed syntax global.

```
local f = lua_compiler.syntax_factory
local s = f.syntax
```

Use the factory to construct a complete chunk, then compile it:

```
lua_compiler.compile_syntax(chunk, chunk_name, environment, external_scope)
```

Argument	Meaning
`chunk`	A chunk node whose `body` is a block node.
`chunk_name`	Diagnostic name. Supply a meaningful string; this entry point supplies no default name.
`environment`	Optional environment value; normally a Table for name indexing. Omitted or `nil` selects ordinary global-name access.
`external_scope`	Optional external-name access interface, described below.

Successful compilation returns **one root callable closure**. It does not call that closure. The root has no syntax parameters. Compilation binds and compiles every nested function and links the resulting program; it skips text parsing, not the remaining compilation work. There is no `mode` argument on this entry point. Compilation errors raise directly: this is not an interface returning `nil, message` on failure.

6.4.1 Node ownership, lists, and locations

Each constructor or helper returns one value. Constructors store the supplied children, lists, values, and symbol keys directly; they do not clone them or supply default arguments.

Every tagged node has `kind`, `line = 1`, and `column = 1`, in addition to the fields listed in the reference below. The helper records returned by `generated_symbol`, `if_clause`, and `else_clause` are untagged and have no location fields.

A node is an **owned syntax occurrence**. Construct a fresh node for every declaration, reference, expression, and statement occurrence. Do not put the same node into multiple positions or trees. Share a generated-symbol key when several occurrences must name the same binding, not the identifier node carrying that key.

Binding and compilation annotate nodes in place. Do not recompile an already annotated tree, retain its nodes for another tree, or retry a failed compilation with the same tree. Failure can occur after annotations have been written. Construct a fresh tree for each compilation attempt.

All lists in this interface are ordinary consecutive, 1-based Tables. They are consumed through `#list` and `list[index]`; a packed table's `.n` field has no role. Pass `{}` for an empty statement, parameter, argument, return-expression, or clause list. There are no default empty lists or hole-filling rules.

6.4.2 Chunk, blocks, names, and literals

The names below are members of `lua_compiler.syntax_factory`.

Callable signature	Returned fields beyond the common location fields
`chunk(body)`	`kind = s.chunk`, `body`.
`block(statements)`	`kind = s.block`, `statements`.
`identifier(name)`	`kind = s.identifier_expression`, `name`.
`generated_symbol(name)`	Untagged `{ name = name }` record; no common node fields.
`reference(symbol)`	`kind = s.identifier_expression`, `name = symbol`.

`chunk.body` must be a block node. `block.statements` is a statement-node list. A function body and every conditional or loop block also use a block node, not a raw statement list.

`identifier('x')` creates an occurrence using the spelling string `'x'` as its lexical key. `generated_symbol('x')` creates a distinct table key whose name is used for diagnostic spelling. Two generated symbols with the same display name remain different keys. `reference(symbol)` creates an occurrence carrying the exact supplied symbol key.

Callable signature	Returned fields beyond the common location fields
`number_literal(value)`	`kind = s.number_literal_expression`, `value`.
`string_literal(value)`	`kind = s.string_literal_expression`, `value`.
`boolean_literal(value)`	`kind = s.boolean_literal_expression`, `value`.
`nil_literal()`	`kind = s.nil_literal_expression`; no value field.
`literal(value)`	Selects the number, string, or boolean constructor according to `type(value)`.

Supply the corresponding language value to each literal constructor. Number and string values become program constants or compiler-selected numeric immediates. Boolean emission uses the value's truth of the stored value. `literal(nil)` is **not** equivalent to `nil_literal()`: the dispatch has no nil entry and raises during construction. Tables and functions are also outside `literal`'s dispatch. Use `nil_literal()` to construct nil.

Callable signature	Returned fields beyond the common location fields

6.4.3 Paths, calls, operators, and functions

Callable signature	Returned fields beyond the common location fields
`member_expression(base, identifier)`	`kind = s.member_expression`, `base`, `identifier`.
`index_expression(base, index)`	`kind = s.index_expression`, `base`, `index`.
`index_path(base, path)`	Nested index nodes; each path value is passed through `literal`. An empty path returns the original `base`.
`call_expression(callee, arguments, method_name)`	`kind = s.call_expression`, `expands_results = true`, `callee`, `arguments`, `method_name`.
`binary_expression(operator, left, right)`	`kind = s.binary_expression`, `operator`, `left`, `right`.
`unary_expression(operator, operand)`	`kind = s.unary_expression`, `operator`, `operand`.
`function_expression(parameters, body)`	`kind = s.function_expression`, `parameters`, `body`.

The `identifier` argument of `member_expression` is the **member key itself**, normally a spelling string such as `'ready'`; it is not an identifier node. The `index` argument of `index_expression` is an expression node. A nonconstant index is evaluated at execution; a constant index supplies its constant key.

Member and index bases must form an identifier-rooted path, recursively composed of identifier, member, and index nodes. An arbitrary call or other expression is not a supported path base. `index_path` takes a list of raw number, string, or boolean values, not expression nodes.

For an ordinary call, omit `method_name` or pass `nil`; `callee` is the callable expression. For a method call, `callee` is the **receiver expression**, not an already selected member, and `method_name` is the method key, normally its spelling string. `arguments` is an expression-node list. Do not include the receiver as its first explicit argument: the method form supplies the implicit receiver argument itself. Use `{}` for no explicit arguments.

Unary and binary operators are the numeric constants in `f.syntax`, not source spelling strings, token numbers, or instruction words. `s.unary_string_id` accepts a String-valued expression node. Its operand is evaluated once as one value; a call's additional results are discarded. The result is the same String value, not a numeric pointer or raw address.

A function's `parameters` is a list of **identifier occurrence nodes**, not strings. Its `body` is a block node. Every parameter needs its own occurrence. A generated-symbol parameter is represented with a fresh `reference(symbol)` just like a generated-symbol local declaration.

6.4.4 Assignments, declarations, calls, and conditionals

Callable signature	Returned fields beyond the common location fields
`assignment_statement(target, value)`	`kind = s.assignment_statement`, `target`, `value`.
`local_statement(name, initializer, is_const)`	`kind = s.local_statement`, `name`, `initializer`, `is_const`.
`call_statement(expression)`	`kind = s.call_statement`, `expression`.
`if_clause(condition, block)`	Untagged `{ condition = condition, block = block }` record; no common node fields.
`else_clause(block)`	Untagged `{ block = block }` record; no common node fields.
`if_statement(clauses)`	`kind = s.if_statement`, `clauses`.

`assignment_statement` represents one target and one value expression, **not parallel lists**. Identifier targets use identifier occurrence nodes; member and index targets follow the path rules above. Assignment cannot change a const local, captured const binding, or const external binding.

The `name` of `local_statement` is an **identifier occurrence node**, not a spelling string or a bare generated-symbol record. Use `identifier('x')` or a fresh `reference(symbol)`. An omitted or nil initializer initializes a mutable local to nil. Omitted, nil, or false `is_const` leaves the local mutable; a truthy `is_const` requires an initializer.

The initializer is bound **before** the new local is declared. A same-spelling reference in that initializer resolves to the previously visible binding, not automatically to the local being declared. This constructor does not provide an implicit recursive-local declaration.

`call_statement` requires a **call-expression node** as its `expression`. It is not a general expression-statement constructor.

`if_statement.clauses` is a list of the untagged records made by `if_clause` and `else_clause`. A missing or nil `condition` means unconditional. Each clause's `block` is a block node. Put the unconditional else clause last when constructing an ordinary if/elseif/else sequence.

6.4.5 Loops and returns

The numeric-loop signature is shown separately to keep its argument positions visible:

```
numeric_for_statement(
    variable,
    start_expression,
    limit_expression,
    step_expression,
    block
)
```

It returns a node with `kind = s.numeric_for_statement`, the common location fields, and the five fields named by its arguments. `variable` is an identifier occurrence node, not a string. Start, limit, and a supplied step are expression nodes; `block` is a block node. A nil step selects positive one. To omit the step, pass `nil` in the **fourth** position and the block in the fifth. Variable, start, limit, and block are required.

Callable signature	Returned fields beyond the common location fields
`while_statement(condition, block)`	`kind = s.while_statement`, `condition`, `block`.
`return_statement(expressions)`	`kind = s.return_statement`, `expressions`.
`break_statement()`	`kind = s.break_statement`.

A while condition is an expression node and its body is a block node. A break must be inside a loop in the **same function**; an enclosing function's loop does not make a nested function's break valid. A return must be the final statement of its block. Its `expressions` argument is an expression-node list, with `{}` representing an empty return.

6.4.6 Result counts

All 27 factory callables return one value. `compile_syntax` also returns exactly one value on success: the compiled root closure. Those return counts are distinct from the result counts of calls represented inside the tree.

Constructed use	Result handling
Call used in a single-value expression position	Requests one result.
Call statement	Requests one result internally, then discards its temporary expression storage; the statement returns no language value.
Final argument of another call	Expands the call's result tuple only when the final node is a call expression with truthy `expands_results`.
Final expression of a return list	Uses the same call-kind and `expands_results` rule for tuple expansion.
Fixed return list without a final expanding call	Returns its expression count.
Empty return, or function fallthrough	Returns **one nil value**.

`call_expression` sets `expands_results = true`. To represent a final call restricted to one result, set that owned call node's **existing** `expands_results` field to `false` before compilation. There is no separate factory callable for this operation. It does not require a replacement node format or encoding helper.

6.4.7 Lexical names and captured bindings

Ordinary spelling strings obey lexical shadowing. Generated-symbol occurrences match the exact shared symbol key; their display names do not establish identity. Use a separate fresh occurrence for the declaration and every later reference.

Name resolution considers current locals and parameters, then visible enclosing bindings, which nested functions capture as upvalues. An unresolved **string** key then tries external names and finally the supplied environment or ordinary global access. An unresolved generated-symbol key raises instead of becoming a global.

Nested functions inherit access to the supplied environment and external names. They do not acquire arbitrary locals from the BLua function that called `compile_syntax`. The synthetic root has no enclosing lexical function and an empty parameter list.

6.4.8 Environment and external-name access

Every non-nil `environment` selects environment-name access. Generated reads and writes index the supplied value at execution; there is no environment-type check or snapshot of its entries. Without an explicit environment, named global access uses the compiler's captured global accessors, not mutable public replacement functions.

The consumed external access interface has this shape:

```
external_scope = {
    names = name_keyed_descriptors,
    read = read_binding,
    write = write_binding,
}
```

Each name maps to a descriptor Table. The fields below control binding; additional fields can hold the location data needed by that descriptor's read and write callbacks.

Member or descriptor field	Consumed meaning
`external_scope.names`	Table mapping unresolved spelling strings to their exact binding descriptors.
`descriptor.is_type`	A type-only descriptor cannot be used as a value binding.
`descriptor.available`	An unavailable external location cannot be used.
`descriptor.is_const`	A truthy value prevents assignment to that binding.
`external_scope.read`	Access callback selected for an external read.
`external_scope.write`	Access callback selected for an external assignment.

A compiled external read calls:

```
read_binding(descriptor)
```

A compiled external assignment calls:

```
write_binding(descriptor, value)
```

The descriptor is the exact object selected from `names`; it is not reconstructed. Compilation retains that descriptor and the selected callback for the generated access. These callbacks run when the compiled code performs the access, not as a name-lookup substitute during binding.

Local, parameter, and enclosing lexical bindings take precedence over an external descriptor with the same spelling. A matching external entry is checked for type-only and unavailable status; it does not silently fall through to the environment when unusable. Without a matching external name, the environment or ordinary global route is selected. Const checking also applies to external assignment.

6.4.9 Exported node constants

All constants below are members of `f.syntax`. Node kinds and operators occupy overlapping integer ranges; the field using a value determines its category. No token table is exported by the factory.

Node constant	Value
`chunk`	1
`block`	2
`assignment_statement`	3
`return_statement`	4
`function_expression`	5
`identifier_expression`	6
`member_expression`	7

Node constant	Value
`index_expression`	8
`number_literal_expression`	9
`string_literal_expression`	10
`boolean_literal_expression`	11
`nil_literal_expression`	12
`unary_expression`	13
`binary_expression`	14

Node constant	Value
`call_expression`	15
`local_statement`	16
`if_statement`	17
`while_statement`	18
`break_statement`	19
`call_statement`	20
`numeric_for_statement`	21

6.4.10 Exported unary operators

Operator constant	Value	Category
`unary_negate`	1	Numeric negation.
`unary_string_id`	2	String-ID form; retains the operand’s String value.
`unary_not`	3	Logical negation.
`unary_length`	4	Length.

6.4.11 Exported binary operators

Arithmetic operator constant	Value
`binary_add`	1
`binary_subtract`	2
`binary_multiply`	3
`binary_divide`	4
`binary_floor_divide`	5
`binary_modulus`	6

Comparison operator constant	Value
`binary_equal`	7
`binary_not_equal`	8
`binary_less`	9
`binary_less_equal`	10
`binary_greater`	11
`binary_greater_equal`	12

Logical or bitwise operator constant	Value	Category
`binary_and`	13	Logical.
`binary_or`	14	Logical.
`binary_bitwise_and`	15	Bitwise.

The exported tables are ordinary tables, but changing compiler constants is not a supported customization mechanism.

6.4.12 Syntax diagnostics and reserved annotations

Semantic diagnostics use this form:

```
[load:<chunk_name>] <message> at <line>:<column>
```

Fresh factory nodes therefore report location `1:1`. Supplying a diagnostic chunk name is important: this route does not fill one in before error formatting.

The following conditions explicitly raise during binding:

Condition	Rule
Duplicate parameter keys	Parameter binding keys must be distinct.
Unresolved generated-symbol key	A generated symbol must resolve lexically.
Invalid member/index path root	A path must have a supported visible identifier root.
Assignment to a const binding	Applies to locals, captured bindings, and external bindings.
Const local without initializer	A const declaration needs an initializer.
Break outside a loop of the current function	Nested functions start with no loop context.

Condition	Rule
Non-final return in a block	A return must end its block.
Unsupported statement or literal kind	Only supported kinds can be compiled.
Type-only external entry used as a value	The descriptor is not a value binding.
Unavailable external entry	Its location cannot be used.

Excessive register requirements raise:

```
[load:<chunk_name>] function or expression needs too many registers
```

Invalid `literal` dispatch raises during tree construction, before `compile_syntax` is called. Supply the node and argument forms listed in the factory tables, including the call-statement and String operand restrictions. Incorrect node shapes can fail at a later binding or emission operation rather than at construction.

Supply the constructor inputs listed above. The following fields are reserved for annotations written by BIOS compilation; do not supply precomputed values for them:

Area	Fields written by compilation
Name binding	`binding`, `upvalue`, `external_binding`, `environment_key`, `global_key`.
Paths and constants	`key_value`, `constant_value`, `constant_index`, `accessor_constant_index`, `immediate_index`, `method_constant_index`, `immediate_number`.
Numeric loops	`limit_local_slot`, `step_local_slot`, `zero_constant_index`.
Functions	`semantic_state`, `local_count`, `function_address_constant_index`.

Compilation changes the owned syntax occurrences through these annotations. Construct a fresh tree for every attempt, including after failure.

6.5 Code-arena capacity and allocation

Code allocation and language values use separate capacities. The following limits describe the BIOS arena containing generated function records and instructions. Keep returned closures reachable for as long as their code is required; closure lifetime defines when an allocation can be reused.

6.5.1 Storage limits

Property	Specification
Arena capacity	`0x00040000` bytes (256 KiB) declared storage. The usable span is its 16-byte-aligned interior, not a guaranteed 256 KiB of payload.
Allocation alignment	16 bytes.
Allocation header	16 bytes per allocation.
Allocation size	Header plus linked payload, rounded upward to a multiple of 16 bytes. An unsplittable remainder is consumed by the allocation.
Minimum split remainder	32 bytes: one header plus one alignment unit. Smaller remainders do not become separate free blocks.
Allocation contents	All functions, capture metadata, and instruction streams from one compilation share one block. Constant-pool tables and closures use the separate language-allocation budget.
Final arena-exhaustion error	`runtime compiler memory exhausted`

The arena belongs to the Book I, “Memory and bus access”, under “Reserved and cartridge storage in main RAM”. The capacity above is shared by live compilations; use the separate language allocation budget for Tables, closures and other language objects.

6.5.2 Resource accounting

Linked payload item	Bytes per item	Count
Generated function record	32	F: root and nested generated functions.
Capture descriptor	4	D: actual emitted descriptors, including compiler-required captures.
Instruction word	4	W: actual emitted instruction words.

Reference arithmetic, not BLua source:

Payload bytes = 32F + 4D + 4W.
Block bytes before an unsplittable remainder = $16 \times \lceil (16 + \textbf{payload bytes})/16 \rceil$**.**

These are emitted counts, not counts of user locals or source statements. The block calculation includes the 16-byte header and allocation rounding; a remainder smaller than 32 bytes is additionally consumed rather than split off.

Each generated function has a ceiling of **16,384 total register slots**. This total includes required constants, the allocation owner where present, and temporaries. It is not a guarantee of 16,384 user locals or parameters, nor that every smaller programme will compile. Exceeding the ceiling raises:

```
[load:<chunk_name>] function or expression needs too many registers
```

6.5.3 Allocation, collection, and failure

Allocation first tries the current candidate block. If that cannot satisfy the request, it scans the arena and combines adjacent reusable blocks, stopping at the first sufficiently large run. If that search fails, allocation performs synchronous garbage collection and makes one further scan with coalescing. Failure of that retry raises `runtime compiler memory exhausted`.

A load may perform collection before it returns. The second unsuccessful arena search raises the exhaustion error. Reserve compilation for a point where its variable parsing, allocation and collection time can be accepted.

The language allocation budget is accounted for separately and can also be exhausted. Available language-object capacity does not guarantee a suitable code block, and available code-arena capacity does not guarantee space for closures or constant-pool Tables.

6.5.4 System startup and compiled-code storage

Normal BIOS startup initializes its DATA and BSS, then prepares the runtime code arena before program entry. Starting a program's own DATA and BSS leaves that BIOS arena in place. Keep program sections outside firmware-reserved RAM.

A system reboot clears the execution environment and starts BIOS initialization again. A CPU-only restart or a transfer with `EXEC` has a different state boundary; see CPU execution control and Book I, "System control and timing". System reboot begins a new runtime-compilation lifetime.

7 BIOS diagnostics

The BIOS monitor retains a stopped program's fault state so that its registers, memory and source context can be inspected before continuation. Clearing a report, repairing the cause and resuming execution are separate operations.

During normal execution, a program can send diagnostic text through SYSTEM or use BIOS `print` to retain text for the console display as well. The first example distinguishes those two destinations. The memory examples then follow a fault from inspection to retry. The optional ROM diagnostic directory supplies source locations and named frame scopes; its binary reference follows the examples.

7.1 Diagnostic text and the BIOS console

BIOS `print` sends text to both the diagnostic channel and the console history. Direct SYSTEM port writes send only diagnostic text. In this example, the first and last lines use `print`; the middle line sends four codepoints directly, then publishes them. Assume no other diagnostic writer and enough queue space for each line. A zero FLUSH word is still a flush command.

```
local print_char<const>: *word = 0x08010230
local print_flush<const>: *word = 0x08010234

print('SYSTEM timer experiment')
*print_char = 0x00000053
*print_char = 0x000000e9
*print_char = 0x0001f600
*print_char = 0xffffffff
local char_readback<const> = *print_char
*print_flush = 0
local flush_readback<const> = *print_flush
print('SYSTEM samples complete')
```

CHAR readback is `ffffffffH`, even though the invalid codepoint emits `?`. FLUSH readback is zero. Both are last-written words, not receiver-ready or completion flags. The raw middle line is encoded as:

```
53 C3 A9 F0 9F 98 80 3F 0A
```

The diagnostic text is:

```
SYSTEM timer experiment
Sé😀?
SYSTEM samples complete
```

Only the two BIOS `print` calls also add text to the BIOS console history. The direct codepoint line neither enters that history nor draws glyphs.



Figure 7.1: BIOS console after the diagnostic-output example

The monitor displays the two BIOS print lines from retained console history. The raw diagnostic line does not appear in that history.

7.2 Memory faults and the monitor

A synchronous memory exception enters the **system exception entry**, even when the faulting instruction belongs to a cartridge. This differs from a maskable IRQ, whose entry is selected according to the interrupted mode. Declaring a cartridge `exception` function does not install it as the handler for this memory-fault route.

The system exception entry calls the BIOS exception function, which enters the BIOS monitor. Acknowledging the Memory fault record is not exception return: it clears the retained report but does not repair an address or advance the faulting instruction. Likewise, returning from an exception does not itself acknowledge that record.

The BIOS captures the CPU diagnostic words through SYSTEM's FAULT command. It prepares the monitor display and waits another VBlank before issuing FAULT_PUBLISH. The resulting supervisor-fault sequence change therefore marks publication, later than CPU exception entry. An NMI entry alone does not refresh the fault report.

7.2.1 Inspect an actual unmapped load

This cartridge fragment deliberately accesses unmapped storage. Its Word view addresses `0x01000000`, the first byte of the unmapped gap after the system-ROM window. Startup initially clears `reached_after_load`; the attempted load faults before its result can be assigned there.

```
module<entry>

bss reached_after_load: word
local unmapped<const>: *word = 0x01000000

-- This unmapped load enters the BIOS monitor before the assignment completes.
*reached_after_load = *unmapped

while true do
    halt_until_irq
end
```

This load produces a data bus error, displayed as **DBE**. The CPU cause field is `0x0000001c` (code 7); the Memory fault CODE is instead `0x00000001` (unmapped). These are different reports, not two encodings to write to the same register. DBE does not establish a new CP0 bad-address value. Inspect the Memory ADDR word for this bus fault's address.

At the BIOS monitor prompt, type the following command and press Enter. Its address is hexadecimal, and the count selects three consecutive 32-bit words:

```
MEM 08010208 3
```

The displayed result is:

```
08010208: 00000001 01000000 00000801
```

Displayed word	Meaning for this load
`00000001`	Memory CODE: unmapped access
`01000000`	Memory ADDR: the invalid byte address
`00000801`	Memory ACCESS: read direction and Word access kind

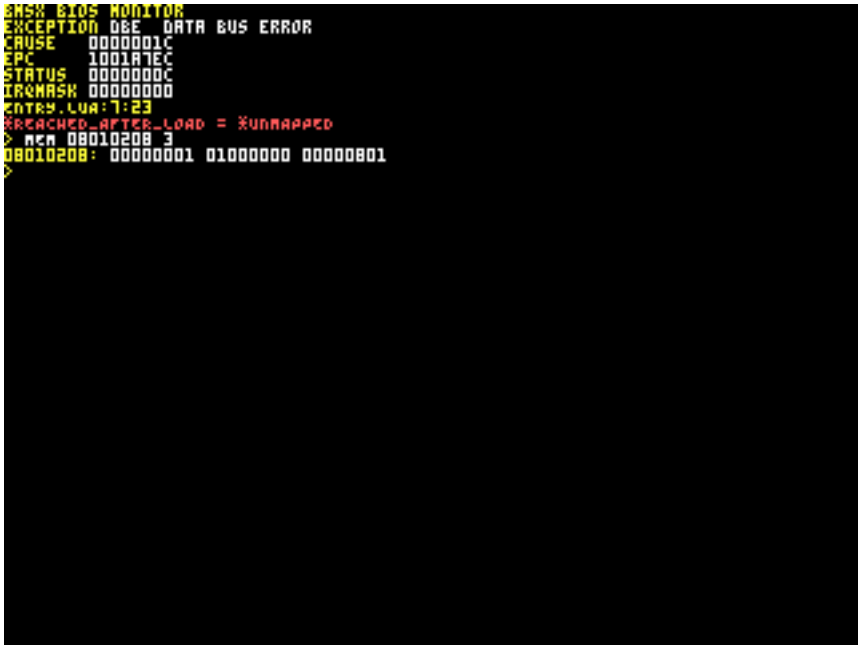


Figure 7.2: BIOS monitor after the unmapped Word load and the three-word MEM command

Unmapped-load output. The BIOS reports DBE, the faulting source statement and its EPC; the `MEM` line shows the three Memory report words. EPC is the instruction's address in this program, not a constant to copy into another cartridge.

7.2.2 Clearing a report is not repairing the program

The monitor's `FAULT` command displays its saved fault report when one is retained. `FAULT CLEAR` discards that saved BIOS report; it does not write `IO_SYS_BUS_FAULT_ACK`, clear the Memory CODE/ADDR/ACCESS words, or resume the cartridge.

`CONT` restores the saved EPC and CPU status. When the monitor owns the supervisor device context, it also issues LEAVE before returning. CPU exception return and device-context restoration are separate operations; neither advances the saved EPC. In the program above, the invalid address is unchanged, so `CONT` retries the load and enters the monitor again with another DBE. The first Memory fault remains latched. Clearing either report does not turn an unmapped address into usable storage.

CP0 reads and writes require non-user mode; attempting them from ordinary cartridge user mode causes the coprocessor-unusable exception. Do not attempt to repair this example by reading or replacing CP0 fields in ordinary cartridge code. The example demonstrates inspection and an unchanged-instruction retry, not a cartridge recovery handler or a successful recovery.

7.2.3 Repair a word load and retry

The preceding constant-address probe remains faulty after its report is cleared. This example instead makes the address a mutable local pointer, so you can repair the operand of the stopped read. Include the BIOS diagnostic directory for source and frame-scope lookup. Frame evaluation needs the correct source scope and an available, mutable live binding; it cannot recover arbitrary optimized-out locals.

```
module<entry>

struct memory_recovery_result
    first: word
    second: word
    fault_before_ack: word
    fault_after_ack: word
    completed: word
end

data source_address: word = 0x01000000
data safe_words: word[2] = { 0x12345678, 0x9abcdef0 }
bss result: memory_recovery_result

local source: *word = *source_address
local fault_code<const>: *word = 0x08010208
local fault_ack<const>: *word = 0x08010214

result.first = *source
source = source + sizeof(word)
result.second = *source
result.fault_before_ack = *fault_code
*fault_ack = 1
result.fault_after_ack = *fault_code
result.completed = 1
print('RECOVERED')
print(string.format('WORDS %08X %08X', result.first, result.second))
print(string.format('FAULT %08X -> %08X', result.fault_before_ack, result.fault_after_ack))

while true do
    halt_until_irq
end
```

`source` first receives the invalid address stored in `source_address`. The first read faults before assigning `result.first`. `safe_words`, by contrast, names the address of two DATA words in main RAM. Repair the existing `source` binding to that address: changing the `source_address` word alone would leave the stopped read's already loaded pointer unchanged. Do not change EPC; the same read must be retried with the repaired operand.

At the monitor, inspect `FRAMES`. Each physical frame row gives its index, function-record address and PC. Following `DEPTH` rows list the installed source-scope names active at that frame's suspension PC, including inlined functions. Choose the physical index from the preceding frame row and the depth from the matching source-scope row. Missing diagnostic information is reported instead of supplying a guessed scope. A binding must also be live and mutable at that PC to be assigned.

In the supplied sample, the source frame is physical index `00000001`, depth `00000000`, and its PC matches the fault EPC. Both selectors are displayed as eight hexadecimal digits without a prefix; bare command selectors are decimal, so use `0x` when copying them. Other programs or builds need not use this pair. For the supplied sample, the sequence is:

```
FRAMES
LUA --FRAME 0x00000001 0x00000000 source = safe_words
CONT
```

The single `LUA` assignment changes the selected frame's live pointer; it does not redeclare a typed local or change the `safe_words` address binding. It returns to the prompt without resuming execution or acknowledging Memory's report. A success banner is not required.



Figure 7.3: Monitor after inspecting frames and assigning the repaired pointer, before continuing.

Repair before retry. The monitor after `FRAMES` and `source = safe_words`, still at the prompt before `CONT`; this is not a completed-recovery screen.

`CONT` retries the saved instruction. The first read now succeeds; the program advances `source` by `sizeof(word)` (four bytes) and reads the second word. The old Memory CODE can remain `1` throughout both successful reads: a pending report does not make a valid address invalid. Only afterward does the program save that old code, write `1` to ACK, and read the cleared code.

The successful run produces these diagnostic lines, not newly drawn text in the monitor image:

```
RECOVERED
WORDS 12345678 9ABCDEF0
FAULT 00000001 -> 00000000
```

`REBOOT` is a restart request, not this repair or a saved-EPC continuation. With the cartridge unchanged, its bad initial pointer is encountered again. See Book I, “System control and timing”, under “Reset requests”.

For a programming example of privileged CP0 reads inside an ordinary cartridge IRQ callback, see CPU — Read CP0 in a cartridge IRQ. The callback returns normally, allowing the standard IRQ entry to perform RFE and resume the interrupted cartridge code.

7.3 Optional BIOS diagnostic directory

The ROM word at byte offset `0x3c` supplies a **ROM-relative byte offset** to an optional diagnostic directory. Zero means absent. This word extends beyond the CPU’s 60-byte boot prefix: it is consumed by BIOS source reporting and selected-frame evaluation, not by instruction execution or image activation. Without the directory, executable code and ordinary boot remain usable, but those BIOS services cannot recover its source locations or named frame scopes.

All directory words are unsigned 32-bit little-endian. The header occupies **76 bytes**. Table offsets below are relative to the directory base; a count is a number of records, not a byte count.

Header byte offset	Field
`0x00`, `0x04`	Source-range count; range-table offset
`0x08`, `0x0c`	File count; file-table offset
`0x10`, `0x14`	Line-offset count; line-offset-table offset
`0x18`, `0x1c`	Path-byte-table offset; path byte count
`0x20`, `0x24`	Scope-function count; function-table offset

Header byte offset	Field
`0x28`, `0x2c`	Scope-frame count; frame-table offset
`0x30`, `0x34`	Binding count; binding-table offset
`0x38`, `0x3c`	Interval count; interval-table offset
`0x40`, `0x44`	Scope-name byte count; name-byte-table offset
`0x48`	Static-global binding count; these occupy the first binding records

7.3.1 Source locations

Record	Bytes	Four-byte fields, in increasing byte-offset order
Range	12	Absolute PC start; packed line/file; column
File	20	Directory-relative display-path offset; path byte count; ROM-relative source offset; source byte count; first line-offset index
Line offset	4	UTF-8 byte offset from the file's source start

Range records are sorted by PC. Lookup selects the last record whose start is not greater than the PC. The packed location holds the one-based line in bits 31:16 and the zero-based file index in bits 15:0; `FFFFFFFFH` marks no source. Column is one-based. Include a no-source record at PC zero and another at the text end so addresses outside mapped source do not inherit its final location.

Each file's line-offset sequence starts at zero and includes an end-of-source sentinel. For line *L*, BIOS reads offsets at `first + L - 1` and `first + L`; it strips terminal CR/LF bytes from the displayed line. Paths and source spans use explicit byte counts, not NUL termination. Source bytes may occupy their ordinary ROM location; they need not be duplicated inside the directory.

7.3.2 Named frame scopes

Scope functions are sorted by their absolute function-record address. BIOS matches that address, computes instruction-word position `(PC - codeAddress) >> 2`, and selects a frame at the requested inline depth whose active interval contains that word. These are source scopes for BIOS evaluation, not additional CPU call frames or instruction-fetch bounds.

Record	Bytes	Four-byte fields, in increasing byte-offset order
Scope function	16	Absolute function-record address; absolute code address; first frame index; frame count
Scope frame	28	Inline depth; directory-relative name offset; name byte count; first active-interval index; active-interval count; first binding index; binding count
Binding	32	Directory-relative name offset; name byte count; flags; location; first visible-interval index; visible-interval count; first live-interval index; live-interval count
Interval	8	First instruction-word position; exclusive end position

Interval lists are sorted nonoverlapping half-open ranges in instruction-word positions relative to the function’s code address. Names use UTF-8 byte spans. A frame’s visible bindings overlay the initial static-global bindings; later same-named bindings shadow earlier ones even when their value is unavailable. Visibility and availability are separate: the visible interval selects the name, while the live interval determines whether its stored value is available.

Binding flags are bit 0 UPVALUE, bit 1 CONST, bit 2 ADDRESS and bit 3 TYPE. Without ADDRESS or TYPE, location identifies a register or, with UPVALUE, an upvalue slot. ADDRESS selects a raw static-storage address and remains available without a live interval. TYPE describes a type rather than a readable value. `FFFFFFFFH` is the no-location word. CONST marks a binding that evaluation must not reassign; it does not freeze a Table’s fields.

The BIOS locates this directory directly through the ROM-header word described above.

8 BIOS math library

The BIOS math library supplies numeric operations, angular functions and a shared pseudorandom generator. Its `math` Table contains twenty-five callable entries and four numeric constants. Numbers use binary64 arithmetic; a function preserves the input scale unless its definition states a unit or representation conversion.

8.1 Calling the library

The standard BIOS environment supplies `math` as a Table of functions and constants. Call its functions directly. The separate named import `math/sin` uses turn-phase words and integer Q16 results; the ordinary trigonometric functions use radians and unscaled Numbers.

Numbers use binary64 arithmetic. Descriptions such as “integer component” and the result `"integer"` classify a Number’s value; they do not select a second numeric type. A function preserves the input scale unless its entry specifies a unit conversion. Register-word conversion follows the CPU rules.

Missing fixed parameters arrive as nil and surplus arguments are discarded. `min` and `max` receive a variable argument tail, including its nil positions. The following entries state the returned count before caller result adjustment.

8.2 Numeric notation

The formulas in this chapter use these explanatory operations:

- **`D(a, b)`**: BLua `a // b`. Form the binary64 quotient `q = a / b`, truncate it toward zero as `t`, then return `t - 1` when `t > q`, otherwise `t`. Floating division and rounding precede selection of the result.
- **`T(x)`**: truncate toward zero, using `-D(-x, 1)` for negative `x` and `D(x, 1)` otherwise.
- **`M(a, b)`**: BLua `a % b`. Form the truncation-based floating remainder `r`; when `r` is positive and `b` negative, or `r` negative and `b` positive, return `r + b`. Otherwise return `r`, including its zero sign.

Arithmetic on a non-Number yields NaN. Numeric Strings require an explicit `tonumber` conversion. Number/Number and String/String pairs support ordering; other kind pairs compare false. NaN is unordered and the two numeric zero signs compare equal. See numeric instructions.

These rules apply to the written formulas, including overflow, division by zero and rounding of intermediate values. In particular, `fmod` uses its quotient formula while `ult` compares modulo results.

8.3 Constants

Field	Numeric value	Meaning and limits
`math.huge`	Positive infinity, initialized by `1.0 / 0.0`	A numeric infinity, not a maximum finite number or an argument validator.
`math.mininteger`	`-9007199254740991`	The exported lower integer constant. It does not bound all numbers or all values accepted by `tointeger`.
`math.maxinteger`	`9007199254740991`	The exported upper integer constant. The pair is symmetric, not the asymmetric bounds of a signed 64-bit integer.
`math.pi`	Literal `3.14159265358979323846264338327950288`4, stored as a number	The literal spelling does not imply that every displayed decimal digit survives in binary64 precision.

These are value fields, not callable functions. They are not fixed-point constants, register words, or precision metadata. There is no exported NaN constant among the fields covered here, although several operations can produce numeric NaN.

8.4 `math.abs(value)`

Returns one value: `-value` when `value < 0`, otherwise the original value. For Numbers this gives the magnitude, preserving either zero sign. Negative infinity becomes positive infinity; NaN and non-Number values are returned unchanged. A missing argument therefore returns nil.

8.5 `math.floor(value)`

Returns one Number, `D(value, 1)`. For finite values it selects the lower whole value at unit spacing in the supplied scale. Infinities and signed zero retain their signs; NaN and non-Number input produce NaN.

8.6 `math.ceil(value)`

Returns one Number, `-D(-value, 1)`. For finite values it selects the upper whole value at unit spacing. Its infinity, signed-zero and non-Number cases are the same as `floor`.

8.7 `math.round(value)`

Returns one Number. For `value >= 0`, the result is `D(value + 0.5, 1)`; otherwise it is `-D((-value) + 0.5, 1)`. Exact halfway values within ordinary finite precision round away from zero. At large magnitudes, the addition of `0.5` is itself subject to binary64 rounding.

Both zero signs produce positive zero. A negative value whose rounded magnitude becomes zero produces negative zero. Infinities retain their signs; NaN and non-Number inputs produce NaN.

8.8 `math.fmod(left, right)`

Returns one Number, calculated in this order:

```
left - (T(left / right) * right)
```

The result uses the input scale. Division can overflow before truncation, including for finite operands. A zero divisor, an infinity operand, NaN or a non-Number operand produces NaN. A zero numerator and finite nonzero divisor produce positive zero, including for a negative-zero numerator.

8.9 `math.modf(value)`

Returns two Numbers: the integer component `T(value)`, then `value - T(value)`. The first result truncates toward zero; the residual retains its signed contribution rather than being made positive.

For either zero sign, the first result keeps that sign and the residual is positive zero. An infinity returns that infinity followed by NaN. NaN or non-Number input returns two NaNs. Both components use the original scale.

8.10 math.min(first, ...)

Returns one selected argument, unchanged. Selection starts with `first` and visits each subsequent argument in order, replacing the selection only when `candidate < selected`. Ties retain the earlier value, including its zero sign. Nil as the first argument raises:

```
min expects at least one argument.
```

Number candidates compare numerically; String candidates compare as Strings. A candidate of an incomparable kind leaves the selection unchanged. An initial NaN remains selected, and a later NaN cannot replace a numeric selection. Explicit nil tail arguments count as arguments but do not replace a nonnil selection. A nonnil Boolean, Table, function or Thread initially selected remains selected.

8.11 math.max(first, ...)

Returns one selected argument, using the same scan and tie rules as `min`, but replacing the selection only when `candidate > selected`. Nil as the first argument raises:

```
max expects at least one argument.
```

The mixed-kind, NaN and nil-tail behavior is the same as for `min`.

8.12 math.tointeger(value)

Returns one Number when `value` is a finite Number equal to `T(value)`; otherwise returns one nil. Thus finite whole values are accepted, fractions are rejected, and either zero sign is preserved. Accepted whole values may lie outside the exported `mininteger` and `maxinteger` constants. This call classifies integrality; use explicit word conversion for a register encoding.

8.13 math.type(value)

Returns `"integer"` for a Number accepted by `tointeger`, `"float"` for other Numbers, and nil for other kinds. Infinities and NaN classify as `"float"`. Either zero sign classifies as `"integer"`. These labels describe values in the single Number representation.

8.14 math.ult(left, right)

Returns one Boolean:

```
M(left, 4294967296.0) < M(right, 4294967296.0)
```

For integer inputs this compares their modulo-2^32 residues. Fractional inputs participate directly in modulo without truncation. Equal zero residues compare false regardless of their signs. Infinity, NaN or non-Number input produces an unordered residue and therefore false.

8.15 math.sign(value)

Returns one Number: `1` for `value > 0`, `-1` for `value < 0`, otherwise `0`. This includes the corresponding signs for infinities. Either zero sign, NaN and non-Number input produce positive zero. The result is dimensionless.

8.16 Execution cost and library bindings

Elementary operations execute their stated arithmetic and calls. `min` and `max` make one comparison per tail argument and repeatedly forward the tail through `select`; selecting one final result leaves those preceding calls in place. For `n` ordinary positive selector positions, the selected suffixes contain `n + (n - 1) + ... + 1` values in total.

The primitive surcharges are one cycle for `type` or `select`, and two for `error`, in addition to executed BIOS instructions and ordinary calls. Use CPU cycle accounting when budgeting a complete operation.

Reassigning a Math field leaves the library's saved numeric operations and constants unchanged. `math.type` retains its integrality operation; `math.type` and `tointeger` read the current global `type`. `min` and `max` read current `select` and `error`. Their entries describe the installed base functions.

8.17 Angles and transcendental operations

Use `math.rad` and `math.deg` to change angular units, `math.sin`, `math.cos` and `math.tan` for periodic quantities, and the inverse functions to recover angles. `math.sqrt`, `math.exp` and `math.log` provide roots, exponentials and logarithms. All eleven interfaces return exactly one Number.

Interface	Meaning
`math.deg(value)`	Radian-scale value to degrees.
`math.rad(value)`	Degree-scale value to radians.
`math.sin(radians)`	Approximate sine.
`math.cos(radians)`	Approximate cosine.
`math.tan(radians)`	Approximate tangent.
`math.atan(y, x)`	Approximate angle in radians; `x == nil` selects the one-value form.
`math.asin(value)`	Approximate inverse sine in radians.
`math.acos(value)`	Approximate inverse cosine in radians.
`math.sqrt(value)`	Approximate square root.
`math.exp(value)`	Approximate natural exponential.
`math.log(value, base)`	Approximate logarithm; `base == nil` selects the natural logarithm.

These are fixed-parameter interfaces, not varargs. Missing fixed parameters have value `nil`; surplus parameters are not retained. The signatures show all fixed parameters. An explicit `nil` selects the alternate forms `math.atan(y, nil)` and `math.log(value, nil)`; omitting the final parameter also supplies `nil`. No other parameter has a default.

The formulas below use the Number arithmetic and `D` and `T` notation defined earlier. For word conversion, bit masks and signed right shifts, see Bitwise operations and shifts. The formulas specify the approximations and their operation order. Rounding follows Number arithmetic at each step. Nonnumeric arguments follow the same arithmetic rules and produce the results stated below; numeric text requires explicit conversion.

Here `pi` is the BIOS Number constant and `h = pi * 0.5`. The calculations use their original constants and numeric operations after public Math fields are reassigned. Named intermediate approximations below are formula notation, not additional Math entries.

The text blocks in this reference section are explanatory formulas and pseudocode, not BLua source listings. The programming examples below use BLua source.

8.17.1 Converting angular units

The conversion functions perform one multiplication:

```
math.deg(value): value * (180 / pi)
math.rad(value): value * (pi / 180)
```

There is no magnitude reduction or fixed-point conversion. Either zero sign is preserved, infinities retain their sign, and NaN or a nonnumeric argument produces NaN.

8.17.2 Sine, cosine and tangent

These functions accept radian-scale Numbers and return dimensionless binary64 approximations. They do **not** convert their arguments to 32-bit phases or quantize their results to Q16. Small angles therefore retain their floating-point scale: `math.sin(0.000001)` is nonzero.

The BIOS reduces an argument to a nearby multiple of pi/2 and evaluates floating polynomials on the remaining angle. The remainder has separate leading and tail components to retain cancellation information. Large finite angles use a multiword 2/pi reduction covering the binary64 exponent range, rather than a remainder against a single rounded `2*pi` constant.

Input	`math.sin`	`math.cos`	`math.tan`
`+0`	`+0`	`1`	`+0`
`-0`	`-0`	`1`	`-0`
NaN or either infinity	NaN	NaN	NaN

The result is a binary64 approximation. For example, `math.pi / 2` is a rounded Number rather than an exact tangent pole, so its tangent can be large and finite. Argument reduction and cancellation affect the executed path and cycle count.

8.17.3 Turn-phase sine

The named BIOS import `math/sin` takes one **32-bit turn-phase word** and returns one **signed integer Q16 sine**. Use it when an angle is already stored as a wrapping phase word, rather than converting that phase to radians:

```
local sin_phase<const> = require('math/sin')
local peak = sin_phase(0x40000000)     -- 65536, representing +1
local opposite = sin_phase(0xc0000000) -- -65536, representing -1
```

One complete turn spans `2^32` phase units. Bit 31 selects the negative half of the sine wave; the low 30 bits and quadrant select a position within its quarter-wave table. Ordinary word conversion applies to the bitwise phase selection. The returned Number is an integer, not a floating sine in the interval −1 through +1 and not an unsigned encoded negative word.

Phase word	Angle	Returned Number
`00000000H`	0 turns	0
`20000000H`	1/8 turn	46340
`40000000H`	1/4 turn	65536
`80000000H`	1/2 turn	0
`C0000000H`	3/4 turn	−65536

The quarter-wave table has 257 samples, giving 256 equal phase intervals. Within an interval, the low 22 phase bits interpolate between its two integer samples; the interpolated positive increment is shifted right by 22 before the quadrant sign is applied. Results therefore retain integer Q16 quantization. The radian functions `math.sin`, `math.cos` and `math.tan` do not use this interface.

8.17.4 Inverse angles

The private approximation used by `math.atan` is:

```
P(v) = (pi * 0.25 * v) + (0.273 * v * (1 - abs(v)))
```

Here `abs` is the retained private absolute-value operation. Define `A(v)` by these ordered cases:

Condition	Result
`v > 1`	`h - P(1 / v)`
`v < -1`	`-h - P(1 / v)`

Condition	Result
Otherwise	`P(v)`

`math.atan(y, x)` selects its result using the following order. The `x < 0` rows are considered before either remaining `y`-axis row.

Condition	Result
`x == nil`	`A(y)`
`y ~= y` or `x ~= x`	NaN
`x > 0`	`A(y / x)`
`x < 0` and `y >= 0`	`A(y / x) + pi`
`x < 0`, otherwise	`A(y / x) - pi`
`y > 0`	`h`
`y < 0`	`-h`
Otherwise	Literal positive zero.

The result uses radian scale. A supplied `false` is not the nil form: `math.atan(1, false)` reaches the axis branch and returns `h`. A nonnumeric `y` in the nil form produces NaN through `P`. Numeric NaN in the two-value form is handled by the self-inequality checks. With numerical zero axes, the last row returns positive zero regardless of either zero sign. These polynomial and branch rules also determine results at infinite arguments; the approximation is not a correctly rounded `atan2` operation.

The other inverse functions use retained private operations:

```
math.asin(value): atan(value, sqrt(1 - (value * value)))
math.acos(value): h - asin(value)
```

Their results are in radians. Numerical inputs outside `[-1, 1]` produce NaN through the square root and angle calculation, rather than an explicit domain error. Missing or nonnumeric inputs likewise reach NaN through the arithmetic. The inverse-sine calculation includes the square root's normalization and refinements; inverse-angle work is therefore not uniformly fixed. No uniform inverse-trigonometric accuracy or result-range guarantee is specified here.

8.17.5 Square root

`math.sqrt(value)` first returns NaN for `value < 0`, then returns `value` unchanged if it equals zero or positive infinity. Consequently negative infinity produces NaN, positive infinity is preserved, and either zero sign is preserved.

For other inputs, initialize `normalized = value` and `scale = 1`, then perform these loops in order:

```
while normalized >= 4: normalized *= 0.25; scale *= 2
while normalized < 1:  normalized *= 4;    scale *= 0.5
```

Set `estimate = normalized`, perform exactly eight refinements,

```
estimate = (estimate + normalized / estimate) * 0.5
```

and return `estimate * scale`. Positive finite values are normalized using powers of four and a power-of-two scale. The normalization work depends on magnitude. NaN and nonnumeric inputs skip both loops and produce NaN during refinement arithmetic. The eight refinements and the displayed scaling specify the approximation; the executed normalization path determines its variable part of the cycle count.

8.17.6 Natural exponential

`math.exp(value)` approximates the natural exponential. Its initial tests are ordered:

Condition	Result
`value ~= value`	NaN
`value > 709.782712893384`	Positive infinity.
`value < -745.133219101941`	Literal zero.

The two numerical cutoffs are **strict**: equality proceeds to the approximation. Both infinities take their corresponding cutoff branch. After these tests, a negative value returns the reciprocal of the positive approximation at `-value`; other values use the positive approximation directly.

The retained constants are:

```
ln2 = 0.69314718055994530942
inv_ln2 = 1.44269504088896340736
```

For the positive approximation at `v`, reduce the argument as follows:

```
power = D(v * inv_ln2 + 0.5, 1)
reduced = v - power * ln2
```

Start `term = 1` and `sum = 1`. For each integer divisor from 1 through 16, in order, perform:

```
term = (term * reduced) / divisor
sum = sum + term
```

Set `result = sum`, then scale in this order:

```
while power >= 16: result *= 65536; power -= 16
while power > 0:   result *= 2;     power -= 1
```

The resulting Number is the positive approximation. There are exactly 16 series updates, but the scaling work varies. Nonnumeric input passes the initial comparisons, produces NaN in reduction and the series, and skips the scaling loops.

For negative inputs, the positive intermediate can overflow **before** the reciprocal is taken. The reciprocal then becomes zero even when the argument has not crossed the explicit underflow cutoff. Applications requiring very small exponential results must account for that limitation.

8.17.7 Logarithms

Use `math.log(value, nil)` for the natural logarithm, or supply a base. The natural logarithm approximation `N(v)` first tests, in order:

Condition	Result
`v < 0`	NaN
`v == 0`	Negative infinity.
`v == positive infinity`	Positive infinity.

Thus negative infinity produces NaN and either zero sign produces negative infinity. Otherwise initialize `exponent = 0` and `mantissa = v`. The retained normalization constants are:

```
sqrt_two = 1.41421356237309504880
sqrt_half = 0.70710678118654752440
```

Perform these loops in order:

```
while mantissa >= sqrt_two: mantissa *= 0.5; exponent += 1
while mantissa < sqrt_half: mantissa *= 2;   exponent -= 1
```

Then calculate:

```
z = (mantissa - 1) / (mantissa + 1)
z2 = z * z
term = z
sum = term
```

For denominators 3, 5, 7, …, 31, in order, perform 15 updates:

```
term = term * z2
sum = sum + term / denominator
```

The natural result is `(2 * sum) + (exponent * ln2)`, using the same retained `ln2` as the exponential. NaN and nonnumeric inputs skip normalization and produce NaN through the arithmetic.

`math.log(value, base)` calculates `natural = N(value)` **first**, before selecting the base branch:

Condition	Result
`base == nil`	`natural`
`base == 10`	`natural / ln10`
`base == 2`	`natural / ln2`
Otherwise	`natural / N(base)`

Here `ln10 = 2.30258509299404568402` is another retained Number constant. The general-base branch applies the displayed division to every supplied base. Since `N(1)` is zero, base one causes division by zero and `math.log(1, 1)` produces NaN. A supplied false or another nonnumeric base produces NaN through `N(base)`; only nil selects the natural-log form.

The divisions can produce infinities, signed zeros or NaN. Natural normalization depends on input magnitude, followed by 15 series updates; the general-base branch may perform normalization again for the base. There cycle count depends on the selected calculation.

All decimal constants above are rounded to their stored Number values.

8.18 Random numbers

8.18.1 `math.random([lower [, upper]])` → Number

Returns exactly one Number on success. The function has two fixed parameters and no variable-argument parameter. An omitted fixed parameter arrives as nil. False is not nil and does not request an omitted-argument default. See calls, closures, globals, and tables for the call model.

Both bound-rejection branches call the current public `error` binding. The raised String behavior below assumes its installed BIOS definition.

Every call advances the shared generator **before inspecting either bound**. Consequently, an explicit bound error consumes an update, as does a call whose result is NaN.

The branches are selected in this order:

1. If `lower` is nil, return the unit sample immediately. `upper` is not examined. Thus `random()` and `random(nil, upper)` select this branch, including `random(nil, false)`.

2. Otherwise, if `upper` is nil, truncate `lower` toward zero to obtain `upper_int`. If `upper_int < 1`, raise exactly:

   ```
   random upper bound must be positive.
   ```

   Otherwise return:

   ```
   trunc(value * upper_int) + 1
   ```

3. Otherwise, truncate `lower` first and `upper` second to obtain `lower_int` and `upper_int`. If `upper_int < lower_int`, raise exactly:

   ```
   random upper bound must be greater than or equal to lower bound.
   ```

   Otherwise return:

   ```
   lower_int + trunc(value * ((upper_int - lower_int) + 1))
   ```

Here `value` is the unit sample already generated. In these formulas, `trunc` denotes the private truncation operation, not an additional public function. For finite Numbers it truncates toward zero.

The unit sample lies in **[0, 1)**: zero is included and one is excluded. Its largest possible value is `1 - 2^-32`. For ordinary finite bounded inputs, the source-defined closed integer intervals are **[1, upper_int]** for one bound and **[lower_int, upper_int]** for two bounds. Both endpoints of those interval definitions are included. The formulas above remain authoritative: floating arithmetic and rounding do not support extending this description into a guarantee for every extreme Number, or a promise that every integer is sampled or equally likely.

`random(u)` and `random(u, nil)` use the one-bound branch. Fractional finite bounds are truncated, not rounded to the nearest integer. A one-bound zero, negative integer, or finite value that truncates below one takes the positive-bound error.

Supply finite Number bounds for a bounded integer result. Nonnumeric or NaN bounds propagate NaN through the result formula after consuming a generator update. With a positive-infinity upper bound, the result is positive infinity for a positive unit sample and NaN for zero. A negative-infinity upper bound raises the positive-bound error. These cases follow ordinary Number arithmetic.

8.18.2 `math.randomseed([seed])` → nil

Returns **exactly one nil result**, not an empty result list. It has one fixed parameter and no variable-argument parameter.

If `seed` is nil, including an omitted parameter, the function calls the current `os.clock()` binding, multiplies that result by 1000, applies the private truncation operation, reduces it modulo `4294967296.0`, and stores it in the shared generator word. With the BIOS-defined `os.clock`, the source expression is:

```
trunc(((*time_ms / 1000) * 1000)) % 4294967296.0
```

Here `time_ms` denotes the unsigned millisecond word sampled by the BIOS clock function. The divide-then-multiply expression is significant; it is not replaced here by a direct word value. The seed therefore depends on the sampled system counter, rather than a civil clock or an entropy source. See Book I, “System control and timing”.

Otherwise, the function stores:

```
trunc(seed) % 4294967296.0
```

An explicit seed does not use the clock. Finite seeds are truncated toward zero, reduced using positive-modulus semantics, and stored as an unsigned word. Both `-1` and `-1.9` therefore store `0xffffffff`; zero stores zero. False selects this supplied-seed branch, not the clock branch. Nonnumeric seeds, NaN, and either infinity reach a NaN modulus result and store word zero. There is no local explicit seed error.

Seeding performs no generation step or warm-up loop and returns no seed value. After seeding zero, the next unit output is exactly `1013904223 / 4294967296`.

8.18.3 Generator state and update

The public bindings are `math.random` and `math.randomseed`. They use the same private four-byte word, whose declared initializer is `0x12345678`. Neither that word nor the private unit generator is returned by the Math Table.

Each generation step reads the old word and computes:

```
((old * 1664525) + 1013904223) % 4294967296.0
```

It stores the result in the word, reads the stored unsigned word back, and divides it by `4294967296.0` to obtain the unit sample. Seeding replaces that same word. All callers advance and seed this same stream. Its declared initializer is applied during normal BIOS DATA initialization.

8.19 Programming examples

8.19.1 Convert an angle and form an offset

The angular functions take radians. To place a point on a circle with radius 64 position units, convert the authored degree angle once and scale its sine and cosine:

```
local angle<const> = math.rad(30)
local dx<const> = 64 * math.cos(angle)
local dy<const> = 64 * math.sin(angle)
local pixel_x<const> = math.round(dx)
local pixel_y<const> = math.round(dy)
```

The offsets are approximately `(55.4256,32)` position units; rounding selects `(55,32)`. Floating results need not equal ideal decimal fractions exactly. Choose rounding or truncation deliberately when converting a calculated position to whole pixels. These functions do not produce GTE coefficients or GPU packet fields; use the destination unit's documented representation.

8.19.2 Repeat a pseudorandom sequence

Seed the shared generator when starting a sequence. Reusing the seed repeats its subsequent updates:

```
math.randomseed(1)
local first<const> = math.random()
local die<const> = math.random(6)
math.randomseed(1)
local replay<const> = math.random()
```

`first` and `replay` are equal, approximately `0.236456`; `die` is `3`. All users of the library share this state, so an intervening call advances the same sequence. A rejected bound also consumes an update, as specified above.

9 BIOS time and calendar services

The BIOS provides sampled elapsed time and civil-date conversion through the `os` Table. Use `clock` or the no-argument `time` form to sample system time. Use explicit timestamps with `time(date)` and `date(format, timestamp)` for stored calendar dates. `difftime` subtracts whole-second values.

The sampled counter is the system millisecond word, not a calendar clock. Civil conversion uses its own 1970 epoch and fixed BMSX zone convention. The Book I, “System control and timing” specifies the underlying counter; Number arithmetic determines precision in date calculations. These services provide no filesystem or process operations.

Call	Returned value	Operation
`os.clock()`	One Number, seconds	Sample the millisecond word and divide by 1000.
`os.time(date_table)`	One Number, whole seconds	With nil, sample time; otherwise normalize the supplied civil fields in place and return their timestamp.
`os.date(format, timestamp)`	One String or new Table	Format or decompose the selected timestamp.
`os.difftime(t2, t1)`	One Number, seconds	Check each argument against its integer component, then subtract.

Use sampled time for intervals within one counter range. Use an explicit civil timestamp for stored dates. Passing a sampled `os.time()` result to `os.date()` applies the civil epoch to that number; it does not obtain the current calendar date from a clock or preserve elapsed time across counter wrap. The two uses share seconds as a unit, not a common time origin.

9.1 Example: time and calendar operations

Use an explicit timestamp for a stored calendar date. `os.time` normalizes the caller-owned Table, while `os.date('*t', ...)` creates a separate Table:

```
local date = { year = 2024, month = 2, day = 28, hour = 25, min = 2, sec = 3 }
local timestamp = os.time(date)             -- 1709168523
-- date is now 2024-02-29 at 01:02:03; wday = 5, yday = 60, isdst = false.
local fields = os.date('*t', timestamp)
local label = os.date('%F %T', timestamp)   -- '2024-02-29 01:02:03'
local elapsed = os.difftime(timestamp, timestamp - 90) -- 90 seconds
```

Calendar-year and ISO-week-year fields can differ at a year boundary:

Call	Result
`os.date('%F %G-W%V-%u', 1609459200)`	`'2021-01-01 2020-W53-5'`
`os.date('%F %G-W%V-%u', 1546214400)`	`'2018-12-31 2019-W01-1'`
`os.date('%c', 0)`	`'Thu Jan 01 00:00:00 1970'`
`os.date('%F %T', -1)`	`'1969-12-31 23:59:59'`
`os.date('offset %z zone %Z %%', 0)`	`'offset +0000 zone BMSX %'`

9.2 `os.clock()` — sampled time in seconds

`os.clock()` returns one Number: the current sampled millisecond word divided by 1000. Use it when a seconds value with millisecond granularity is more useful than the whole-second result of `os.time()`.

The sampled word contains whole milliseconds modulo 2^{32}. `os.clock()` therefore ranges from 0 through 4294967.295 seconds and wraps to zero with that word. Decimal millisecond fractions are rounded to Number precision. The counter measures system time, including intervals when a particular program is not executing.

For example, two readings can be subtracted to measure a span within the same uninterrupted counter range:

```
local before = os.clock()
-- Perform the work being measured.
local after = os.clock()
local elapsed = after - before
```

Each call takes a new sample. If the word wraps between readings, direct subtraction crosses that discontinuity. Track wrap at the counter level when an interval must span it; `os.difftime` also performs ordinary subtraction.

9.3 `os.time([date_table])` — whole seconds or civil conversion

`os.time()` and `os.time(nil)` return one Number: the sampled millisecond word divided by 1000 with floor division. The result is a whole-second value in the range 0 through 4294967. Its wrap is still the wrap of the underlying 2^{32}-millisecond word, not a 2^{32}-second interval. This call takes its own sample; it does not reuse a preceding `os.clock()` reading.

Supply a date Table for civil conversion. The result is a whole-second timestamp, and the same Table receives normalized calendar fields. Nil selects sampling; every other supplied value selects the date-field route. Conversion leaves the sampled clock unchanged.

9.3.1 Civil timestamp convention

The civil-conversion formulas map timestamp 0 to the following fields:

Field	Value
`year`	1970
`month`	1
`day`	1
`hour`	0
`min`	0
`sec`	0
`wday`	5
`yday`	1
`isdst`	false

This origin belongs to civil-date conversion; it is independent of the running millisecond counter.

For example:

```
local date = { year = 1970, month = 1, day = 1, hour = 0 }
local timestamp = os.time(date)
```

Here `timestamp` is 0, and `date` receives all nine fields shown above. If `hour` had also been omitted, its default would have been 12, producing 43200 instead.

9.3.2 Input fields, defaults, and bounds

Fields are read and checked in this exact order: `year`, `month`, `day`, `hour`, `min`, `sec`. Each field's lookup and validation finish before the next field is read.

Field	Value when nil	Inclusive accepted integer range
`year`	Required	−2147481748 through 2147485547
`month`	Required	−2147483647 through 2147483648
`day`	Required	−2147483648 through 2147483647
`hour`	12	−2147483648 through 2147483647
`min`	0	−2147483648 through 2147483647
`sec`	0	−2147483648 through 2147483647

Only nil selects a missing-field error or an optional default. False is a supplied value, not a default. A nil optional field uses its default without undergoing the integer check.

A supplied field must equal its truncated integer component and pass the listed bounds. Fractions, NaN and non-Number values fail the integer test; infinities fail the bounds test. Supply Numbers, not numeric Strings.

Out-of-range calendar fields within those numeric bounds normalize arithmetically. For example, month 13 carries into the next year and hour 25 carries into the next day. Only the six listed fields contribute to the timestamp; `wday`, `yday` and `isdst` are recomputed outputs.

9.3.3 Normalization and result checking

Months first carry into the year. Days then participate arithmetically in the civil-date conversion; they are not checked against the selected month's length. Hours, minutes, and seconds contribute their corresponding numbers of seconds. Thus, outlying fields normalize through the combined timestamp rather than being rejected merely for crossing a calendar boundary.

For example:

```
local date = {
    year = 1970, month = 1, day = 1,
    hour = 25, min = 0, sec = 0
}
local timestamp = os.time(date)
```

This produces timestamp 90000 and normalizes the Table to 1970-01-02 at 01:00:00, together with its recomputed `wday`, `yday`, and `isdst` fields.

After the six fields pass, their combined timestamp must equal its integer component and lie in the inclusive range −9007199254740991 through 9007199254740991. Intermediate arithmetic uses Number precision, so the combined calculation can fail even when each individual field is accepted.

Negative timestamps use floor-day decomposition. For example, timestamp −1 belongs to the day before timestamp 0, at 23:59:59; the day is not obtained by truncating toward zero.

9.3.4 In-place Table update

After all six input validations and the combined timestamp check succeed, the function completes the inverse civil decomposition before making its first Table write. It then writes these fields in this order:

`year`, `month`, `day`, `hour`, `min`, `sec`, `wday`, `yday`, `isdst`.

`wday` uses a seven-day index. As a mathematical formula, with `days` denoting the floor-day index of the timestamp:

weekday index = ((days + 4) modulo 7) + 1.

`yday` is the one-based day within the resulting year. `isdst` is always written as false; no daylight-saving adjustment is applied.

The supplied Table is not replaced, and other fields are not cleared. Reads use ordinary field lookup, including Table-valued `__index` chains; function-valued `__index` entries are not called. The output stores go directly into the supplied Table and are not redirected through `__newindex`.

9.3.5 Errors and their order

The messages below describe calls with the normally installed public `error` function.

Missing required fields produce these exact messages:

- `field 'year' missing in date table`
- `field 'month' missing in date table`
- `field 'day' missing in date table`

For each supplied field, the integer check precedes the bounds check. Their exact message forms are:

- `field '<field>' is not an integer`
- `field '<field>' is out-of-bound`

Here `<field>` is replaced by `year`, `month`, `day`, `hour`, `min`, or `sec`, as appropriate. Because fields are processed in order, a failure in an earlier field is reached before a later field is read.

After the fields, the combined timestamp checks use these exact messages, in order:

- `time is not an integer`
- `time is out-of-bound`

Date fields use ordinary indexed reads and writes. Passing false selects conversion and fails its first field read with `Attempted to index field on a non-table value.` Other indexed-access errors retain their ordinary texts, including `Attempted to assign to a non-table value.` and `Metatable __index loop detected.`

9.3.6 Work involved

Sampling reads one time word and divides. Date conversion reads six input fields, calculates the timestamp and normalized date, then writes nine fields. Its calendar calculation uses arithmetic rather than advancing one day at a time. Ordinary calls, field accesses and arithmetic contribute their CPU instruction costs.

9.4 `os.difftime(t2, t1)` — subtract whole-second values

`os.difftime(t2, t1)` returns one Number, computed as the integer part of `t2` minus the integer part of `t1`, after checking that each argument equals its own integer part. For whole-second timestamps, the result is a signed difference in seconds: positive when `t2` is greater than `t1`.

```
local first = os.time({ year = 1970, month = 1, day = 1, hour = 0 })
local second = os.time({ year = 1970, month = 1, day = 2, hour = 0 })
local seconds = os.difftime(second, first)
-- seconds is 86400.
```

The function obtains the integer parts of `t2` and then `t1` before making either equality check. It then checks `t2` first and `t1` second. With the normally installed public `error`, the exact messages are:

- `os.difftime t2 must be an integer.`
- `os.difftime t1 must be an integer.`

Fractions, NaN, missing/nil arguments, false, and other non-Number values fail the corresponding equality check; numeric strings are not parsed. If both arguments fail, the `t2` message is reached first, although both integer-part calculations have already occurred.

The operands are checked for integer equality without the civil-field or timestamp bounds. Infinities therefore pass those checks; their subtraction can yield infinity or NaN. The result uses ordinary Number precision.

`difftime` reads no clock and applies no calendar conversion. For samples straddling the millisecond-counter wrap, account for the discontinuity before using ordinary whole-second subtraction.

9.4.1 Retained dependencies and call arguments

Integer checks use the truncation function saved at BIOS installation; replacing public `math.modf` leaves those checks unchanged. Reported errors call the current global `error`.

Missing fixed parameters are nil and surplus arguments are ignored. Thus an omitted date Table selects sampling, while missing `difftime` operands fail their integer checks. BLua source-call diagnostics also apply when the callee signature is known at compilation.

9.5 Shared civil-day calculations

The formulas below define `D(year, month, day)`, the civil-day index, and `C(days)`, its inverse. They use Number arithmetic in the written order, including `//` and `%`. These are reference formulas, not additional BIOS functions or BLua program listings.

Forward calculation: `D(year, month, day)`

```
if month <= 2: year = year - 1
era = year // 400
year_of_era = year - era * 400
month_prime = month + (if month > 2 then -3 else 9)
day_of_year = (153 * month_prime + 2) // 5
day_of_year = day_of_year + day - 1
day_of_era = year_of_era * 365 + year_of_era // 4
day_of_era = day_of_era - year_of_era // 100
day_of_era = day_of_era + day_of_year
result = era * 146097 + day_of_era - 719468
```

Inverse calculation: `C(days)`

```
shifted = days + 719468
era = shifted // 146097
day_of_era = shifted - era * 146097
year_numerator = day_of_era - day_of_era // 1460
year_numerator = year_numerator + day_of_era // 36524
year_numerator = year_numerator - day_of_era // 146096
year_of_era = year_numerator // 365
year = year_of_era + era * 400
year_days = 365 * year_of_era + year_of_era // 4
year_days = year_days - year_of_era // 100
day_of_year = day_of_era - year_days
month_prime = (5 * day_of_year + 2) // 153
day = day_of_year - (153 * month_prime + 2) // 5 + 1
month = month_prime + (if month_prime < 10 then 3 else -9)
if month <= 2: year = year + 1
result = (year, month, day)
```

Month normalization for `os.time`

Before applying `D`, `os.time` calculates the normalized year and month as follows. The timestamp expression is split into successive additions to retain its calculation order.

```
month_index = month - 1
year_offset = month_index // 12
normalized_year = year + year_offset
normalized_month = month_index - year_offset * 12 + 1
timestamp = D(normalized_year, normalized_month, day) * 86400
timestamp = timestamp + hour * 3600
timestamp = timestamp + min * 60
timestamp = timestamp + sec
```

The resulting timestamp then undergoes the ordinary time-value check. Timestamp decomposition and the date/week rules use the same `D` and `C` calculations.

9.6 `os.date`

```
os.date(format, timestamp)
```

`os.date` returns one value: either a formatted String or a newly created Table. Choose the Table form when the individual calendar fields are needed; choose a format string when textual output is needed. It defines no date-specific processing for additional arguments.

9.6.1 Selecting the result

Only `nil` selects a default. An omitted parameter is `nil` on an ordinary call.

Argument	Selection
`format == nil`	Use `"%c"`.
A format beginning with `!`	Remove exactly one leading `!` before selecting the result kind.
Format exactly `"*t"` after that removal	Return a fresh Table.
Any other accepted format	Return a String.
`timestamp == nil`	Read the word at address `0x08010228` in this call and use that value `// 1000`.
Explicit timestamp	Apply the integer and range checks below.

Thus `"*t"` and `"!*t"` select the Table result. `"!!*t"` does not: after removing one `!`, the remaining `"!*t"` is an ordinary format string. With a valid timestamp it is copied literally. The removed `!` is not retained as a mode flag and does not select a different calendar or time-zone calculation.

`date` selects the format, removes one leading `!`, then selects or checks the timestamp. It decomposes the calendar fields before producing the chosen Table or scanning a String format. An empty format still checks its timestamp; a malformed conversion is reached after timestamp processing.

`false` is not a substitute for `nil`. A false format reaches the length operation within `string.sub` before timestamp processing and raises `Attempted to get length of an unsupported value.` (Lua reason 0); it does not produce a date-specific argument error. A false timestamp is passed to the retained `math.modf` function rather than selecting the default timestamp. Its integer check invokes `error` with `time is not an integer`, before the bounds check.

9.6.2 Explicit timestamp checks

The explicit timestamp check first obtains the integer result of the retained `math.modf` function. In the supplied Math library, this truncates toward zero: for a negative value the calculation is `-((-value) // 1)`, otherwise it is `value // 1`.

If the retained integer differs from the input, the call invokes `error` with exactly:

```
time is not an integer
```

Only after that check does it require the integer to lie within the inclusive range `-0x1fffffffffffff` through `0x1fffffffffffff`. A value outside that range invokes `error` with exactly:

```
time is out-of-bound
```

The accepted integer is then used for decomposition. There is no separate date-specific type-dispatch or numeric-text parsing layer. The default timestamp read is not passed through these explicit-timestamp checks.

9.6.3 Table result and calendar fields

The `"*t"` form creates a new Table on every call and writes exactly these fields:

Field	Meaning
`year`	Civil year.
`month`	Civil month, with January numbered 1.
`day`	Day of the civil month.
`hour`	Hour derived from the seconds within the day.
`min`	Minute within that hour.
`sec`	Remaining seconds within that minute.
`wday`	Weekday with Sunday numbered 1 and Saturday numbered 7.
`yday`	Day of the civil year, with January 1 numbered 1.
`isdst`	Always `false`.

This form does not consume a caller-supplied date Table. It returns the newly created Table itself.

The following is documentary formula notation, not a BLua program. The shared civil-day calculations above define `D(y, m, d)` and `C(days)`; both result forms use those same calculations.

```
days           = timestamp // 86400
seconds_of_day = timestamp - days * 86400
(year, month, day) = C(days)
hour           = seconds_of_day // 3600
min            = (seconds_of_day - hour * 3600) // 60
sec            = seconds_of_day - hour * 3600 - min * 60
yday           = days - D(year, 1, 1) + 1
wday           = ((days + 4) % 7) + 1
```

The Table and String forms use this same decomposition, including after a leading `!`.

9.6.4 Format grammar

The formatter scans String codepoints, not UTF-8 byte offsets. Every codepoint other than `%` is copied literally. A `%` consumes one following conversion code; it may first consume exactly one `E` or `O` modifier.

There are no format flags, field-width syntax, precision syntax, repeated modifiers, or alternative escape introducers. The letters in the modifier table are the complete permitted sets:

Modifier	Permitted following codes	Effect
`E`	`c C x X y Y`	Use exactly the corresponding unmodified conversion.
`O`	`d e H I m M S u U V w W y`	Use exactly the corresponding unmodified conversion.

The modifier is checked before conversion dispatch. It does not select alternative names, digits, or eras. For example, `%Ec` follows the `%c` path and `%OV` follows the `%V` path. A code may be accepted without a modifier but rejected with one.

9.6.4.1 Conversions

In this table, `Z(value, n)` means: obtain the value's text with the current global `tostring`, then prepend `0` until the total text length is at least `n`. `S(value, n)` does the same with spaces. These are explanatory symbols, not callable BLua functions. Neither operation truncates text or places padding specially around a sign. The numeric-text limitations below apply to every such entry.

`hour12` is 12 when `hour % 12 == 0`, otherwise `hour % 12`. `AMPM` is `AM` when `hour < 12`, otherwise `PM`. `iso_year`, `iso_week`, `sunday_week`, and `monday_week` are defined under Week numbering.

9.6.4.2 Calendar fields

Conversion	Output
%C	Z(year // 100, 2)
%D, %x	Z(month, 2)/Z(day, 2)/Z(year % 100, 2)
%F	Z(year, 4)-Z(month, 2)-Z(day, 2)
%G	Z(iso_year, 4)
%g	Z(iso_year % 100, 2)
%Y	Z(year, 4)
%y	Z(year % 100, 2)
%m	Z(month, 2)
%d	Z(day, 2)
%e	S(day, 2)

9.6.4.3 Time of day

Conversion	Output
%H	Z(hour, 2)
%M	Z(min, 2)
%S	Z(sec, 2)
%I	Z(hour12, 2)
%R	Z(hour, 2):Z(min, 2)
%r	Z(hour12, 2):Z(min, 2):Z(sec, 2) AMPM
%T, %X	Z(hour, 2):Z(min, 2):Z(sec, 2)
%p	AMPM

9.6.4.4 Names and day of year

Conversion	Output
%a	Short weekday name.
%A	Long weekday name.
%h, %b	Short month name.
%B	Long month name.
%j	Z(yday, 3)

9.6.4.5 Literal characters

Conversion	Output
%n	One newline codepoint.
%t	One tab codepoint.

9.6.4.6 Week numbers and composite output

Conversion	Output
%U	Z(sunday_week, 2)
%u	tostring(wday == 1 and 7 or wday - 1), without padding: Monday 1 through Sunday 7.
%V	Z(iso_week, 2)
%w	tostring(wday - 1), without padding: Sunday 0 through Saturday 6.
%W	Z(monday_week, 2)

Conversion	Output
%c	shortWeekday shortMonth Z(day, 2) Z(hour, 2):Z(min, 2):Z(sec, 2) Z(year, 4)
%z	Literal +0000.
%Z	Literal BMSX.
%%	Literal %.

Slashes, hyphens, colons, and spaces shown between components are literal characters. In particular, %c has one space between each displayed component and uses a **zero-padded** day, not the space-padded %e conversion.

The names are fixed, not locale lookups:

Kind	Names in index order
Short weekdays	Sun Mon Tue Wed Thu Fri Sat
Long weekdays	Sunday Monday Tuesday Wednesday Thursday Friday Saturday
Short months	Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
Long months	January February March April May June July August September October November December

9.6.4.7 Week numbering

The following formulas are explanatory notation. They use the decomposed civil year and the D civil-day calculation defined above.

```
sunday_week = (yday - 1 + 7 - (wday - 1)) // 7
monday_week = (yday - 1 + 7 - ((wday + 5) % 7)) // 7
```

For %U, week 0 is the part of the year before its first Sunday; week 1 starts on that Sunday. For %W, the corresponding boundary is Monday. These two conversions do not adjust the civil year.

The ISO conversions first remap weekdays to Monday 1 through Sunday 7, then calculate:

```
iso_weekday = 7 if wday == 1, otherwise wday - 1
candidate   = (yday - iso_weekday + 10) // 7
```

An ISO year has 53 weeks exactly when its January 1 is Thursday, or when January 1 is Wednesday and the difference D(year + 1, 1, 1) - D(year, 1, 1) is 366. Otherwise it has 52 weeks. The January 1 weekday for this rule is ((D(year, 1, 1) + 3) % 7) + 1, with Monday numbered 1.

If the candidate is below 1, iso_year is the preceding civil year and iso_week is that year's week count. If the candidate exceeds the current year's week count, iso_year is the following civil year and iso_week is 1. Otherwise iso_year is the civil year and iso_week is the candidate. %G and %g format this ISO year, not necessarily the year used by %Y and %y.

9.6.4.8 Invalid conversions and embedded zero

Any unsupported conversion invokes the current global error with this assembled message:

```
invalid conversion specifier '%<modifier><code>'
```

Here <modifier> is empty, E, or O, and <code> is the consumed codepoint. These angle-bracket terms describe assembly; they are not literal message text. With an unfinished conversion, the absent code is the empty String:

Format ending	Error String
`%`	`invalid conversion specifier '%'`
`%E`	`invalid conversion specifier '%E'`
`%O`	`invalid conversion specifier '%O'`
`%EO`	`invalid conversion specifier '%EO'`

In `%EO`, `O` is checked as the code after `E`; it is not parsed as a second modifier.

A zero codepoint is not a String terminator. An embedded literal zero is copied and scanning continues. A zero used as the code after `%` is unsupported; the assembled error message contains that zero codepoint.

9.6.5 Numeric text and padding limits

Padding operates on the complete text returned by the current global `tostring`. For example, zero-prepending the text of `-1` to a minimum length of four produces `00-1`, not `-001`. A padding width is a minimum total text length, not a fixed digit count.

With the supplied `tostring`, numbers use the shared numeric-text conversion:

- Zero has signed-zero handling.
- Integral magnitudes strictly below `1,000,000` use complete ordinary decimal integer text.
- Other finite values are normalized to a decimal exponent and rounded to six significant digits, with ties to even.
- After rounding, fixed form is selected when the adjusted exponent is at least `-4` and below `6`; otherwise scientific form is selected.
- Scientific form uses lowercase `e`, an explicit exponent sign, and at least two exponent digits. Trailing fractional zeros and a trailing decimal point are removed.

The formatter pads that result unchanged. `%Y`, `%G` and `%C` can therefore contain rounded scientific text for large years or centuries. `%Y` is padded to a minimum length of four, not constrained to four decimal digits. Calendar arithmetic uses Number operations, including at the extremes of the accepted timestamp range.

9.6.6 Library bindings and execution cost

Date validation retains its installation-time truncation function. Formatting uses `sub` from the original String Table, while `tostring` and `error` are read through their current global bindings. Replacing the `string` global leaves that original Table in use; replacing its `sub` field changes slicing.

The Table result performs one calendar decomposition and stores the nine fields. String output additionally scans and concatenates text. ISO-week conversions calculate their year/week pair for each occurrence, and padding adds characters until the requested minimum length is reached. Format length, selected conversions and output size therefore affect execution cost.

`%z`, `%Z` and `isdst` always use the fixed values listed above. They represent the BIOS civil-time convention rather than a configurable zone or daylight-saving rule.

10 BIOS String operations

Use `string` for text construction, character access, slicing, formatting, pattern matching and formatted String records. Normal BIOS startup installs the Table and registers it for String method lookup. Call `string.sub(text, start, finish)` or the corresponding `text:sub(start, finish)` form.

10.1 Text values and encoded byte storage

String positions begin at 1 and count Unicode codepoints. `string.byte` selects one complete codepoint; `string.char` constructs a String from numeric codepoints. A String's language value is separate from an encoded byte range in ROM or RAM.

Use `string.from_utf8(address, byte_count)` to construct text from mapped UTF-8 bytes. The address selects actual byte storage and the count measures encoded bytes. BLua storage declarations define those ranges; Book I, "Memory and bus access" gives their physical mapping and reservations. CPU value rules specify indexing and Number conversion, and protected calls describe language-error handling.

The following example contains four codepoints. String positions start at one; the mapped byte offsets start at zero. The last two codepoints each occupy more than one encoded byte.

String position	Codepoint	Byte offsets in the example	Encoded bytes
`1`	`41H`	`0`	`41H`
`2`	`7AH`	`1`	`7AH`
`3`	`E9H`	`2–3`	`C3H A9H`
`4`	`1F680H`	`4–7`	`F0H 9FH 9AH 80H`

Thus eight encoded bytes and four String codepoints describe the same example content. A zero codepoint is also content, not a terminator. Codepoint operations do not imply that every text display can draw every codepoint.

Public library fields are mutable. String method lookup continues to use the installed String-index Table after the global `string` name is rebound. For character selection, use `string.byte` or a substring operation rather than numeric Table-style indexing.

Choose the interface according to the input representation:

Input or task	Interface	Units
Inspect or slice an existing String	`string.byte`, `string.sub`, `string.len`	One-based codepoint positions and codepoint counts.
Construct text from character values	`string.char(...)`	Numeric Unicode codepoints.
Decode an encoded memory span	`string.from_utf8(address, byte_count)`	Mapped byte address and encoded byte count.
Format a Number or other value for display	`string.format(format, ...)`	The format selects textual representation, width and precision.
Build or read a formatted String record	`string.pack`, `string.unpack`	Codepoint-valued packet units; these calls do not transfer a record to mapped storage.
Interpret existing floating-point words	Named import `string/float/decode`	One binary32 word or high/low binary64 words; one Number result.

In particular, the size of a packed String is not a DMA transfer length. Define a byte layout in mapped storage when a device consumes bytes or words; use String operations for language text and String records.

10.2 Example: codepoints and mapped UTF-8

Declare the encoded input as byte storage, then pass its address and byte count to the BIOS. The resulting String is an independent text value:

```
rodata utf8_bytes: u8[8] = {
    0x41, 0x7a, 0xc3, 0xa9, 0xf0, 0x9f, 0x9a, 0x80,
}
local text = string.from_utf8(utf8_bytes, #utf8_bytes)
local length = string.len(text)              -- 4 codepoints, from 8 bytes
local second = string.byte(text, 2)          -- 122
local tail = text:sub(2, -1)                 -- codepoints 122, 233, 128640
local capitals = string.upper(text)          -- codepoints 65, 90, 233, 128640
local backwards = string.reverse(text)       -- codepoints 128640, 233, 122, 65
local repeated = string.rep('go', 3, '/')     -- 'go/go/go'
```

The same text can be constructed as `string.char(0x41, 0x7a, 0xe9, 0x1f680)`. Method syntax uses the installed String-index Table; it does not make String positions into memory addresses.

10.3 Codepoint access, construction, length and UTF-8 input

These four interfaces operate on codepoints or bring encoded bytes into a String. Their names do not make a String a byte buffer.

Interface	Arguments	Normal result
`string.byte(value, position)`	Required String; optional Number position	Exactly one Number, or exactly one `nil` when the position is outside the String
`string.char(...)`	Zero or more Numbers	Exactly one String
`string.len(value)`	Required value supporting the length operation	Exactly one Number, if the operation completes
`string.from_utf8(address, byte_count)`	Numeric mapped byte address and a byte count	Exactly one String, if decoding completes with the installed bindings

10.3.1 `string.byte(value, position)`

Positions are one-based **codepoint positions**. Omitting `position`, or passing `nil`, selects position 1. An empty String has no valid position. A position below 1 or above the codepoint count returns one `nil`; negative positions do not count back from the end.

A supplied non-`nil` position must be a Number. The conversion to a signed 32-bit word truncates a finite value toward zero, wraps modulo 4,294,967,296, and interprets the high bit as the sign bit. NaN and either infinity convert to zero. This conversion is not the rounding used by every other String operation.

For `value = string.char(65, 122, 233, 128640)`:

Call	Result
`string.byte(value)`	`65`
`string.byte(value, nil)`	`65`
`string.byte(value, 2.9)`	`122`
`string.byte(value, 0x100000001)`	`65`
`string.byte(value, -1)`	`nil`
`string.byte(value, 5)`	`nil`
`string.byte(value, 2, 4)`	One result: `122`

There is no end-position argument or range-return form. Additional arguments are ignored. Each call scans forward from the first codepoint to the selected position; repeatedly selecting later positions repeats that traversal.

A missing or non-String first argument, or a nonnumeric non-`nil` position, raises `INVALID_ARGUMENT`. A numeric-looking String such as `"2"` is not accepted as a position.

10.3.2 `string.char(...)`

Pass one Number for each Unicode scalar to put in the result. Values are not restricted to 0 through 255. Each Number is converted to an unsigned 32-bit word: a finite value is truncated toward zero and wrapped modulo 4,294,967,296; NaN and either infinity convert to zero. The converted value must be in `0x000000` through `0x10ffff`, excluding `0xd800` through `0xdfff`.

Call	Result
`string.char()`	One empty String
`string.char(65)`	One String containing codepoint 65
`string.char(65.9)`	One String containing codepoint 65
`string.char(0x100000041)`	One String containing codepoint 65
`string.char(0)`	One String containing one zero codepoint
`string.char(233, 128640)`	One String containing those two codepoints
`string.char(-1)`	`INVALID_ARGUMENT` after conversion
`string.char(0xd800)`	`INVALID_ARGUMENT`
`string.char("65")`	`INVALID_ARGUMENT`

Zero is an ordinary codepoint, not a String terminator. NaN and infinity therefore produce a zero codepoint rather than an empty String. A nonnumeric argument or a converted value outside the accepted scalar range raises `INVALID_ARGUMENT`; decimal String arguments are not converted to Numbers. The result is text, not an allocation of raw bytes at a mapped address.

For `byte` and `char`, `INVALID_ARGUMENT` is a language argument error. A protected call receives the original error String `"Invalid argument."`. It is not a Memory interrupt.

Each `byte` or `char` call has a builtin surcharge of two cycles, with no per-argument or per-result term in that surcharge. Scanning or constructing the String and the surrounding call instructions add their own work.

10.3.3 `string.len(value)`

`string.len(value)` performs the ordinary length operation, `#value`, and returns its result. On a String it counts codepoints, including zero codepoints. It does not report an encoded UTF-8 byte count.

On a Table it returns the maintained cached length, not the number of keys or the largest numeric key. For the Table length operation, its state rules and its fault handling, follow the CPU chapter's length-operation procedure. Other value categories raise the ordinary length error, `"Attempted to get length of an unsupported value."`.

10.3.4 `string.from_utf8(address, byte_count)`

Use this interface when the input already exists as mapped UTF-8 bytes. `address` is the numeric address of the first byte; `byte_count` is the number of **encoded bytes**, not the number of codepoints. Supply both arguments. A String value, or the address of a field holding a String constant index, is not the address of its encoded text.

10.3.4.1 Text, codepoints and UTF-8 bytes

A static `string` field stores an image constant reference. Its byte address locates that reference, not inline UTF-8. Similarly, `string.pack` produces a String value rather than an addressed hardware buffer. Use explicit byte storage for a device payload; the decoded String is not a retained DMA buffer.

This eight-byte span includes both a zero byte and a four-byte sequence:

```
rodata utf8_bytes: u8[8] = { 0x41, 0xc3, 0xa9, 0, 0xf0, 0x9f, 0x99, 0x82 }
local text = string.from_utf8(utf8_bytes, #utf8_bytes)
local count = #text
local zero = string.byte(text, 3)
local face = string.byte(text, 4)
```

Eight bytes decode to four codepoints: `0041H`, `00E9H`, `0000H`, `1F642H`. `count` is 4, `zero` is 0 and `face` is `1F642H`. Embedded zero does not terminate the String, and the supplementary codepoint counts as one. The declaration's `#utf8_bytes` counts byte-array elements; `#text` counts codepoints. These are different units even when both are expressed as Numbers.

10.3.4.2 Input lifetime and sequence reads

The caller must provide backed storage containing complete, well-formed UTF-8 sequences and keep those bytes valid and stable while decoding reads them. The function does not allocate the input storage or grant access to it. An embedded byte `0x00` becomes codepoint zero and does not end the input. With the installed bindings, a zero byte count returns one empty String.

The byte count controls whether another lead byte is read. It is **not a hard read barrier** for the continuation bytes of a sequence already begun:

Lead byte value	Bytes read for that sequence, including the lead
`0x00–0x7f`	1
`0x80–0xdf`	2
`0xe0–0xef`	3
`0xf0–0xff`	4

These are the decoder's read shapes, not a validation table. The decoder does not check continuation prefixes or reject overlong encodings before constructing the codepoint. Continuation reads are not compared with the end of the declared span. For example, if mapped bytes `0xc3, 0xa9` are both present, declaring only one byte still reads the second byte and produces codepoint `0xe9`. Do not use a shortened count to make an incomplete sequence safe.

Malformed input does not have a replacement-character policy. For example, the overlong sequence `0xc0, 0xaf` produces codepoint 47. A decoded surrogate, such as the value produced by `0xed, 0xa0, 0x80`, reaches `char` and raises its `INVALID_ARGUMENT` error. Mapped-access faults and their recovery belong to the Memory, CPU and SYSTEM rules; there is no separate invalid-UTF-8 error.

Literal escapes do not perform this decoding. The String literal `"\xc3\xa9"` contains two codepoints, `0xc3` and `0xa9`. To obtain codepoint `0xe9` from the encoded bytes `0xc3, 0xa9`, put those bytes in mapped storage and call `string.from_utf8` with that address and a byte count of 2.

The decoder stages 256 codepoints per chunk and joins its chunks into the returned String. This is not a 256-codepoint size limit: a 257-byte input containing only `0x41` produces 257 codepoints. After normal return, the String is not a live view of the source bytes; rewriting those bytes does not change the returned text.

The decoder retains its `char`, `table.unpack` and `table.concat` function values. Replacing their public fields does not replace these calls. The captured `unpack` still uses the current global `select` for an empty range; the captured `concat` still uses the current global `tostring` for its joins. Changing those globals can therefore change the empty-span or chunk-join result. BIOS table library describes the helper procedures.

10.4 Case conversion, slicing, reversal and repetition

The contracts below describe the installed bindings. Each call returns exactly one result when it completes. These functions do not write into their input. For a String, positions and lengths count codepoints, including codepoint zero; they do not count UTF-8 bytes.

Call	Operation on a String
`string.upper(value)`	Change ASCII `a–z` to `A–Z`; preserve every other codepoint.
`string.lower(value)`	Change ASCII `A–Z` to `a–z`; preserve every other codepoint.

Call	Operation on a String
`string.sub(value, start_arg, end_arg)`	Select the inclusive codepoint range described below.
`string.reverse(value)`	Reverse the codepoint order.
`string.rep(value, count_arg, separator)`	Repeat the value; a non-nil separator is placed between copies.

10.4.1 ASCII case conversion

`upper` subtracts 32 from codes 97 through 122. `lower` adds 32 to codes 65 through 90. Neither operation performs locale-sensitive conversion or Unicode case folding.

```
local text = string.char(65, 122, 233, 128640)
local capitals = string.upper(text) -- codes 65, 90, 233, 128640
local small = string.lower(text)    -- codes 97, 122, 233, 128640
```

The result contains the mapped codepoints in their original order. Empty input returns an empty String. Supply a String: a positive-length Table fails at character access with `Invalid argument.`, while unsupported length kinds raise `Attempted to get length of an unsupported value.` A zero-length Table reaches no character read and produces an empty String.

10.4.2 Inclusive slices

Pass an explicit `nil` for `start_arg` when requesting its default; do not use `string.sub(value)` as the direct-call form. A nil start means 1, and an omitted or nil end means L, where `L = #value` is captured at the beginning of the call.

For each non-nil endpoint, first compute `q = endpoint // 1`. This is floor division: for example, 2.9 becomes 2 and -1.2 becomes -2.

Value of q	Start position	End position
Positive	q	q
Negative	L + q + 1	L + q + 1
Zero or NaN	1	0

After normalization, a start below 1 is raised to 1, and an end above L is lowered to L. Thus -1 selects the last position. Unlike `table.concat` and `table.unpack`, an explicit end of 0 means 0, **not** L.

The normalized range determines the result in this order:

Condition	Result
End < start	One empty String.
Start = 1 and end = L	The original value, unchanged.
Start = end	The String constructed from that one codepoint.
Otherwise	Codepoints from start through end, joined in order.

```
local middle = string.sub('ABCDE', 2.9, 4.9) -- 'BCD'
local tail = string.sub('ABCDE', -2, nil)     -- 'DE'
local empty = string.sub('ABCDE', 0, 0)       -- ''
local whole = string.sub('ABCDE', nil, nil)  -- original value
```

Non-Number endpoint values participate in arithmetic as NaN; numeric-looking Strings are not parsed. Consequently `string.sub('ABCDE', '2', '4')` selects an empty range: the start becomes 1 and the end becomes 0. This differs from `string.byte`, which requires a supplied non-nil position to be a Number.

There is no preliminary String-only check here either. With a Table, the empty-range branch can return an empty String and the full-span branch can return that original Table. A partial range that reaches a codepoint read raises `Invalid argument.` Other value kinds fail at `#value`.

10.4.3 Codepoint reversal

`string.reverse` captures `L = #value`. If L is at most 1, it returns the original value. Otherwise, it reads positions L down to 1, constructs a String for each codepoint, and joins the temporary Table with captured `table.concat`.

```
local backwards = string.reverse('ABC') -- 'CBA'
```

This reverses codepoints, not UTF-8 bytes or grapheme clusters. A combining character therefore moves independently of its neighbouring codepoint.

The early return also applies to a Table whose captured L is 0 or 1: the result is that Table, not a String. With a Table whose L is greater than 1, the first codepoint read raises `Invalid argument.` Other value kinds fail at the length operation.

10.4.4 Repetition and separators

A missing or nil count means 1. Otherwise, the count is `count_arg // 1`; non-Number counts become NaN rather than being parsed or rejected by a count-type check. A count at most zero returns one empty String before the value or separator is concatenated.

For String inputs and a finite normalized count N > 0, the result contains N copies. A non-nil separator goes between those copies, not before the first or after the last.

```
local joined = string.rep('go', 3, '/') -- 'go/go/go'
local once = string.rep('go', nil)      -- 'go'
local none = string.rep('go', 0, '/')  -- ''
```

The function does not require the value or separator to be a String. Its branch order determines the result:

Count and separator	Procedure and result
Count ≤ 0, any separator	Return `''`; do not concatenate the supplied value or separator.
Count = 1, nil separator	Concatenate `'' .. value`; return the resulting String.
Count = 1, non-nil separator	Compute `separator .. value`, then return the original value.
NaN count, nil separator	Return `''`.
NaN count, non-nil separator	Compute `separator .. value`, then return the original value.

For example, `string.rep(7, 1)` returns the String `'7'`, whereas `string.rep(7, 1, '|')` returns the Number 7. `string.rep(7, 0, {})` returns `''`. The String count `'3'` follows the NaN rows, not the finite-count rule.

Concatenation converts values to Strings using the ordinary conversion rules: Numbers use their Number formatter, and nil, Booleans, Tables and functions use their ordinary labels. It does not call `__tostring`. A separator is absent only when it is nil; `false` is a supplied separator.

For a positive finite count, repetition uses parity tests and repeated halving of the remaining count, doubling an intermediate chunk when more work remains. It does not concatenate one copy per loop iteration. Intermediate Strings and the requested output still consume resources; there is no function-local repetition or output-size cap. Negative infinity takes the empty-result branch. Positive infinity does not decrease under halving, so it is not a terminating repetition request and may exhaust resources.

10.4.5 Captured calls and work performed

The codepoint-building paths above use captured `string.byte`, `string.char` and `table.concat` function values. Replacing those public fields later does not replace the captured calls. Their remaining live dependency is significant: the captured concat function still consults the current global `tostring`. If that global is replaced with a function returning `'x'` for every argument, `string.upper('ab')` produces `'xx'`.

Each codepoint read starts scanning at the beginning of its source String. Repeated indexed reads are therefore repeated scans, not constant-time random access. The mapping, slice and reversal paths also construct temporary Tables and Strings; they are not in-place transformations or single-pass byte-buffer operations.

10.5 string.format

```
string.format(format, ...)
```

Number arithmetic and value categories follow Values and numeric instructions. Hexadecimal limits below use the `H` suffix.

Builds formatted text and, on normal completion, returns exactly one String. It does not return a character count or write into a caller-supplied buffer. Unused arguments are ignored.

For a String `format`, positions, field widths and String precisions count Unicode codepoints, not UTF-8 bytes. Literal codepoints are copied in order. An embedded NUL does not terminate the format. The main scan uses the initial format length; decimal width and precision parsing re-evaluate that length at each loop test.

Table-length dependency. There is no String-only entry check: the first operation is `#format`. Categories other than String and Table fail at that operation. A Table whose length is obtained as zero returns an empty String; a positive length leads to a codepoint-read operation that requires a String and fails. General-uppercase conversion and substring operations used by precision-limited `%s` and general-format zero stripping can also construct temporary Tables. These Tables use the CPU's maintained-length rules.

10.5.1 Example: formatted text

Width includes the sign and base prefix. In a starred field, supply width, then precision, then the value:

```
local label = string.format('HP %d/%d', 7, 10)        -- 'HP 7/10'
local signed = string.format('%+06d', 42)             -- '+00042'
local hexadecimal = string.format('%#08x', 42)        -- '0x00002a'
local amount = string.format('%*.*f', 8, 2, 1.25)      -- '    1.25'
local left = string.format('%*s', -5, 'go')           -- 'go   '
local zeros = string.format('%.0d/%#.0o', 0, 0)       -- '/0'
local unsigned = string.format('%u', -1)              -- '4294967295'
local floating = string.format('%.2f/%.1e/%#.3g',
    1.25, 12.5, 100)                                 -- '1.25/1.3e+01/100.'
```

`string.format('%.0f', 1000000)` returns `'1e+06'`, because its whole-number piece uses ordinary Number-to-text conversion. The conversion rules below explain this output when choosing a numeric format.

10.5.2 Conversion syntax and argument order

An immediate `%%` emits one percent and consumes no argument. All other conversions have this order:

```
% flags width .precision modifiers conversion
```

Each component except `%` and the final conversion is optional. The dot belongs to the precision component.

Component	Accepted form
Flags	Any order and repetition of `-`, `+`, space, `0`, `#`. Repeating a flag has no additional effect.
Width	`*` or a run of ASCII decimal digits. Leading zeros are parsed as flags before the width stage.
Precision	`.` followed by `*` or zero or more ASCII decimal digits. A dot with no following digits records precision zero, provided a conversion still follows.
Modifiers	Any order and repetition of `l`, `L`, `h`. They are ignored and do not change the numeric representation.
Conversion	Exactly one of `s c d i u o x X f F e E g G q`.

Decimal fields are accumulated by multiplying the previous value by ten and adding the next digit. There is no field digit-count or numeric cap. There is no positional `$` syntax. No additional conversion is supplied for `a`, `A`, `p` or `n`; a percent reached after parsing flags, width or precision is not the `%%` escape.

Arguments are selected sequentially: width `*` first, precision `*` second, then one value for a recognized conversion. An unsupported conversion does not select a conversion value, although its stars have already selected their arguments. There is no argument-count preflight or too-many-arguments error. With the installed argument-selection function, an exhausted argument list supplies Nil to a scalar selection; an omitted `format` is also Nil.

A star uses the first result of `math.modf`. For finite Numbers this truncates toward zero. A negative width enables left alignment and uses the negated width. A precision is retained only when its truncated value is nonnegative; a negative or NaN precision is absent. A non-Number star argument reaches this arithmetic as Number NaN, rather than being parsed as numeric text or rejected by a formatter-specific type check. For that width, the negative-width test and padding comparison are false; that precision is absent.

10.5.3 Flags, signs and padding

Width is a minimum, never a truncation limit. Padding measures the combined codepoint lengths of sign, base prefix and content. If padding is needed, `-` appends spaces and overrides `0`. Otherwise `0` inserts zeros after the sign and base prefix; ordinary padding inserts spaces before both.

For signed conversions, a value less than zero selects `-`; otherwise `+` selects `+`, then space selects one leading space. Thus `+` overrides space. Sign selection does not inspect a negative-zero sign bit.

Conversion group	Flag and precision effects
`s`, `c`, `q`	Space padding only; `+`, space and `#` have no conversion effect.
`d`, `i`	Signed decimal; `+` and space select a nonnegative sign. `#` adds no prefix.
`u`, `o`, `x`, `X`	No sign from `+` or space. `#` affects octal and hexadecimal as described below.
All integers	Any present precision disables width zero-padding, including precision zero.
Floating conversions	Width zero-padding remains active when precision is present. `#` retains a point at zero precision in fixed/scientific construction and disables general-format zero stripping.

10.5.4 Text, codepoint and quoted conversions

`%s` — text. Calls the current `tostring` on the argument, then, if precision is present, takes codepoints 1 through that precision before applying width. For an ordinary converted String, precision zero gives empty content. The argument need not originally be a String. Substring construction uses the maintained-length rule above.

`%c` — one codepoint. Takes the integer component from `math.modf` and passes that one value to `string.char`. Conversion to an unsigned 32-bit word truncates and wraps. A word above `10FFFFH` or within `D800H`–`DFFFH` raises the codepoint error. NaN and either infinity convert to codepoint zero; non-Number arguments also reach zero through the arithmetic conversion.

`string.format('%c', 0)` contains one NUL codepoint. Both `string.format('%c', 65)` and `string.format('%c', 65.75)` produce `'A'`. `string.format('%3c', 65)` produces two spaces followed by `'A'`. Precision is ignored, although a precision star still consumes an argument.

`%q` — quoted text. Calls the current `tostring`, surrounds the resulting text with double quotes, and escapes its codepoints as follows:

Codepoint	Emitted escape
10	`\n`
13	`\r`
9	`\t`
92, backslash	`\\`
34, double quote	`\"`

Codepoint	Emitted escape
Other values below 32, and 127	Backslash followed by the decimal code, zero-padded to three digits.

Every other codepoint is copied unchanged. Width applies after quoting. Precision is ignored, including after consuming a precision star. Because conversion to text happens first, `%q` is not a promise of round-trip serialization of the original value.

10.5.5 Integer conversions

`%d` and `%i` take the first `math.modf` result. Finite fractions truncate toward zero, and a negative integer is formatted using its positive magnitude and a separate minus sign. These conversions do not impose a 32-bit word limit.

`%u`, `%o`, `%x` and `%X` take that same first result, apply a bitwise AND with `FFFFFFFFH`, and interpret the result as an unsigned 32-bit magnitude. `%u` uses decimal, `%o` octal, and `%x`/`%X` hexadecimal with lowercase/uppercase letters.

Zero initially produces one `0`. Other magnitudes are converted by repeated remainder and floor division in the selected base. Integer precision adds leading zeros to the requested minimum digit count. Precision zero clears the digits when the magnitude is zero.

For `#` octal, empty digits are restored to one `0`; otherwise a leading `0` is added only if the first digit is not already zero. For `#` hexadecimal, `0x` or `0X` is added only for a nonzero magnitude. The common width rules then place sign, prefix and padding.

A signed NaN magnitude produces no initial digits; sign flags, precision and width can still add text. Non-Number arguments reach signed integer formatting as Number NaN. Unsigned conversion maps both that result and Number NaN or either infinity to zero. Either infinity in signed integer conversion produces a positive-infinite magnitude; repeated base division then fails to make progress toward completion.

10.5.6 Floating conversions

`f`/`F` select fixed construction, `e`/`E` scientific construction, and `g`/`G` general construction. They do not validate the original argument as a Number or parse numeric Strings.

A Number NaN is handled before construction as `nan` or `NAN`; either infinity is handled as `inf` or `INF`. Sign selection is separate, so negative infinity receives a minus sign and nonnegative special values may receive `+` or space. For other arguments, sign is selected from the original value and a value less than zero is negated to obtain the magnitude.

Non-Number arguments do not match the initial Number-special cases. Comparisons with the numeric constants are false, and subsequent arithmetic produces Number NaN. The selected construction below therefore proceeds with NaN arithmetic and text pieces; this is not the initial `nan`/`NAN` shortcut or a formatter-specific argument error.

Fixed construction (`f`, `F`). Precision defaults to six; explicit zero is retained. For magnitude `v` and precision `p`, construct `scale` by starting at one and multiplying by ten `p` times, then calculate:

```
rounded  = first modf result of (v * scale + 0.5)
whole    = rounded // scale
fraction = rounded % scale
```

Convert `whole` with the current `tostring`. When `p > 0`, append a dot and `tostring(fraction)`, padding that fraction text on the left with zeros to at least `p` codepoints. At precision zero, append a dot only for `#`.

Scientific construction (`e`, `E`). Precision also defaults to six. Zero uses fixed construction followed by exponent zero. Otherwise repeatedly divide the magnitude by ten while it is at least ten, or multiply it by ten while it is positive and below one, tracking the exponent from zero. Round the resulting mantissa using the same multiply-plus-half and first-`modf` operation. If the rounded value is at least `10 * scale`, divide it by ten using floor division and increment the exponent. Construct the whole/fraction text as above, then append `e` or `E`, an explicit exponent sign, and the current `tostring` of the absolute exponent, left-padded with zeros to at least two codepoints. Two is a minimum, not an exponent limit.

General construction (`g`, `G`). Precision defaults to six significant digits; explicit zero becomes one. Determine the decimal exponent from the unrounded magnitude, using exponent zero for zero. If the exponent is below −4 or at least the significant-digit precision, select scientific construction with precision one less than the significant-digit count. Otherwise select fixed construction with precision `significant − exponent − 1`. The choice is not repeated after rounding.

Without `#`, detach the first `e`/`E` suffix, remove every trailing zero from the preceding text, remove a trailing dot if present, and reattach the suffix. This stripping does not require a decimal point: it can remove integer trailing zeros as well. `#` skips stripping. `G` then applies ASCII-only uppercasing to the resulting text. Stripping and uppercasing use the maintained-length rule above.

Default numeric text dependency. The whole, fraction and exponent text pieces above use `tostring`; precision does not substitute an arbitrary-length decimal writer. Under the installed default, Nil becomes `nil`, Booleans become `false` or `true`, Tables become `table`, Threads become `thread`, function values become `function`, and Strings retain their contents. Number specials become `nan`, `-inf` or `inf`.

For finite Numbers, default text conversion writes positive zero as `0`, preserves a received negative zero as `-0`, and writes nonzero integral magnitudes below 1,000,000 exactly. Other finite-number text uses a six-significant-digit general algorithm: normalize, scale by 100,000, round with ties to even, adjust a six-digit carry, then choose fixed notation only for a post-rounding exponent from −4 through 5. Otherwise it uses lowercase `e`, an explicit sign and at least two exponent digits. Fractional trailing zeros are removed. This text algorithm is distinct from the formatter's multiply-plus-half rounding. Consequently, requested floating precision does not guarantee that all whole/fraction pieces are plain fixed decimal digits, or that every requested decimal digit is accurate.

10.5.7 Errors, completed work and limits

With the installed `error`, an incomplete conversion raises the String:

```
string.format incomplete format specifier.
```

This includes a final bare `%`, a final dot, and a conversion that ends after its flags, fields or ignored modifiers. An unsupported conversion raises `string.format unsupported format specifier '%C'.`, where `C` is replaced by that actual codepoint, not restricted to ASCII.

The formatter processes conversions in order. An error can occur after earlier argument selections or conversions; the call returns no partial String or count. Length and codepoint errors retain their ordinary texts.

The formatter keeps its original character, substring, repetition, concatenation and truncation functions. Its `tostring`, `select` and `error` calls use current globals. Replacing a saved helper's public field leaves the saved function in place; that helper can still use current globals of its own.

Choose finite widths and precisions appropriate to the output. Large padding and decimal precision create more text and require more processing. Positive-infinite precision prevents the scaling loop from progressing; positive-infinite width can prevent repetition from completing. String codepoint access is sequential, so repeated selections at distant positions also repeat the intervening traversal.

10.6 Pattern language

Patterns examine String codepoints, not UTF-8 bytes or grapheme clusters. Positions are codepoint positions. Character classes are ASCII classes; they do not expand to Unicode categories or locale-dependent character groups.

A pattern is interpreted as matching reaches it. There is no separate whole-pattern validation pass. An empty remaining pattern succeeds without consuming input.

10.6.1 Example: patterns and substitutions

Capture a record's text fields, then convert the numeric fields explicitly:

```
local name, x_text, y_text = string.match('ship_17 x=12 y=-3',
    '^(%w+)%s+x=(%-?%d+)%s+y=(%-?%d+)$')
local x, y = tonumber(x_text), tonumber(y_text)       -- 12, -3
local start, finish = string.find('a.b', '.', 1, true) -- 2, 2: literal dot
local position, digits = string.match('tag=42', '()(%d+)') -- 5, '42'

for word in string.gmatch('cat dog ship_17', '%w+') do
    print(word)                                       -- cat, dog, ship_17
```

```
end
local text, count = string.gsub('red blue', '(%a+)', '[%1]')
-- text is '[red] [blue]'; count is 2.
```

The following worked values distinguish matching and replacement choices:

Call	Results
`string.match('a1b2b', 'a.*b')`	`'a1b2b'`
`string.match('a1b2b', 'a.-b')`	`'a1b'`
`string.match('x(a(b)c)y', '%b()')`	`'(a(b)c)'`
`string.match('cat scatter', '%f[%w]cat%f[%W]')`	`'cat'`
`string.match('go go stop', '(%w+)%s+%1')`	`'go'`, the captured text
`string.find('abc', '', 4)`	One `nil`
`string.match('abc', '', 4)`	`''`
`string.gsub('red blue', '(%a+)', { red = false })`	`'false blue'`, `2`
`string.gsub('abc', '', '_')`	`'____'`, `4`
`string.gsub('x', '[', false, 0)`	`'x'`, `0`; matching is skipped

A Function replacement can retain a match by returning false. Unlike a Table replacement's false value, that return is not converted to text:

```
local changed, count = string.gsub('red blue', '(%a+)', function(word)
    if word == 'red' then
        return false
    end
    return 'B', 'ignored'
end)
-- changed is 'red B'; count is 2. Only the first return value is used.
```

10.6.2 Single-codepoint atoms and escapes

An ordinary character matches the same codepoint. A dot, `.`, matches any available codepoint, including a newline. An escaped punctuation character matches that punctuation literally: for example, `%.` denotes a literal dot.

The named classes are:

Class	Members
`%a`	ASCII `A`–`Z` and `a`–`z`
`%c`	Codepoints at most 31, and codepoint 127
`%d`	`0`–`9`
`%g`	Codepoints 33–126
`%l`	`a`–`z`
`%p`	Codepoints 33–47, 58–64, 91–96, and 123–126
`%s`	Space and codepoints 9–13
`%u`	`A`–`Z`
`%w`	ASCII letters, digits, and underscore
`%x`	`0`–`9`, `A`–`F`, and `a`–`f`
`%z`	Codepoint 0

Every uppercase ASCII letter following `%` complements the match for its lowercase letter. This rule is not restricted to the named classes: `%D` matches a non-digit, while `%Q` matches anything except lowercase `q`. An otherwise unrecognised nonuppercase escape matches its token literally; `%q` matches `q`, and `%0` matches `0`. Outside bracket sets, `%b`, `%f`, and `%1` through `%9` instead introduce the special constructs described below.

The character `|` does not introduce alternation. Braces do not introduce counted repetition, and backslash is not the pattern escape character. The characters `*`, `+`, `-`, and `?` act as repetition suffixes only when examined immediately after an ordinary atom. When encountered as a new atom, one of these characters is literal and can itself have a following repetition suffix.

The ordinary single-character check rejects positions beyond the source length, but does not itself reject positions below 1. A search begun sufficiently far below the first codepoint therefore follows below-range codepoint reads and String substring indexing; it does not acquire an implied lower clamp or a guarantee of ordinary 1-based matching boundaries.

10.6.3 Bracket sets

A bracket set, `[...]`, is an ordinary one-codepoint atom. A `^` immediately after `[` negates membership. The set scanner skips `%` and its following position, and the first other `]` closes the set. An unescaped `]` closes it even when it would be the first member. Write `%]` to include a literal closing bracket.

Inside a set, escapes use the class rules above. Named classes, uppercase complements, and escaped literals are available. The outside-set constructs are not available there: `%b` and `%f` mean literal `b` and `f`, and `%1` means literal `1`.

A range is recognised when an unescaped, single-item left member is followed by `-`. Its right endpoint must also parse as one item for range membership to match. Endpoints are compared as numeric codepoints, inclusively; reversed endpoints match nothing. An escaped endpoint does not form an ordinary codepoint range. A right-side escape can be consumed without contributing range membership.

Do not assume a final hyphen is literal. The range scanner can take the set's closing `]` as the right endpoint. Write `%-` when literal hyphen membership is required.

An unescaped `[` encountered while set membership is traversed is itself passed to the bracket-item parser. This can skip subsequent text; it is not a nested-set union operation. Write `%[` for literal opening-bracket membership.

An empty positive set matches nothing. An empty negated set matches any available source codepoint. A reached `[` without an unescaped closing `]` raises `invalid string pattern`.

10.6.4 Anchors and search order

Only a `^` at the first pattern position is an anchor. It is removed before matching and restricts the search to its supplied starting position. It does not independently assert the absolute beginning of the String. In repeated matching or substitution, it restricts each search to that operation's current search position. A `^` elsewhere is literal, except for set negation immediately after `[`.

Only a `$` at the final pattern position is an end anchor. It succeeds exactly when the current position is one past the source's last codepoint. A `$` elsewhere is literal.

Without a leading `^`, candidate starts are tried in increasing order, through the position one past the source end. The first successful candidate wins. Capture bookkeeping starts afresh for each candidate. The search does not silently switch to literal searching merely because a pattern looks literal.

10.6.5 Greedy and minimal repetition

Repetition applies to ordinary single-codepoint atoms, including bracket sets and ordinary escaped classes.

Suffix	Matching order
`?`	Try one occurrence, then zero
`*`	Greedy zero or more: try the longest span first, then shorter spans
`+`	Require one occurrence, then greedily expand the remainder
`-`	Minimal zero or more: try the remaining pattern first, then consume another codepoint if needed

These choices depend on whether the rest of the pattern succeeds. Greedy matching can retreat from its longest span; minimal matching can grow beyond an empty span.

Capture delimiters, balanced matching, frontier matching, and backreferences take their own matching branches before repetition suffixes are considered. They cannot be repeated as groups or special constructs. A following `*`, `+`, `?`, or `-` is processed separately as an ordinary atom, not as repetition of that preceding construct. There is no alternation or counted-repeat construct.

10.6.6 Captures and backreferences

An opening `(` records the current source position. A closing `)` searches backwards for the most recent capture entry whose finish is still unset and records its finish at the current position. A text capture is extracted from its recorded start through one codepoint before its finish. Capture entries are created in opening order.

The form `()` is different: it records the numeric current position without consuming text. It is a position capture, not an empty text capture.

Capture-result dependency. Capture bookkeeping and extraction use the CPU's maintained Table lengths. Returned sequences also follow the ordinary caller's result adjustment.

Outside sets, `%1` through `%9` refer to a previously recorded, finished text capture. A backreference compares that capture's codepoints at the current source position and, on success, advances by the captured length. A missing capture, an unfinished capture, or a position capture raises `invalid string pattern capture` when that reference is reached.

A backreference has exactly one digit. `%10` is `%1` followed by literal `0`; it is not a reference to capture 10. Capture creation has no pattern-local count guard, but the backreference notation still names only entries 1 through 9.

A reached `)` raises `invalid string pattern` if the backwards search finds no entry with an unset finish. That search tests only the finish field; it does not exclude position-capture entries. Structurally odd combinations of position captures and closing parentheses therefore follow that same procedure, rather than a separate balanced-parenthesis validity rule.

Reaching the pattern end does not check for unfinished opening captures. If such a capture is later extracted, extraction computes its finish minus one even though the finish is unset. There is no dedicated unclosed-capture error at pattern end.

10.6.7 Balanced regions

The form `%bxy` requires the current source codepoint to be `x`. It scans forwards with an iterative depth counter, increasing depth for nested `x` and decreasing it for `y`. When a closing `y` reduces depth to zero, matching continues just after that closing codepoint and after the four-codepoint `%bxy` construct in the pattern. A different starting codepoint, or failure to find a closing delimiter, makes this match fail.

The closing-delimiter comparison is performed before the opening-delimiter comparison. This order also applies when `x` and `y` are the same codepoint.

There is no dedicated preliminary check that both delimiter positions exist. An incomplete `%b` construct proceeds through two String delimiter reads and then the balance scan.

10.6.8 Frontiers

The form `%f[set]` is zero-width. It requires the preceding codepoint not to belong to the set and the current codepoint to belong to it. The preceding value is codepoint 0 when the current position is at most 1; otherwise it is read from the preceding String position. The current value is read from the String when the position is at most the source length; otherwise it is codepoint 0. The read applies the signed-32 position conversion of `string.byte`. A converted position below 1 returns `nil`, not codepoint 0. These values take part in the ordinary set-membership test.

The `[` must immediately follow `%f`; otherwise a reached frontier construct raises `invalid string pattern`. Its set uses the same scanner, membership rules, and missing-closing-bracket error as an ordinary bracket set. A successful frontier consumes no source codepoint.

10.6.9 Reached errors and incomplete forms

Pattern syntax has these explicit error conditions:

Reached condition	Error text
A bracket item has no unescaped closing `]`	`invalid string pattern`
`%f` is not immediately followed by `[`	`invalid string pattern`
`)` finds no capture entry with an unset finish	`invalid string pattern`
`%1`–`%9` names a missing, unfinished, or position capture	`invalid string pattern capture`

These error Strings are passed to the current global `error` function. String, Table and arithmetic operations retain their own errors.

Only reached pattern text is parsed. A failed earlier atom can leave later malformed text unexamined, and a search can finish after its first successful candidate or its single anchored candidate. Conversely, a bracket atom is parsed before the ordinary character check rejects a position beyond the source end; source exhaustion does not universally shield a reached bracket error.

Some operations can avoid the matcher altogether: `find` can return at its initial start bound, creating a `gmatch` iterator does not invoke it, and `gsub` can skip its first matching iteration when its limit comparison is false. Those paths do not validate the unused pattern.

A terminal `%` has no dedicated rejection. Special-escape dispatch first reads a missing token; the ordinary item scan then advances by two pattern positions. If single-codepoint matching is reached, a source position greater than the String length fails before class matching; otherwise class matching receives the missing escape token. Numeric comparisons reached with that missing token are false. Incomplete balanced constructs, unfinished captures and odd capture closures follow their respective operations above rather than a separate syntax-validation pass.

10.7 Pattern searches and substitutions

Pattern calls return positions, captured substrings or replacement text as specified below. Capture and result sequences use ordinary Table lengths and ordinary call-result adjustment. The CPU reference defines those rules. Matching and substitution also allocate temporary language objects; budget for them when processing large text.

10.7.1 `string.find`

```
string.find(source, pattern [, start [, plain]])
```

Searches `source` and constructs the start and inclusive end positions of the first match. Positions count String codepoints, beginning at 1; they are neither UTF-8 byte offsets nor grapheme-cluster positions. See Pattern language for pattern syntax, capture ordering and position captures.

Arguments and starting position. Omitted or nil `start` means 1. Otherwise the function computes `start // 1`: a positive result is used directly, a negative result becomes `#source + result + 1`, and zero becomes 1. A result still below 1 is not clamped. Do not rely on such a start producing ordinary in-range match boundaries. This arithmetic does not parse numeric Strings; a nonnil non-Number makes both sign comparisons false and therefore normalizes to 1.

If the normalized start is greater than `#source`, `find` returns one nil before inspecting `plain` or the pattern. In particular, a start of `#source + 1` takes this return, even for an empty pattern. Otherwise a pattern search can consider candidates through `#source + 1`. A leading `^` restricts the search to the supplied start; it does not independently assert the beginning of the complete source.

Literal search. A truthy `plain` selects literal codepoint comparison instead of the pattern language. Only nil and false disable this option: 0 and the empty String enable it. An empty literal needle constructs `(start, start - 1)` after the initial bound check above. A nonempty needle searches candidate positions in increasing order and constructs the first start/inclusive-end pair; failure returns one nil. Literal search does not interpret captures or any other pattern syntax.

Pattern results. Failure returns one nil. On success, `find` builds a result sequence beginning with the match's start and inclusive end. If there are captures, it appends their values in opening order: substring captures are Strings and position captures are Numbers. With no captures, it builds only the two positions. The result sequence is then expanded into return values.

Shared arguments and errors. Supply Strings for `source` and `pattern`. Missing fixed parameters are nil and surplus arguments are ignored. Any failure is reported by the length, character, substring or other operation reached; the pattern interface does not coerce supplied values into text.

Patterns are interpreted as matching reaches them, not validated in a separate pass. Reached syntax errors explicitly raise `invalid string pattern` for an unterminated bracket set, `%f` without its immediately following `[`, or a closing capture with no unfinished capture to close. A reached backreference to a missing, unfinished or position capture raises `invalid string pattern capture`. A terminal `%`, incomplete `%b` delimiters and unfinished opening captures do not have dedicated validation errors; they follow the corresponding operations. See Pattern language for the corresponding parsing rules.

Pattern operations save their installation-time `byte`, `sub`, `concat`, `unpack`, `tonumber`, `tostring` and `type` functions. Their explicit errors use current global `error`. Changing a helper's public binding leaves the saved reference unchanged, while any globals used by that helper remain live.

Pattern search can backtrack and try multiple candidate positions. Codepoint reads scan from the beginning rather than providing constant-time random access. Pattern size, nesting and input therefore affect work; there is no pattern-local work-step or recursion-depth limit.

10.7.2 `string.match`

```
string.match(source, pattern [, start])
```

Searches for the first pattern match, using the codepoint positions, start normalization, binding dependencies and reached-error rules described under `string.find`. See Pattern language for pattern syntax.

The default start is 1. A nonnil start is processed with `// 1`, negative positions are relative to the end, zero becomes 1, and an excessively negative result is not clamped. Unlike `find`, `match` does not first reject a start greater than `#source`; its search considers candidates from the supplied start through `#source + 1`. A leading `^` restricts it to the supplied start.

Failure returns one nil. On success, `match` builds a result sequence containing the whole matched substring if there are no captures, or the capture values in opening order otherwise. Substring captures are Strings; position captures are Numbers. It expands that sequence into return values, without adding start/end positions.

There is no plain-search argument. Surplus arguments are ignored, rather than selecting a different search mode.

10.7.3 `string.gmatch`

```
string.gmatch(source, pattern)
```

Returns one iterator function. The outer call captures `source` and `pattern` and initializes its search position to 1; it does not inspect either input or parse the pattern before returning. It does not return separate state and control values. Surplus outer-call arguments are ignored.

Call the returned function with no arguments; any supplied arguments are ignored. Each call searches from its current codepoint position. Success constructs and expands the same result sequence as `string.match`: the whole match when there are no captures, otherwise capture values in opening order. Failure returns one nil.

After a nonempty match, the next search starts at the match's exclusive end. After an empty match, it starts one position beyond that exclusive end. The search position changes before capture extraction and return-value construction, so an error during those operations can occur after the iterator has advanced. A failed search leaves the position unchanged; there is no latched finished state, and another call repeats that failed search.

A leading `^` applies at each current search position, not just at the absolute beginning of `source`. The iterator does not search for overlapping nonempty matches inside a span it has already consumed. See Pattern language for the matching rules and `string.find` for shared bindings and reached errors; those errors are deferred until an iterator call reaches the relevant operation.

One iterator call performs one complete search, which can examine all remaining candidate positions and backtrack.

10.7.4 `string.gsub`

```
string.gsub(source, pattern, replacement [, limit])
```

Builds a substituted String and returns it together with the substitution count. Positions and substring boundaries are codepoint-based. See Pattern language for pattern syntax and `string.find` for shared binding dependencies and reached pattern errors.

Limit and count. Omitted or nil `limit` means `#source + 1`. Otherwise the function computes `limit // 1` and replaces a negative result with 0. A zero limit skips matching and replacement entirely, assembles the unchanged source tail, and returns that String with count 0. A nonnil non-Number limit does not parse as a numeric String or raise a `gsub`-specific integer error: its arithmetic result fails the loop comparison, so the replacement loop is likewise skipped.

The count increases after a successful replacement evaluation, including an evaluation that chooses to keep the original match. The limit bounds completed replacement iterations, not the work spent finding a match, backtracking, looking up a replacement or running a replacement function.

Replacement arguments. For each match, the function constructs a replacement-argument sequence: the whole matched substring when there are no explicit captures, otherwise capture values in opening order. Substring captures are Strings and position captures are Numbers. The whole match remains separately available to the replacement rules below.

- **String or Number:** first convert the replacement with the retained `tostring`, then parse its text. Ordinary codepoints copy literally. `%%` emits `%`; `%0` emits the whole match. For another codepoint following `%`, the retained `tonumber` is used: a numeric result selects that numbered replacement-argument entry, and a missing entry contributes an empty String; a nonnumeric token is emitted without the `%`. A final lone `%` is copied literally. Consequently `%1` refers to the whole match when there are no captures. Numeric position-capture values reach the output conversion as Numbers.
- **Table:** use the first replacement argument as the key when the argument sequence has entries, otherwise use the whole match. This is an ordinary table lookup. Its metatable lookup follows Table-valued `__index` chains; it does not call a Function-valued `__index`. A nil result keeps the whole match. Every other result, including false, is converted with the retained `tostring`. The returned text is not reparsed for `%` substitutions.
- **Function:** pass the replacement arguments separately when their sequence has entries, otherwise pass the whole match. Only the first return value is used. Nil or false keeps the whole match; any truthy value is converted with the retained `tostring`. The returned text is not reparsed for `%` substitutions.

A replacement of any other kind raises exactly `string.gsub replacement must be a string, number, function, or table.` only when a match reaches replacement evaluation. A zero limit or an unsuccessful search does not prevalidate the replacement.

Search advancement and assembly. Searching begins at position 1. For each match, the function appends the unmatched prefix and the evaluated replacement. A nonempty match moves both the next search position and the beginning of the remaining source tail to the match's exclusive end; it does not search for overlaps inside the consumed span.

For an empty match, after replacement evaluation the function first checks the *previous* search position. If that position is already greater than `#source`, it stops. Otherwise it sets both the next search position and the remaining-tail position to one position beyond the empty match's exclusive end. This advances past a source position without appending that position's codepoint. An empty match immediately after a nonempty match is not automatically suppressed.

After substitution stops, the remaining source tail is appended and the output pieces are concatenated. A leading `^` is applied at every current search position, not as a one-time absolute-source-beginning restriction.

A replacement function can have observable effects before a later match or operation fails. Those effects are not rolled back. If a reached operation raises an error, `gsub` does not return a partial output or count. Its private count advances only after replacement evaluation returns. The substitution limit does not bound output size or matching work.

10.8 Floating-point word decoder

The named BIOS import `string/float/decode` binds a static call target, not a runtime library Table or a first-class function value:

```
local decode_float<const> = require('string/float/decode')
local sample = decode_float(0x3fa00000, 0, 4)        -- 1.25
local calibration = decode_float(0xc0040000, 0, 8)  -- -2.5
```

Its signature is `decode_float(high, low, byte_count)`, with one Number result. Pass the raw unsigned words after reading them in the record's byte order:

Argument	Four-byte form	Eight-byte form
`high`	Complete 32-bit field	High 32 bits, containing sign and exponent
`low`	Unused; conventionally 0	Low 32 bits
`byte_count`	4	8

Exactly `byte_count == 4` selects the four-byte form; every other value selects the eight-byte calculation. There are no separate word-range or argument-type checks. Both forms use the field extraction and Number arithmetic under Decoding, without reading a String or mapped memory. Surplus arguments are ignored. This import does not supply a separate public encoder.

10.9 String packet packing

`string.pack`, `string.packsize`, and `string.unpack` construct and read formatted packets held in a BLua String. The format determines field widths, signedness, byte order, and padding.

```
packet = string.pack(format, ...)
size = string.packsize(format)
v1, v2, ..., next_pos = string.unpack(format, packet [, pos])
```

- `pack` returns one String. Each integer, floating-point, or text field consumes one argument, including a zero-width `c0` field. Controls and padding consume none. Extra arguments are ignored.
- `packsize` returns one Number: the size of a fixed-size format, including its padding. A format containing `z` or `s` is variable-length and causes an error.
- `unpack` returns one value for each integer, floating-point, or text field, followed by the next one-based source position. Controls and padding produce no values. An empty or control-only format normally returns only that position.

Shared Table dependency. Temporary packet pieces and unpacked-result collections use the CPU's maintained Table lengths, including when every supplied format and operand is a String. Accepted Table operands are retained in the argument-category account below. Ordinary caller result adjustment still applies.

For example, these calls construct and read a fixed-width record without transferring it to mapped memory:

```
local format = "<I2c4"
local packet = string.pack(format, 513, "BMSX")
local size = string.packsize(format)
local kind, tag, next_pos = string.unpack(format, packet)
```

The packet contains codepoints 1, 2, 66, 77, 83, 88. Its size is 6; unpacking returns 513, 'BMSX', then next position 7.

10.9.1 Packet units are String codepoints

BLua String lengths and positions count Unicode codepoints, not UTF-8 bytes. This also applies to packing formats and packets. In this section, a **packet unit** means one String codepoint.

Numeric fields and padding written by `pack` contain codepoints in the range 0–255. Text fields copy their source codepoints without imposing that range. Consequently, a packed String is not necessarily a sequence of octets, and it is not a mapped byte buffer or an address.

`unpack` reads numeric digits as codepoint values. It neither checks that a digit is at most 255 nor masks it to eight bits. A text field containing a larger codepoint can therefore affect a later numeric interpretation.

For example, `string.pack("c1", string.char(256))` copies codepoint 256 into one packet unit. Reading that one-unit String with `B` accumulates the Number 256; reading it with `b` applies the signed subtraction once and obtains 0. In either calculation the next position is 2. These are consequences of the codepoint representation, not octet-storage behavior.

The returned packet is a String value. Packing does not allocate mapped storage or copy packet units to a byte address.

10.9.2 Example: a packed record

The format below combines a fixed header, a length-prefixed payload and a zero-terminated status. Maximum alignment 4 places the eight-unit `d` field at offset 12 rather than 16. The payload's zero codepoint is content in `s1`; using `z` for that payload would raise an error.

```
local header_format = '< !4 B H > h < f d c3'
local payload = 'A' .. string.char(0) .. 'B'
local record = string.pack(header_format .. ' s1 z',
    1, 4660, -300, 1.25, -2.5, 'POSITION', payload, 'ready')
local header_size = string.packsize(header_format)   -- 23
local record_length = #record                        -- 33
local version, flags, delta, sample, calibration, tag, next_pos =
    string.unpack(header_format, record)
local decoded_payload, status, after_record =
    string.unpack('s1 z', record, next_pos)
```

The header returns `1`, `4660`, `-300`, `1.25`, `-2.5`, `'POS'`, then next position 24. The second call returns the three-codepoint payload, `'ready'`, then position 34. These are one-based continuation positions, not lengths.

Zero-based offset	Field	Packet codepoints
0	`B` version	1
1	Alignment padding	0
2–3	Little-endian `H` flags	52, 18: value 4660
4–5	Big-endian `h` delta	254, 212: value −300
6–7	Alignment padding	0, 0
8–11	Little-endian `f` sample	0, 0, 160, 63: value 1.25
12–19	Little-endian `d` calibration	0, 0, 0, 0, 0, 0, 4, 192: value −2.5
20–22	`c3` tag	80, 79, 83: `POS`, truncated from `POSITION`
23	`s1` payload prefix	3
24–26	Payload	65, 0, 66
27–32	`z` status	114, 101, 97, 100, 121, 0: `ready` and terminator

10.9.3 Formats

Each call begins in little-endian order with maximum alignment 8. `pack` and `packsize` start at zero-based offset 0. `unpack` begins at the source offset selected by `pos`.

Only the following whitespace is ignored between format tokens: space and codepoints 9–13. A numeric suffix consists of consecutive ASCII decimal digits. There is no sign, decimal point, separator, or internal whitespace in a suffix. The suffix reader has no general digit-count or numeric upper bound; the individual options below impose their own restrictions.

In the tables, `A` is the current maximum alignment, and all widths and alignments count packet units. Defaults are fixed, not inferred from a machine word size.

Option	Field	Width	Alignment	Arguments / data results
`b`	Signed integer	1	`min(A, 1)`	1 / 1
`B`	Unsigned integer	1	`min(A, 1)`	1 / 1
`h`	Signed integer	2	`min(A, 2)`	1 / 1
`H`	Unsigned integer	2	`min(A, 2)`	1 / 1
`l`	Signed integer	4	`min(A, 4)`	1 / 1
`L`	Unsigned integer	4	`min(A, 4)`	1 / 1
`j`	Signed integer	8	`min(A, 8)`	1 / 1
`J`	Unsigned integer	8	`min(A, 8)`	1 / 1
`i[N]`	Signed integer	`N`, default 4; explicit `N` must be 1–8	`min(A, N)`	1 / 1

Option	Field	Width	Alignment	Arguments / data results
`I[N]`	Unsigned integer	`N`, default 4; explicit `N` must be 1–8	`min(A, N)`	1 / 1
`T`	Unsigned integer	4	**4, regardless of `A`**	1 / 1
`f`	Floating-point field	4	`min(A, 4)`	1 / 1
`d`	Floating-point field	8	`min(A, 8)`	1 / 1
`n`	Floating-point field	8	`min(A, 8)`	1 / 1
`cN`	Fixed-width text	`N`; suffix required, zero permitted	1; no automatic padding	1 / 1
`z`	Zero-terminated text	Content length + 1	1	1 / 1
`s[N]`	Length-prefixed text	Prefix width + content length; prefix defaults to 4, explicit width 1–8	`min(A, prefix width)` before the prefix	1 / 1
`x`	Padding	1	No additional alignment	0 / 0
`X`	Align for a following field	Padding only; see below	Determined by lookahead	0 / 0

`[N]` marks an optional suffix in this table; the square brackets are not format characters. The `f`, `d`, `n`, and `z` options do not read a width suffix. `cN` has no parser-imposed maximum width.

Control	Effect
`<`	Select little-endian numeric order.
`>`	Select big-endian numeric order.
`=`	Select the fixed default: little-endian order.
`!N`	Set maximum alignment to positive decimal `N`. The suffix is required. It need not be a power of two; there is no upper-bound check in this control.

Controls consume neither packet units nor arguments and return no data values. Every other option is unsupported. Parsing and field processing are interleaved; there is no separate up-front format-validation pass.

10.9.3.1 Alignment and padding

Before an aligned field, padding is calculated from the current zero-based offset:

```
padding = 0                                      when alignment <= 1
padding = (alignment - offset % alignment) % alignment   otherwise
```

Offsets include all earlier fields and padding. In `unpack`, alignment uses the absolute source offset, including the starting position; it does not begin a new alignment origin at the selected subrecord. No call adds implicit final record padding.

`pack` writes zero codepoints for all padding. `unpack` checks that skipped padding fits within the source but does not inspect its contents. Padding need not contain zero to be accepted.

`X` looks ahead to find an alignment without consuming the following field. Its lookahead skips `x`, further `X` options, and endian/alignment controls, while tracking the controls' future state locally. The main parser still processes all those options later. The chosen alignment is:

- an integer field's alignment, including the fixed alignment 4 for `T`;
- a floating-point field's alignment;
- an `s` field's prefix alignment;
- the full width `N` for `cN`, **not** 1 and not a width limited by `A`;
- 1 for `z` or the end of the format.

For example, `string.packsize("!1bT")` is 8, and `string.packsize("bXc3")` is 6. A trailing `X` adds no padding.

10.9.4 Integer fields

Integer fields use ordinary BLua Numbers, not a separate exact 64-bit integer type.

For packing, the value is first divided by 1 with `//`. If the result differs from the original value, the call reports an integer-value error. Fractional Numbers are therefore rejected, not rounded into valid integer arguments. Numeric Strings, Booleans, Tables, nil, and a missing argument are not coerced into integers. Numeric NaN fails the same test. Positive and negative infinity reach the safe-range error.

The value must then lie within the safe integer interval:

```
-9007199254740991 <= value <= 9007199254740991
```

It must also fit the selected field width `w`:

Field	Accepted width range before the safe-integer restriction
Signed	`-(256^w)/2` through `(256^w)/2 - 1`
Unsigned	`0` through `256^w - 1`

A negative signed value is encoded by adding `256^w` once. Exactly `w` base-256 digits are then written in the selected order. Little-endian writing takes `% 256` and repeatedly divides by 256 with `//`; big-endian writing uses descending powers of 256.

Seven- and eight-unit signed fields require particular care. The positive value after that addition can exceed the exact integer interval even when the accepted input does not. Do not assume exact 64-bit wrapping or a general signed seven-/eight-unit round trip.

On reading, digits are accumulated as an unsigned Number in the selected order. An accumulated value greater than 9007199254740991 is rejected **before** signed interpretation. Only after that check does a signed field subtract `256^w` once if its value is at least `(256^w)/2`. There is no additional lower-bound or digit-range check.

Thus an ordinary octet-valued seven- or eight-unit representation with its sign bit set is rejected before it can become a negative result: its unsigned value is at least `2^55` or `2^63`, respectively. For example, eight codepoints of value 255 read with `j` reach the safe-integer error rather than returning −1. Packing −1 with `j` is likewise not a promise of an exact all-255 representation: it first performs the Number addition `-1 + 2^64`.

10.9.5 Text fields

Text widths and lengths are codepoint counts. Embedded zero codepoints are ordinary String content except where `z` explicitly forbids or terminates them.

10.9.5.1 `cN`: fixed width

`pack` copies the first `N` codepoints. It truncates longer input without error and pads shorter input with zero codepoints. There is no automatic field alignment.

`c0` still consumes one argument but performs no input-length operation and writes no field data.

`unpack` returns exactly the selected `N`-unit substring, including zero codepoints, then advances by `N`. On a String source, `c0` returns an empty String.

10.9.5.2 `z`: zero terminated

`pack` scans the complete input. A zero codepoint anywhere in it causes `string.pack z strings must not contain zero bytes.` Otherwise it copies the content and appends one zero terminator. The error says “bytes”; the scan itself compares codepoints to zero.

`unpack` searches for the first zero codepoint, returns the preceding content, and advances past the terminator. If no terminator is found within the source, it reports the zero-terminated-string error.

10.9.5.3 `s[N]`: length prefixed

`pack` aligns the prefix, writes the content's codepoint count as an unsigned integer of the selected prefix width, then copies the entire content. The prefix uses the current endian order and the integer packing checks. Zero codepoints in the content are permitted.

`unpack` reads the unsigned prefix, checks that the indicated content span fits, returns exactly that span, and advances past both prefix and content. Zero codepoints are preserved.

`packsize` rejects both `z` and `s` immediately when it reaches them, regardless of any intended content.

10.9.6 Floating-point fields

`f` uses a four-unit field; `d` and `n` use eight-unit fields. These are specific Number-to-field transformations. Use the transformations below, including their rounding and subnormal rules, when constructing an interchange record.

The following equations specify the transformations, including their subnormal branch. `//` denotes BLua floor division.

10.9.6.1 Encoding

Pass a Number. `f` encodes IEEE 754 binary32; `d` and `n` encode IEEE 754 binary64. Finite values are rounded to the nearest representable value, with an exact halfway case selecting the even significand. Binary32 conversion can therefore lose precision or overflow even when the original Number is finite. Binary64 retains a finite Number's representation.

Subnormal results use gradual underflow. Positive and negative zero retain their signs, including when a small value rounds to zero. Infinities retain their signs. Number NaNs are written with exponent bits all set and fraction 1; their original payload and sign are not preserved.

Input Number	Binary32 word	Binary64 high word	Binary64 low word
`0`	`00000000H`	`00000000H`	`00000000H`
`-0.0`	`80000000H`	`80000000H`	`00000000H`
`1.25`	`3FA00000H`	`3FF40000H`	`00000000H`
`2^-149`	`00000001H`	`36A00000H`	`00000000H`
`2^-1074`	`00000000H`	`00000000H`	`00000001H`
Positive infinity	`7F800000H`	`7FF00000H`	`00000000H`
NaN	`7F800001H`	`7FF00000H`	`00000001H`

A binary32 field writes its word in the selected byte order. A little-endian binary64 field writes the low word first, then the high word, with each word little-endian. A big-endian binary64 field writes the high word first, then the low word, with each word big-endian. Each output octet is held as one String codepoint.

For decoding below, the field parameters are:

Field	Exponent bits `E`	Mantissa bits `M`	Bias `B = 2^(E-1)-1`	Scale `S = 2^M`
`f`	8	23	127	`2^23`
`d`, `n`	11	52	1023	`2^52`

Let `emax = 2^E - 1`. These are field values, not additional format options. The encoding operation does not parse numeric Strings or perform the integer field's argument checks. Supply Number values rather than using packing as a general value conversion.

10.9.6.2 Decoding

`f` reads one four-unit word as `high` and supplies `low = 0`. `d` and `n` read the two four-unit words in the inverse order of encoding.

The sign multiplier is −1 when `high >= 2^31`, otherwise 1. Extract the fields as follows:

```
f:
    e = (high // 2^23) % 2^8
    m = high % 2^23

d, n:
    e = (high // 2^20) % 2^11
    m = (high % 2^20) * 2^32 + low
```

Using that field's `B`, `S`, and `emax`:

```
e == emax and m == 0:  sign / 0
e == emax and m ~= 0:  0 / 0
e == 0:                sign * (m / S) * 2^(1 - B)
otherwise:             sign * (1 + m / S) * 2^(e - B)
```

Source digits are not validated as octets. NaN payloads are reduced to a Number NaN by the arithmetic above.

10.9.7 Unpack positions, bounds, and results

`pos` defaults to 1 when absent or nil. Otherwise `pos // 1` supplies the starting position; the zero-based offset is that result minus 1. Fractional finite Numbers therefore round down here. Negative positions are **not** relative to the source's end.

For an ordinary numeric starting position, the accepted interval is 1 through `#packet + 1`, inclusive. The final position can serve an empty/control-only or zero-width format, but cannot supply packet data. A position outside the interval causes `string.unpack start index out of range.`

Alignment remains relative to the absolute source offset. A packet packed at offset 0 is therefore not automatically aligned the same way when unpacked from an arbitrary interior position.

The position argument has no Number-category or integer check. A nonnumeric value or NaN becomes NaN through `//`, evades the range comparisons, and is not repaired. For example, an empty format and String source with a numeric String position produce NaN as the only next-position result. Either infinity fails the explicit range check.

Before reading or skipping `x`, `X`, integer, float, fixed-width text, length-prefix, or prefixed-content spans, `unpack` checks whether the needed span extends beyond the source. Such a failure reports `string.unpack string is too short.` The `z` scan instead has its own missing-terminator error. Earlier processing is not rolled back when a later field fails.

Successful unpacking need not consume the whole String: trailing content is allowed. The final result is the **next one-based codepoint position**, not the number of units consumed. It can be passed as the next call's `pos`, provided the desired absolute-offset alignment is retained:

```
local first, next_pos = string.unpack("<I2", packet)
local tag, after_tag = string.unpack("c4", packet, next_pos)
```

10.9.8 Argument categories and library dependencies

Use Strings for formats, text fields and input records. Length and substring operations still obey their general language rules: `#` accepts Tables as well as Strings, while `string.byte` requires a String. As a result, these edge cases follow the operations selected by the format:

- A format Table whose length is zero bypasses token reads; a positive-length format Table reaches `string.byte` and fails its String requirement.
- `c0` can consume even an argument with no supported length operation, because it never requests that length.
- A Table input with length zero can supply padding to `cN`, an empty payload to `s`, or only the terminator to `z`.
- A substring covering an operand's entire span can return the original Table. Later concatenation then uses the current global `tostring` for it rather than necessarily producing an early String-category error.
- A Table source can pass through control/padding-only unpacking without byte access. A full-span `cN` substring can likewise return the Table itself.

These branches use the CPU’s maintained-length rule.

Packing keeps its original character, substring, Table-concatenation, Table-unpack and floating-word conversion functions. Argument selection reads current `select` and reported errors call current `error`. A saved helper continues to use any current globals required by that helper, including `tostring` during singleton concatenation. Successful unpacking always includes the next-position result, even for a format with no data fields.

10.9.9 Errors

The packing module directly reports the following messages, with the capitalization and punctuation shown. The shared format parser also uses `string.pack`-prefixed messages during `packsize` and `unpack`.

Condition	Message
Integer input fails the integer-value test	`string.pack integer value must be an integer.`
Integer input exceeds the safe Number integer interval	`string.pack integer value exceeds safe integer range.`
Integer input does not fit its field width	`string.pack integer value out of range.`
Accumulated unsigned input exceeds the safe integer interval	`string.unpack integer exceeds safe integer range.`
Missing or nonpositive `!N` suffix	`string.pack alignment must be a positive integer.`
Explicit `i`/`I` width outside 1–8	`string.pack invalid integer size.`
Missing `c` width	`string.pack expected a size for c format.`
Explicit `s` prefix width outside 1–8	`string.pack invalid length size.`
Unsupported format option	`string.pack unsupported format option.`
Zero codepoint in a packed `z` input	`string.pack z strings must not contain zero bytes.`
`z` or `s` encountered by `packsize`	`string.packsize format is variable-length.`
Starting offset outside the source bounds	`string.unpack start index out of range.`
Required fixed or prefixed span does not fit	`string.unpack string is too short.`
No zero terminator found for `z`	`string.unpack zero-terminated string not found.`

Length, character and substring failures retain their ordinary errors. Formats are processed alongside their fields, including `X` lookahead, so a later error can follow earlier reads, conversions and argument selections.

10.9.10 Work and interface boundaries

Packet size and field kind determine the processing and temporary language storage required. Character access scans from the String’s beginning; integer fields convert their digits and floating fields perform word conversion. Large fixed-width text fields construct correspondingly large Strings.

Each `string.byte` or `string.char` invocation adds its two-cycle primitive surcharge, in addition to the executed BIOS instructions and ordinary calls. Use the CPU timing rules for the complete sequence.

11 BIOS table library

Use the BIOS `table` functions to keep argument tuples, edit sequences, join values into text and sort a numeric-keyed range. The six functions operate on ordinary Tables. Their sequence positions begin at key 1.

Typed arrays provide the separate byte-addressed representation used for memory buffers and device transfers. Choose that representation when fixed element widths, contiguous bytes or a device address are required.

Interface	Purpose	Successful results
`table.pack(...)`	Preserve values and their supplied count	One new Table
`table.insert(target, value)`	Append at the maintained length plus one	One nil
`table.insert(target, position, value)`	Shift and insert at a numeric key	One nil
`table.remove(target [, position])`	Retain one value, shift and clear the old tail	One value, possibly nil
`table.concat(target [, separator [, start [, end]]])`	Join an inclusive numeric-key interval	One String
`table.unpack(target [, start [, end]])`	Expand an inclusive numeric-key interval	Selected values, or zero results
`table.sort(target [, compare])`	Reorder the maintained sequence in place	One nil

These result counts are part of the interface. An ordinary bare return or fallthrough produces one nil; that is not the zero-result tuple returned by an empty `unpack` interval. BIOS base services explains `select` and result adjustment. Protected calls and coroutines explains interception of an error value, including nil, without undoing earlier writes.

`table.pack` creates a Table containing the supplied values, not deep copies of them. A packed Table, closure or Thread remains the same referenced value. Tables and their retained values use the shared language allocation budget; they do not reserve a mapped byte buffer suitable for a device transfer.

11.1 Tables, lengths and visible effects

Sequence keys start at one. Zero and negative numeric keys remain ordinary Table keys; the insertion and removal routines do not convert them into positions counted from the end. The relative-endpoint rule belongs only to `concat` and `unpack`.

In the following procedures, **L** means the Table's maintained length, `#target`, captured by that call. It is not the number of all mappings, the greatest numeric key, a count of metatable-supplied entries, or a tuple's `n` field. CPU — Stored length defines the maintained length and Table operations. A count stored by the cartridge is a separate value; use the explicit `pack`/`unpack` recipe when nil positions must be preserved.

Element reads are ordinary Table indexing: a missing own mapping can be supplied by a Table-valued `__index` chain. Writes go directly to the target; they do not redirect through `__newindex`. Assigning nil removes the target's own mapping; it does not mask a mapping inherited through `__index`. A read can therefore obtain an inherited value that a later write copies into an own mapping. The length snapshot does not count those inherited values.

Pass a Table target. Length and element access follow ordinary language rules; any failure is reported when the corresponding operation is reached. For example, a String has a codepoint length and uses the installed String-index Table for reads; assigning to it raises the shared assignment error. Use String operations when character access is intended.

Insert, remove and sort write the caller's Table directly, in the stated order. There is no deferred commit or restoration on error. Pack creates a new Table; concat and unpack do not themselves assign into the target. A callback or a deliberately replaced public dependency can still have its own effects. The library's public fields and the `select`/`tostring` globals it uses are ordinary mutable bindings, not permanently reserved operations.

The functions execute ordinary instructions. Loops, tuple forwarding, conversions, temporary Strings and comparison callbacks contribute work; a function call is not a single constant-cycle hardware operation. The CPU reference supplies instruction charges.

11.2 Packing and reading table ranges

11.2.1 `table.pack`

```
table.pack(...) -> packed
```

`table.pack` returns exactly one new ordinary Table. Its `n` field records the number of supplied arguments, including interior and trailing `nil` arguments. Arguments occupy their corresponding integer positions starting at 1; a `nil` argument leaves no mapping at that position.

```
local packed = table.pack(10, nil, 30, nil)
-- packed.n is 4; positions 2 and 4 read as nil.
local empty = table.pack()
-- empty.n is 0.
```

The `n` field is an ordinary, mutable String-keyed field, not a special length property. `table.insert`, `table.remove`, `table.sort`, `table.concat`, and `table.unpack` neither maintain it nor automatically use it. To read the entire packed argument list, supply `packed.n` explicitly as the end bound.

Packing uses the current global `select`, rather than a captured copy. The argument-count behavior described here assumes the installed `select`.

11.2.2 Selecting a range

`table.unpack` and `table.concat` read `#target` at the start of every call, even when both bounds are explicit. The resulting value is `L`. An omitted or explicitly `nil` start bound means 1; an omitted or explicitly `nil` end bound means `L`. Neither function discovers a packed Table's `n` field.

For each explicit, non-`nil` bound, first compute `q = bound // 1`. This is floor division, not truncation toward zero. Normalize that value as follows:

Value of `q`	Start bound	End bound
Positive	`q`	`q`
Negative	`L + q + 1`	`L + q + 1`
Zero or NaN	`1`	`L`

For example, when `L` is 4:

Supplied start, end	Selected start, end
`-3, -1`	`2, 4`
`0, 0`	`1, 4`
`1.9, 2.9`	`1, 2`

A non-`nil`, nonnumber bound, including a numeric-looking String, produces NaN in the floor division; it is not parsed as a number. NaN is neither positive nor negative, so it normalizes like zero: to 1 for a start bound or `L` for an end bound. There is no table-library argument error for this case.

Bounds are not clamped to the default interval. The selected interval is inclusive and is empty when the normalized end precedes the normalized start. No finite-value or range-size cap is applied. Very large or infinite endpoints have no termination or resource-use guarantee.

11.2.3 table.unpack

```
table.unpack(target [, start [, end]]) -> ...
```

`table.unpack` reads the selected positions in ascending order. A completed call returns one result for every selected position, including positions that read as `nil`. A single-position interval therefore returns one result even when that result is `nil`. An empty interval returns zero results, not one `nil`.

With `packed` from the earlier example, these calls distinguish the result counts:

```
select('#', table.unpack(packed, 1, packed.n)) -- 4
select('#', table.unpack(packed, 2, 2))        -- 1
select('#', table.unpack(packed, 3, 2))        -- 0
```

Keep the call in a multiple-result context when every result matters, as in the final argument to `select` above. A fixed-size assignment discards excess results and fills missing values with `nil`; it does not preserve a result count on its own.

The reads use ordinary indexing. An explicit positive interval can therefore read metatable-supplied mappings beyond `L`, but inherited entries do not extend the default interval.

Unpacking uses a recursive call chain whose depth grows linearly with the number of selected positions, returning successively wider result lists. It has no result-count or recursion cap of its own. Empty-range handling calls the current global `select` with argument 1; its zero-result behavior assumes the installed `select`.

11.2.4 table.concat

```
table.concat(target [, separator [, start [, end]]]) -> text
```

`table.concat` returns exactly one String, joining the selected positions in ascending order without changing the target. Each position is read by ordinary indexing. A `nil` value contributes an empty String; every other value is converted with the current global `tostring`. Separators remain between adjacent positions even when one or both positions read as `nil`.

An omitted or explicitly `nil` separator is the empty String. Any other separator is converted with `tostring`, so the installed `tostring` accepts values other than Strings here as well. This conversion happens before the selected values are read, even for a single-position interval where no separator is inserted. An empty interval instead returns an empty String before converting the separator or reading any positions.

```
table.concat(packed, '|', 1, packed.n) -- "10||30|"
table.concat({true, {}, nil, 7}, false, 1, 4)
-- "truefalsetablefalsefalse7" with the installed tostring.
```

These conversions use the live global `tostring`, not a permanently captured formatter. Replacing it can change both separator and value conversion.

Concatenation divides the selected interval at `(first + last) // 2`, joins the two halves and inserts the separator between them. It creates intermediate Strings as well as the result. Keep explicit endpoints in an integer range where adding and bisecting the bounds remains exact; extreme Number bounds can prevent the recursive interval from shrinking.

11.3 table.insert

Call	Effect	Successful return
`table.insert(target, value)`	Assigns `value` to `target[#target + 1]`.	One `nil` value.

Call	Effect	Successful return
`table.insert(target, position, value, ...)`	Inserts at `position // 1`, shifting the selected keys upward as described below. Arguments after `value` are ignored.	One `nil` value.

Exactly one argument after `target` selects the append form. A lone explicit `nil` therefore means "append `nil`," not "position omitted." Every other trailing argument count selects the positional form. The signatures above require at least one argument after `target`. An indirect call such as `pcall(table.insert, target)` nevertheless reaches the positional form with no trailing arguments; both selected arguments are then `nil`, so the position arithmetic produces NaN. With a separate value, an explicit `nil` position is still passed to `// 1`.

Use a finite numeric position. This arithmetic does not parse numeric Strings, and a nonnumeric position produces NaN rather than an upfront library argument error. With a NaN position, the descending shift loop has no comparison-driven exit: it can make earlier shift writes without reaching the final insertion. A reached fault or other interruption is separate from normal completion.

For a positional insertion with a finite numeric position:

1. Compute `p = position // 1`, then take the length snapshot `L`.
2. For `i = L, L - 1, …, p`, in that descending order, assign `target[i + 1] = target[i]`.
3. Assign `target[p] = value`.

The position is not checked or clamped. Zero and negative positions name those actual keys; they do not count backward from the end. If `p > L`, no keys are shifted and the only assignment is at `p`. A fractional position is rounded downward by `// 1`, so `5.9` selects key `5`.

For example, starting with the plain table `{10, 20}` and no key `0`:

```
local values = {10, 20}
table.insert(values, 0, 7)
-- values[0] is 7; values[1] is nil;
-- values[2] is 10; values[3] is 20.
```

An insertion beyond the selected prefix leaves a gap rather than filling it:

```
local values = {10, 20}
table.insert(values, 5.9, 99)
-- values[1] is 10; values[2] is 20; values[5] is 99.
-- Keys 3 and 4 are absent.
```

Use the keys actually written when reasoning about these examples; insertion does not promise a universal resulting-length formula. Assigning `nil`, either directly or while shifting, deletes the destination mapping. `table.insert` does not update a field named `n` and does not return a length or the modified table.

11.4 `table.remove`

Call	Selected position	Successful return
`table.remove(target)`	`L`.	Exactly one value: the value read at the selected position, possibly `nil`.
`table.remove(target, nil)`	`L`.	The same as above.
`table.remove(target, position)`	`position // 1`.	The same as above.

Removal first takes the length snapshot `L`. It then selects `p` using the table above. For a finite selected position, it performs these operations in order:

1. Read and retain `target[p]` as the result.

2. For `i = p, p + 1, …, L - 1`, in that ascending order, assign `target[i] = target[i + 1]`.
3. Always assign `nil` to `target[L]`.
4. Return the retained result.

Use a finite numeric explicit position. A nonnil nonnumeric position produces NaN without an upfront argument error. The shift then repeatedly reads and writes the NaN key; its comparison does not terminate the loop. It does not reach the final tail clear or returned result by normal loop termination. A reached fault or other interruption remains possible. Positions are neither checked nor clamped. Zero and negative positions refer to actual keys, not positions relative to the end. They can therefore cause shifting through zero or negative keys. A position above `L` skips the shifting but **still clears key `L`**:

```lua
local values = {10, 20, 30}
local removed = table.remove(values, 99)
-- removed is nil.
-- values[1] is 10; values[2] is 20; key 3 is absent.
```

The default position is also used when `L` is zero. For a table whose length snapshot is zero and whose key `0` contains `7`, `table.remove(target)` returns `7` and deletes key `0`.

Throughout these functions, element reads are ordinary table reads, including a table-valued `__index` where applicable. Assignments write directly to the target and do not call `__newindex`. Copying a `nil` deletes the destination mapping. Removal leaves a field named `n` unchanged. If a later operation fails, earlier assignments remain in effect; these functions do not undo them.

11.5 `table.sort`

Call	Ordering	Successful return
`table.sort(target)`	Ordinary `<`.	One `nil` value.
`table.sort(target, nil)`	Ordinary `<`.	One `nil` value.
`table.sort(target, compare)`	`compare(left, right)` is truthy when `left` belongs before `right`.	One `nil` value.

Sorting modifies `target` in place. It takes the length snapshot `L` first and operates on keys `1` through `L`; it does not make or return a sorted copy. If `L < 2`, it immediately returns one `nil` value without performing a comparison or checking whether `compare` can be called. Thus `table.sort({1}, false)` returns normally, whereas a nontrivial sort with `false` as its comparator attempts to call `false` and raises the ordinary non-function call error.

The sort begins by scanning from index `2` through `L`. At each index it asks whether the current value belongs before its predecessor. Falsey answers continue the scan. The first truthy answer starts a quicksort of the **entire** range `1` through `L`, not just the remaining suffix. If the scan reaches the end, the sort itself makes no assignments; with a valid custom comparator this ordered-input path makes exactly `L - 1` comparator calls.

For each partition from `first` through `last`, the quicksort retains `target[(first + last) // 2]` as its pivot and scans inward. Each swap saves the left value, writes the right value into the left key, then writes the saved value into the right key; the two writes are sequential, not atomic. Do not depend on the order of equal elements: this sort has no stability guarantee. Apart from the ordered-input scan described above, neither a particular comparison schedule nor a particular callback count is promised.

11.5.1 Choosing an ordering

With no comparator, ordinary `<` compares numbers numerically with numbers and strings lexically with strings. For other combinations it yields `false`; it does not raise a comparison error or convert strings to numbers. Do not use the default ordering to establish an order between numbers and strings, or between other categories of values.

A custom comparator receives two values. Only its first returned value is used. `nil` and `false` mean “does not belong before”; every other value, including `0` and `""`, means “belongs before.” Return a Boolean comparison, not a three-way numeric difference.

```
local values = {3, 1, 2}
table.sort(values, function(left, right)
    return left > right
end)
-- values now contains 3, 2, 1 at keys 1, 2, 3.
```

For records, compare the desired field:

```
local jobs = {{priority = 3}, {priority = 1}, {priority = 2}}
table.sort(jobs, function(left, right)
    return left.priority < right.priority
end)
-- The priorities at keys 1, 2, 3 are now 1, 2, 3.
```

The comparator must supply a consistent strict ordering: `compare(x, x)` must be falsey; if `compare(a, b)` is truthy, `compare(b, a)` must be falsey; and the ordering must be transitive. The sort does not validate these rules. An inconsistent comparator can make its scans pass outside the selected prefix, read `nil`, or fail to terminate.

Keep the target's entries and ordering unchanged inside the comparator. The sort retains its initial length and each current pivot across callback calls. A comparator error can be received through `pcall`, but swaps already performed remain visible. The interrupted-sort example shows how to distinguish that failure from successful completion.

11.5.2 Work performed

These functions run ordinary instruction sequences; they are not fixed-cost services. Appending performs one assignment. When the unit-step loop completes normally, positional insertion performs `L - p + 1` shift assignments when `p <= L`, otherwise none, followed by the final assignment at `p`. When its unit-step loop completes normally, removal performs `L - p` shift assignments when `p <= L - 1`, otherwise none, followed by its one unconditional clear at `L`.

For a consistent strict ordering, the sort's initial scan performs at most `L - 1` comparisons. Further work depends on the input and comparator. Do not budget a fixed cycle count or assume a general callback-count bound from these examples.

11.6 Errors and resource limits

Table functions report the errors of the operations they execute. Check sequence positions and comparator behavior at the application boundary; error handling receives a failure but leaves earlier Table changes visible.

Reached operation	Error text
Length of an unsupported value	`Attempted to get length of an unsupported value.`
Indexing a value with no indexing support	`Attempted to index field on a non-table value.`
Assignment to a non-Table	`Attempted to assign to a non-table value.`
Calling a nonfunction	`Attempted to call a non-function value.`
Exhausting the Table-valued index chain	`Metatable __index loop detected.`
Exhausting available heap storage	`Out of memory.`

These errors occur when the corresponding operation is reached. Mixed-category ordering and non-Number arithmetic follow the CPU rules given above. A comparator can also raise any language value of its own. Protected calls and coroutines defines the protected-call boundary; Book I, "System control and timing" defines hardware fault publication. Do not treat error interception as restoration of a modified Table.

Choose finite, usable endpoints and a consistent comparator. `unpack` expands a recursive result chain, `concat` creates intermediate Strings, and `pack` forwards its argument tuple while selecting each value. Budget for those language objects and calls as well as for the final Table or String.

11.7 Programming examples

11.7.1 Preserve the tuple count

Use the n field created by `table.pack` as the explicit end when unpacking a tuple that may contain nil. Table length is not a substitute for this count.

```
local packed = table.pack(10, nil, 30, nil)
local first, gap, third, tail = table.unpack(packed, 1, packed.n)
local count = select('#', table.unpack(packed, 1, packed.n))
local text = table.concat(packed, '|', 1, packed.n)
-- first = 10, gap = nil, third = 30, tail = nil.
-- packed.n and count are 4; text is '10||30|'.
```

11.7.2 Edit and read a sequence

The following sequence has consecutive numeric keys starting at one. Append, sort and the omitted range endpoints use its maintained length.

```
local sequence = {9, 3, 7}
table.insert(sequence, 2, 5.5)       -- 9, 5.5, 3, 7
local removed = table.remove(sequence, 3) -- 3; sequence is 9, 5.5, 7
table.insert(sequence, 8)           -- 9, 5.5, 7, 8
table.sort(sequence)               -- 5.5, 7, 8, 9
local first, second, third, fourth = table.unpack(sequence)
local suffix = table.concat(sequence, ':', -3, -1) -- '7:8:9'
local all = table.concat(sequence, ':', 0, 0)      -- '5.5:7:8:9'
local empty = table.concat(sequence, ':', 3, 2)    -- ''
local count = select('#', table.unpack(sequence, 3, 2)) -- 0
```

The observed length four belongs to this sequence. It does not define the length of a Table with holes or count a packed tuple's nil positions. A reversed range gives an empty String from concat but zero results from unpack.

Continue with a descending comparison and remove the final value:

```
table.sort(sequence, function(left, right) return left > right end)
-- sequence is now 9, 8, 7, 5.5.
local last = table.remove(sequence) -- 5.5; sequence is now 9, 8, 7.
```

11.7.3 Handle an interrupted sort

An error does not undo earlier swaps. This example preserves the error object's identity and shows the partial arrangement for one input and error point. Other inputs or error points need not produce the same arrangement or callback count.

```
local values = {3, 1, 2}
local failure = {code = 72}
local comparisons = 0
local ok, caught = pcall(table.sort, values, function(left, right)
    comparisons = comparisons + 1
    if comparisons == 5 then
        error(failure)
    end
    return left < right
end)
-- ok is false; caught is the same Table as failure.
-- values now has 1, 3, 2 at keys 1, 2, 3.
```